

Introduksjon til programmering i Python

Kompendium i BED-1304

Markus J. Aase

Universitetslektor i matematikk og statistikk

Høst 2026

Handelshøgskolen UiT

Litt informasjon

Dette kompendiet er skrevet som en praktisk introduksjon til programmering for økonomistudenter, samfunnsvitere og andre som ikke har bakgrunn fra informatikk. Selv om mange av eksemplene tar utgangspunkt i økonomiske problemstillinger, egner materialet seg like godt for biologer, psykologer, medisinere og alle andre som har behov for å strukturere, analysere og behandle data på en effektiv måte.

Men hvorfor bør man egentlig lære seg å programmere fra bunnen av i en tid hvor kunstig intelligens og store språkmodeller tilsynelatende kan skrive koden for oss? Svaret er at selv om KI har blitt en fantastisk og uvurderlig assistent, er den ikke ufeilbarlig. For å kunne bruke slike verktøy trygt og effektivt, må du selv sitte i førersetet. Du må ha evnen til å lese og forstå koden, oppdage feilene modellen gjør, og takle sammensatte problemstillinger som en maskin ikke klarer å bryte ned på egen hånd.

Kompendiet dekker en rekke ulike temaer, med intusjon, illustrasjoner og eksempler. Hvert tema avslutter med en case-oppgave. I tillegg, helt bakerst i kompendiet, finner dere enkel repetisjon, oppgaver og løsningsforslag.

Målet med kompendiet er derfor å *ufarliggjøre* koding og bygge en solid, anvendbar forståelse for hvordan Python kan brukes til alt fra enkle beregninger og databehandling, til symbolsk matematikk og komplekse simuleringer.

Vi kommer til å bruke vise til noen datakilder i dette kompendiumet, og data'en kan finnes via nettsiden til kurset: <https://uit-bed-1304-h26.github.io/index.html>

Innhold

1	Introduksjon	4
1.1	Hva kan Python brukes til?	4
1.2	Tre ”måter” å kjøre Python på	5
1.3	Installasjonsveiledning for lokal kjøring	5
1.4	Biblioteker og pakkehåndtering: Hva er pip?	6
1.5	Problemløsning og algoritmisk tenkning	7
1.6	Caseoppgave	10
2	Python Basics	12
2.1	Kommentarer i koden	12
2.2	Variabler og tilordning	12
2.3	Matematiske operatorer	13
2.4	Dat typer og sjekk av type	14
2.5	Typekonvertering (Type Casting)	15
2.6	Innlesing av data med <code>input()</code>	15
2.7	Utskriftsformatering med f-strenger og <code>round()</code>	16
2.8	Sammenlignings- og logiske operatorer	16
2.9	Caseoppgave	18
3	Funksjoner	21
3.1	Innebygde funksjoner i Python	21
3.2	Hva er en funksjon?	21
3.3	Å definere egne funksjoner (<code>def</code>)	22
3.4	Funksjonens anatomi: Parametere vs. Argumenter	23
3.5	Viktigheten av <code>return</code> vs. <code>print</code>	24
3.6	Korte, anonyme funksjoner (Lambda)	27
3.7	Caseoppgave	29
4	Datastrukturer og NumPy	31
4.1	Lister (Lists)	31
4.2	Tupler (Tuples)	32
4.3	Oppslagsverk (Dictionaries / Dicts)	33
4.4	NumPy - Numerisk matematikk	33
4.5	Caseoppgave	35
5	Pandas og databehandling	38
5.1	Anatomien til en DataFrame	38
5.2	Hvor bor dataene våre? (Filsystem og lagring)	38
5.3	Lese inn og inspisere data	39
5.4	Utvelgelse, filtrering og nye kolonner	39
5.5	Henting av data fra eksterne kilder (Databaser og API)	40
5.6	Caseoppgave	42

6	Logikk og beslutninger	45
6.1	Flytskjema: Slik tenker datamaskinen	45
6.2	Den enkle <code>if</code> -setningen og innrykk	50
6.3	Enten/eller: <code>if</code> og <code>else</code>	51
6.4	Flere alternativer: <code>elif</code>	51
6.5	Nøstet logikk (Kompliserte beslutninger)	52
6.6	Caseoppgave	60
7	Datavisualisering med Matplotlib	63
7.1	En god figur skal være selvforklarende	63
7.2	Linjeplot: Utvikling over tid (Økonomi)	63
7.3	Punktplot (Scatter): Sammenhenger mellom to variabler (Biologi)	65
7.4	Caseoppgave	67
8	Symbolsk matematikk med SymPy	69
8.1	Symboler og algebra	69
8.2	Løse ligninger og markedslikevekt	70
8.3	Kalkulus: Derivasjon og Integrasjon	71
8.4	Caseoppgave	73
9	Simulering og usikkerhet	75
9.1	Tilfeldighet med NumPy	75
9.2	Monte Carlo-simulering av aksjepriser	77
9.3	Caseoppgave	81
	Del II: Repetisjon, oppgaver og løsningsforslag	84

1 Introduksjon

Verden blir stadig mer datadrevet. For økonomer, analytikere og samfunnsvitere er det ikke lenger nok å utelukkende kunne bruke tradisjonelle regneark som Microsoft Excel. Selv om regneark har sin styrke i enkle scenarier, møter de raskt veggen når datamengdene blir store, når analysene krever avansert statistikk, eller når arbeidsprosesser skal automatiseres.

Python har vokst frem som det ledende verktøyet for dataanalyse på verdensbasis. Årsaken til Pythons popularitet ligger i at språket har en svært ren og lesbar syntaks som minner om vanlig engelsk tekst. I tillegg har språket et enormt globalt fellesskap som utvikler og vedlikeholder spesialiserte utvidelser (såkalte biblioteker eller pakker) for alt fra finansiell matematikk til kunstig intelligens.

Dette kompendiet tar deg gjennom de viktigste konseptene i programmering, skreddersydd for å løse kvantitative og økonomiske problemstillinger. Du trenger absolutt ingen forkunnskaper i koding eller informatikk for å følge dette materialet. Vi starter helt på skratsj med hvordan du setter opp verktøyene og skriver dine aller første linjer med kode.

Visste du at?

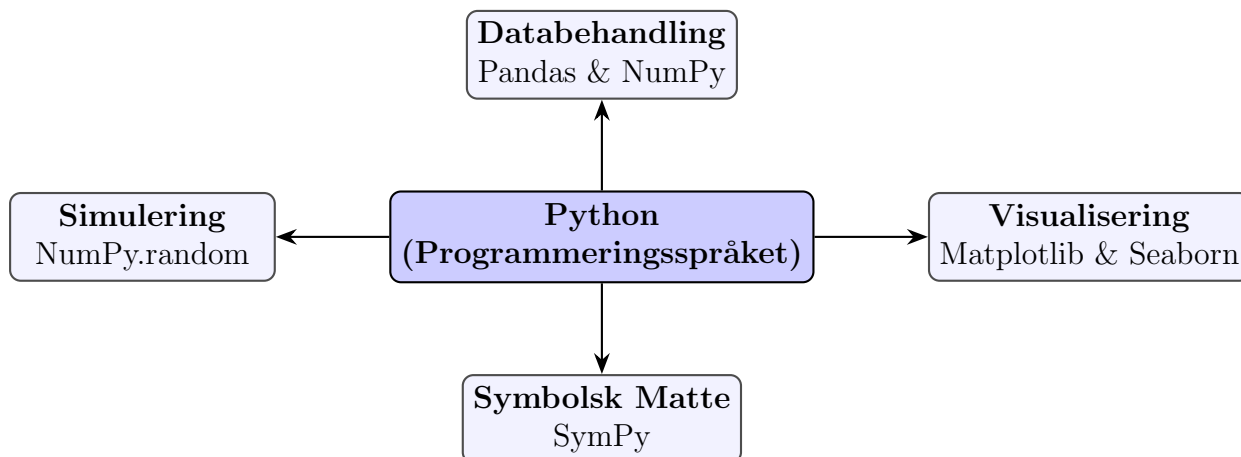
Hva betyr egentlig syntaks?

Når du hører programmerere snakke om *syntaks*, betyr det rett og slett de grammatiske reglene for et programmeringsspråk. Akkurat som vi i det norske språket må bruke punktum, stor forbokstav og riktig ordstilling for at en setning skal gi mening, har Python strenge regler for hvor du skal plassere parenteser, kolon, hermetegn og innrykk.

Hvis du gjør en skrivefeil eller glemmer en parentes, vil ikke datamaskinen forstå koden din. Da vil du få en såkalt **SyntaxError** (syntaksfeil). Det er datamaskinens måte å si: Setningen din gir ikke mening for meg!

1.1 Hva kan Python brukes til?

For en økonom fungerer Python som en "sveitserkniv". I stedet for å hoppe mellom ulike programmer for å hente data, vaske data, kjøre regresjoner og lage grafer, kan hele prosessen gjøres i ett og samme skript. Figur 1 gir en oversikt over de noen viktige områder du vil møte i dette kurset, og hvordan ulike pakker utvider grunnspråket.



Figur 1: Oversikt over hvordan utvidelser gjør Python til et kraftig verktøy for økonomisk analyse.

1.2 Tre ”måter” å kjøre Python på

Når du skal skrive Python-kode, trenger du et program som lar deg skrive teksten og sende den til datamaskinens prosessor. Vi skiller i hovedsak mellom å kjøre Python *lokalt* på egen maskin, eller *i skyen* via en nettleser. I dette kompendiumet så vil vi bruke Jupyter Notebook for å kjøre koden som presenteres her [3]. I dette kurset kan du fritt velge mellom tre gode alternativer:

1. **Visual Studio Code (Lokal installasjon):** Dette er den mest profesjonelle løsningen. VS Code er et gratis og svært populært tekstredigeringsprogram (editor) fra Microsoft. Ved å installere Python og VS Code på egen datamaskin, har du full kontroll, kan jobbe offline, og lærer det samme oppsettet som brukes i arbeidslivet. Dette krever at du følger installasjonsveiledningen under. Per 2026 har Visual Studio Code mange måter og integrere språkmodeller i editoren, slik at man kan skrive kode raskt og effektivt.
2. **Google Colab (Skybasert - ingen installasjon):** Google Colab lar deg skrive og kjøre Python-kode direkte i nettleseren din. Koden kjøres på Googles servere, noe som betyr at du ikke trenger å laste ned noe som helst, og det fungerer uavhengig av om du har en gammel eller ny PC/Mac. Alt du trenger er en Google-konto. Dette er et ypperlig alternativ hvis du vil komme raskt i gang.
3. **jupyter.uit.no (Skybasert for UiT-studenter):** Universitetet i Tromsø har en egen skytjeneste basert på JupyterHub. Ved å logge inn med din vanlige Feide-bruker får du tilgang til et ferdig oppsatt programmeringsmiljø i nettleseren. Her er alle de økonomiske og matematiske pakkene vi trenger i kurset allerede ferdig installert av IT-avdelingen, noe som minimerer risikoen for teknisk krøll underveis.

1.3 Installasjonsveiledning for lokal kjøring

Hvis du velger å kjøre koden lokalt via Visual Studio Code, må du installere to ting: selve programmeringsspråket Python, og editoren VS Code. Følg instruksjonene for ditt

operativsystem.

Viktig for både Windows og Mac

Når du har installert programmene under, må du åpne VS Code, trykke på utvidelses-ikonet (Extensions) på venstresiden (ser ut som fire ruter), søke etter **"Python"** og trykke *Install*. Gjør deretter det samme for utvidelsen **"Jupyter"**. Dette gjør at VS Code forstår Python-kode og lar deg kjøre såkalte "Notebooks" (notatblokker).

Installasjon på Windows

1. **Last ned Python:** Gå til den offisielle nettsiden <https://www.python.org>, trykk på "Downloads" og last ned nyeste versjon for Windows.
2. **Kjør installasjonsfilen (KRITISK STEG):** Dobbeltklikk på den nedlastede filen. Før du trykker på "Install Now", **MÅ du huke av i boksen nederst der det står "Add python.exe to PATH"**. Hvis du glemmer dette, vil ikke datamaskinen eller VS Code klare å finne Python-programmet senere.
3. **Last ned VS Code:** Gå til <https://code.visualstudio.com>, trykk på "Download for Windows" og fullfør installasjonen med standardinnstillingene.

Installasjon på macOS (Mac)

1. **Last ned Python:** Gå til <https://www.python.org>, trykk på "Downloads" og last ned installasjonsfilen (.pkg) for macOS.
2. **Installer:** Følg instruksjonene på skjermen. På Mac blir stiene opprettet automatisk under installasjonen.
3. **Last ned VS Code:** Gå til <https://code.visualstudio.com>, velg "Download for Mac". Du får ned en .zip-fil. Dobbeltklikk på denne, og trekk det synlige "Visual Studio Code"-ikonet over til *Applikasjoner*-mappen (Applications) på Macen din.

Når dette er gjort, er du fullstendig rigget og klar til å kjøre koden du vil møte på i neste kapittel!

1.4 Biblioteker og pakkehåndtering: Hva er pip?

Python kommer med mye funksjonalitet innebygd, men styrken til språket ligger i det enorme biblioteket av ferdiglagde pakker laget av andre. Vi har allerede sett at vi bruker `import pandas` for å få tilgang til verktøy for databehandling. Men hva gjør vi hvis vi vil bruke en pakke som *ikke* er installert på maskinen vår ennå?

Da bruker vi **pip** (står for (eng): *Pip Installs Packages*). Pip er et verktøy som kobler seg til internett, finner den nyeste versjonen av pakken du ber om, og laster den ned til din maskin.

!pip vs %pip: Hva bør du bruke?

Når du jobber i en Jupyter Notebook (som vi gjør i dette kurset), vil du ofte se to måter å installere pakker på: `!pip` og `%pip`. Det er viktig å forstå forskjellen:

- **`!pip install pakkenavn`:** Dette er en "skall-kommando". Den ber operativsystemet ditt om å kjøre pip helt uavhengig av hvilken Python-kjerne som kjører i notatboken din. Dette kan av og til føre til at pakken installeres på feil sted, slik at Jupyter ikke "ser" den når du prøver å importere den.
- **`%pip install pakkenavn`:** Dette er en såkalt "magisk kommando" i Jupyter (engelsk: *magic command*). Den sørger for at pip kjører *inne i den samme Python-kjøringen* som notatboken din bruker. Dette er nesten alltid den tryggeste og mest korrekte måten å installere biblioteker på inne i en notatbok.

Eksempel: Installere og bruke en ny pakke

```
# 1. Installerer pakken (vi bruker %-tegnet for å være trygge i Jupyter)
%pip install pandas

# 2. Etter at installasjonen er ferdig, kan vi importere pakken
import pandas as pd

# 3. Nå er alle Pandas-funksjonene klare til bruk!
data = {"Verdi": [10, 20, 30]}
df = pd.DataFrame(data)
print(df)
```

Hvorfor er dette nyttig for deg?

Som økonom eller analytiker vil du ofte møte på spesialiserte behov. Kanskje du trenger å hente aksjedata fra Yahoo Finance (`yfinance`), utføre avanserte økonometriske tester (`statsmodels`), eller koble deg direkte mot en database.

I stedet for å skrive titusenvs av linjer med kode for å løse disse oppgavene selv, kan du bare installere pakken ved hjelp av `%pip install [pakkenavn]`, så har du tilgang til ferdigutviklede, testede og profesjonelle verktøy i løpet av sekunder. Det er denne tilgangen til fellesskapets samlede kunnskap som gjør Python til et så kraftig verktøy for dataanalyse!

Dette kommer vi til å bruke mer tid på senere i kompendiumet!

1.5 Problemløsning og algoritmisk tenkning

Når man er fersk i programmering, er det fort gjort å tro at faget utelukkende handler om å pugge gloser (syntaks) og skrive kode raskest mulig. Men den egentlige jobben, og den mest verdifulle egenskapen du vil tilegne deg i dette kurset, er *algoritmisk tenkning*.

Algoritmisk tenkning handler om å bryte ned et stort og u håndterlig problem til en serie med små, logiske steg som en datamaskin (eller et menneske) kan utføre.

Pólyas fire steg for problemløsning

For å strukturere denne tankeprosessen støtter vi oss ofte på den ungarske matematikeren George Pólyas velkjente heuristikker (tommelfingerregler) fra boken *How to Solve It* (1945). Selv om disse opprinnelig ble laget for matematikk, er de som skapt for programmering:

1. **Forstå problemet:** Hva er det egentlig vi prøver å løse? Hvilke data har vi (input), og hva skal resultatet bli (output)? Hvis du ikke forstår problemet på papiret, vil du heller ikke klare å forklare det til en datamaskin.
2. **Legg en plan (Pseudokode):** Ikke begynn å kode enda! Planlegg logikken steg for steg i et format som er lett for deg å forstå. Dette kalles *pseudokode*.
3. **Gjennomfør planen:** Nå kan du oversette planen din fra menneskespråk (pseudokode) til Python-syntaks.
4. **Se tilbake (Evaluer):** Sjekk resultatet. Tror du på svaret datamaskinen gir deg? Hvis en lønnsberegning plutselig gir en skatt på 15 millioner, har det mest sannsynlig skjedd en logisk brist underveis.

Hva er pseudokode?

Som nevnt i Pólyas andre steg, er pseudokode et utrolig nyttig verktøy. Pseudokode er rett og slett en uformell måte å beskrive et dataprogram på. Den skrives i et språk som ligner på ekte kode, men uten de strenge grammatiske reglene til Python. Formålet er å bygge en bro mellom menneskelig tankegang og datamaskinens logikk.

Når du sitter fast på en eksamen, en arbeidsoppgave eller skal *prompte* for å få hjelp med en problem, bør du alltid legge fra deg tastaturet og skrive pseudokode med penn og papir først.

Eksempel: Fra problem til pseudokode til Python

Problemet: Vi ønsker å sjekke om et selskap går med overskudd eller underskudd.

Pseudokode (Planen):

- Start
- Hent inn samlede inntekter
- Hent inn samlede kostnader
- Beregn resultat = inntekter minus kostnader
- HVIS resultat er større enn 0:
 - Skriv ut "Selskapet går med overskudd"
- ELLERS:
 - Skriv ut "Selskapet går med underskudd"
- Stopp

Og slik blir den faktiske Python-koden:

```
inntekter = 500000
kostnader = 350000
resultat = inntekter - kostnader

if resultat > 0:
    print("Selskapet går med overskudd")
else:
    print("Selskapet går med underskudd")
```

Å tenke algoritmisk er en treningssak. I starten vil det føles uvant, men etter hvert vil du merke at denne strukturerte måten å bryte ned problemer på gjør deg til en langt mer analytisk og effektiv programmerer. Det vil også hjelpe deg i å *samarbeide* med språkmodeller og/eller andre mennesker om kodingen din.

1.6 Caseoppgave

KAPITTELCASE 1: Algoritmisk tenkning

Fagansvarligs tips: Programmering handler 80 % om å tenke, og 20 % om å skrive kode. I denne første caseoppgaven skal du bruke George Pólyas fire steg for å bryte ned et hverdagslig økonomisk problem. Du trenger ikke bruke datamaskinen din i det hele tatt – finn frem penn og papir!

Scenariet: Studentkafeen

Du har nylig tatt over økonomiansvaret for en liten studentkafé på campus. Kafeen selger kaffe for 35 kr per kopp. De faste kostnadene per dag (leie av kaffemaskin og strøm) er 500 kr. Det koster 10 kr i råvarer (kaffebønner, melk, pappkrus) å lage én kopp kaffe.

Du ønsker å lage et dataprogram der de ansatte kan skrive inn hvor mange kopper kaffe som ble solgt den dagen. Programmet skal deretter regne ut om kafeen gikk med overskudd eller underskudd, og skrive ut resultatet.

Del 1: Forstå problemet (Pólya trinn 1)

Før vi kan løse problemet, må vi vite nøyaktig hva vi har med å gjøre. Finn frem penn og papir, og svar på følgende:

1. Hva er **input** i dette programmet? (Hvilket tall er det vi ikke vet på forhånd, men som brukeren må gi oss?)
2. Hvilke variabler (konstanter) kjenner vi allerede fra teksten?
3. Hva er **output**? (Hva skal programmet gi som endelig svar?)

Del 2: Legg en plan (Pólya trinn 2)

Nå skal vi skrive *pseudokode*. Du skal ikke skrive Python-kode, men bruke vanlige norske ord til å beskrive logikken steg-for-steg ovenfra og ned.

- *Hint:* Husk at du først må regne ut totale inntekter, deretter totale kostnader (både faste og variable), før du til slutt finner dagsresultatet. Avslutt med en sjekk (HVIS dagsresultat er over 0, så...).

Del 3: Gjennomfør planen (Pólya trinn 3)

Nedenfor ser du et påbegynt Python-program basert på scenariet. Noen av ordene og utregningene mangler. Kan du fylle inn de tomme strekene (_____) slik at programmet gir mening, basert på pseudokoden din?

```
# 1. Definerer variablene vi kjenner
salgspris = 35
fast_kostnad = 500
variabel_kostnad = 10

# 2. Input fra brukeren
solgte_kopper = 40

# 3. Beregninger
totale_inntekter = solgte_kopper * _____
totale_kostnader = fast_kostnad + (_____ * variabel_kostnad)

dagsresultat = totale_inntekter - _____

# 4. Logisk beslutning
if dagsresultat > 0:
    print("Gratulerer! Dagen endte med et overskudd på", dagsresultat, "kr")
elif dagsresultat == 0:
    print("Dagen endte nøyaktig i null (Break-even).")
else:
    print("Dagen endte med et underskudd på", _____, "kr")
```

Del 4: Se tilbake og evaluer (Pólya trinn 4)

En av de viktigste egenskapene til en god programmerer er å teste for feil eller svakheter i sin egen logikk ("corner cases").

1. Hva skjer i programmet vårt dersom den ansatte taster inn -5 som `solgte_kopper` ved en feiltakelse? Gir matematikken mening da?
2. Test logikken i hodet ditt: Hvor mange kopper kaffe må vi selge for at programmet skal skrive ut at vi endte "nøyaktig i null"? (Sjekk: Dekningsbidraget per kaffe er 25 kr. For å dekke inn 500 kr, må vi selge...?)

2 Python Basics

Når vi programmerer i Python, gir vi datamaskinen en serie med instruksjoner som den utfører trofast ovenfra og ned. For å kunne bygge økonomiske og statistiske modeller, må vi først mestre de mest grunnleggende byggeklossene i språket: kommentarer, variabler, matematiske operasjoner, datatyper og kommunikasjon med programmet via inn- og utdata (*input* og *output*).

2.1 Kommentarer i koden

Før vi begynner å skrive funksjonell kode, må vi vite hvordan vi tar notater. I Python bruker vi emneknagg/hashtag (#) for å opprette kommentarer. Alt som står på en linje etter dette symbolet, blir fullstendig ignorert av datamaskinen. Kommentarer brukes flittig til å forklare hva koden gjør, både for deg selv og andre som skal lese den senere.

```
# Dette er en kommentar. Datamaskinen hopper over denne linjen.  
inntekt = 500000 # Vi kan også plassere kommentarer etter fungerende kode
```

2.2 Variabler og tilordning

En variabel kan best beskrives som en navngitt boks i datamaskinens minne hvor vi kan lagre en verdi. For å putte noe inn i denne boksen, bruker vi *tilordningsoperatoren* (=).

Det er kritisk å forstå at = i programmering betyr ta verdien eller beregningen på høyre side og lagre den i variabelen på venstre side. Det fungerer altså som en enveis-pil ($\leftarrow$), og er *ikke* det samme som et vanlig likhetstegn fra matematikken.

Eksempel: Variabeltilordning og overskudd

```
# Vi oppretter variabler for en bedrift  
pris_per_enhet = 150  
antall_solgt = 1200  
faste_kostnader = 50000  
  
# Vi bruker de eksisterende variablene til å gjøre nye beregninger  
tot_inntekt = pris_per_enhet * antall_solgt  
tot_kostnad = faste_kostnader + (pris_per_enhet * 0.4 * antall_solgt)  
overskudd = tot_inntekt - tot_kostnad  
  
print("Bedriftens overskudd ble:", overskudd)
```

```
> Bedriftens overskudd ble: 58000.0
```

Hvorfor Python i stedet for Excel?

Eksempelet over er et enkelt, økonomisk eksempel, og mange økonomistudenter kan stille seg et helt betimelig spørsmål: *Kunne jeg ikke bare gjort dette mye raskere i et regneark?*

La oss se på hvordan den nøyaktig samme beregningen ville sett ut i Excel:

	A	B
1	Pris per enhet	150
2	Antall solgt	1200
3	Faste kostnader	50000
4	Tot. inntekt	180000
5	Tot. kostnad	122000
6	Overskudd	58000

$=B3+(B1*0.4*B2)$

$=B4-B5$

Figur 2: Slik ser den samme logikken ut i et regneark. Formlene er abstrakte og skjult bak tallene.

Svaret er naturligvis at regnearket gir oss det samme overskuddet. Men det illustrerer også en av de største svakhetene ved Excel: **Mangel på sporbarhet**.

- **Abstrakte referanser vs. Navngitte variabler:** Hvis du sender Excel-arket til en kollega, og de trykker på celle B5, ser de formelen $=B3+(B1*0.4*B2)$. De må manuelt spore opp hva kolonne B og rad 1, 2 og 3 inneholder for å forstå hva som regnes ut. I Python er koden selv-dokumenterende fordi variablene har faktiske navn: `tot_kostnad = faste_kostnader + (pris_per_enhet * 0.4 * antall_solgt)`.
- **Skjult logikk:** I Excel ser du bare sluttresultatet (58 000). For å finne logikken må du aktivt klikke deg inn på hver enkelt celle og lese formellinjen. I Python leser du beregningene som en oppskrift ovenfra og ned.
- **Feilmarginer:** Hva om noen ved et uhell flytter på en celle eller legger til en ny rad i Excel? Formelen $B1*B2$ kan plutselig referere til feil tall, uten at systemet gir deg en feilmelding. Feil i regneark har kostet bedrifter mye penger, historisk sett. Python kan forhindre enkelte feil ved at alt må defineres eksplisitt og systematisk.

Å lære seg Python handler derfor ikke bare om å kunne behandle store datasett, men om å bygge analysemodeller som er transparente, enkle å feilsøke, og trygge å dele med andre. **OBS:** Når dette er sagt, så er det ikke for å si at regneark er dumt, mens Python er bra. Dette er tatt som et eksempel for å vise at de kan ha hver sine fordeler og ulemper.

2.3 Matematiske operatorer

Python fungerer utmerket som en avansert kalkulator. De standard matematiske operatorene du har til rådighet er oppsummert i tabell 1.

Operator	Beskrivelse	Eksempel i Python	Resultat
+	Addisjon (Pluss)	10 + 5	15
-	Subtraksjon (Minus)	10 - 5	5
*	Multiplikasjon (Gange)	10 * 5	50
/	Divisjon (Dele - gir alltid float)	10 / 4	2.5
**	Potens (x^y)	2 ** 3	8
//	Heltallsdivisjon (Runder ned til heltall)	10 // 4	2
%	Modulo (Gir resten etter divisjon)	10 % 4	2

Table 1: Matematiske operatorer i Python.

For økonomer er operatoren ****** spesielt viktig, da den brukes til å beregne vekst, rentesrente og nåverdier:

Eksempel: Variabeltilordning og overskudd

```
# Beregning av fremtidsverdi (FV) etter t år:  $FV = K * (1 + r)^t$ 
startkapital = 100000
rente = 0.05
aar = 10

fremtidsverdi = startkapital * (1 + rente)**aar
```

2.4 Datatyper og sjekk av type

Verdiene vi putter i variablene har ulike *datatyper*. Python må holde styr på dette for å vite hvilke operasjoner som er logisk tillatt. De fire viktigste grunnleggende datatypene du må kjenne til er:

- **int** (Integer): Heltall. Brukes til tellbare enheter, som antall aksjer eller ansatte (45, -12).
- **float** (Floating-point): Desimaltall. Brukes til kontinuerlige verdier som priser, prosenter og renter (0.035, 99.90). *Husk å alltid bruke punktum som desimaltegn, ikke komma!*
- **str** (String): Tekststrenger. All tekst må omslutes av enten doble eller enkle hermetegn ("Markus", 'Handelshøgskolen').
- **bool** (Boolean): Sannhetsverdier. Kan utelukkende være enten **True** (Sant) eller **False** (Usant).

Hvis du noen gang er usikker på hvilken datatype en variabel inneholder, kan du spørre Python ved hjelp av den innebygde funksjonen **type()**:

Eksempel: Variabeltilordning og overskudd

```
skattesats = 0.22
kursnavn = "Python-Lab"

print(type(skattesats)) # Output: <class 'float'>
print(type(kursnavn))   # Output: <class 'str'>
```

2.5 Typekonvertering (Type Casting)

Av og til må vi tvinge data over fra én datatype til en annen. Dette kalles *typekonvertering*. Vi bruker funksjoner som har samme navn som datatypen vi ønsker å konvertere til:

```
# Gjør om et desimaltall til et heltall (fjerner alt etter punktum)
avrundet_pris = int(149.99) # Resultat: 149
```

```
# Gjør om et heltall til et desimaltall
verdi_float = float(25) # Resultat: 25.0
```

```
# Gjør om et tall til ren tekst
streng_tall = str(2024) # Resultat: "2024"
```

2.6 Innlesing av data med input()

For å lage interaktive programmer kan vi be brukeren om å skrive inn verdier fra tastaturet ved hjelp av funksjonen `input()`.

Viktig huskeregel: `input()` tolker *alltid* det brukeren skriver inn som tekst (`str`), selv om brukeren taster inn rene tall. Skal vi utføre matematiske beregninger med inputen, må vi konvertere den med en gang ved å pakke den inn i `int()` eller `float()`.

Eksempel: Interaktiv skatteberegner

```
# Henter tekst direkte
navn = input("Skriv inn ditt navn: ")

# Henter tall og konverterer dem slik at de kan regnes med
bruttoinntekt = float(input("Hva er din bruttoinntekt i kr? "))
skatteprosent = float(input("Oppgi din skatteprosent (f.eks. 22): "))

# Beregninger
skatt = bruttoinntekt * (skatteprosent / 100)
nettoinntekt = bruttoinntekt - skatt

print("Hei", navn)
print("Du betaler kr", skatt, "i skatt, og sitter igjen med kr",
      nettoinntekt)
```

2.7 Utskriftsformatering med f-strenger og round()

Når vi gjør divisjoner eller komplekse renteberegninger, ender vi ofte opp med uendelig mange desimaler (for eksempel $10000 * (1.05)^{10}$ som blir 162889.4626777066). For å presentere tall på en ryddig måte bruker vi to viktige verktøy: funksjonen `round()` og **f-strenger**.

En f-streng markeres ved å sette en `f` rett foran det første hermetegnet i en print-setning. Dette lar oss flette variabler eller regnestykker direkte inn i teksten ved å pakke dem inn i krøllparenteser `{}`.

Eksempel: Variabeltilordning og overskudd

```
bnp_per_innbygger = 850432.7264

# Alternativ 1: Bruke round()-funksjonen direkte i en f-streng
print(f"BNP per innbygger er {round(bnp_per_innbygger, 2)} NOK.")
# Output: BNP per innbygger er 850432.73 NOK.

# Alternativ 2: Formaterer antall desimaler direkte med :.2f
# inne i krøllparentesen
print(f"Formatert versjon: {bnp_per_innbygger:.2f} NOK.")
# Output: Formatert versjon: 850432.73 NOK.
```

2.8 Sammenlignings- og logiske operatorer

Vi har sett hvordan vi kan bruke *matematiske* operatorer til å regne, men for å få data-maskinen til å ta smarte beslutninger, må vi også kunne *sammenligne* verdier. Gir prosjektet positiv avkastning? Er renten over 5 %? Ligger aksjen i porteføljen vår?

Til dette bruker vi sammenligningsoperatorer, logiske operatorer og medlemskapsoperatorer. Det disse har til felles, er at de alltid returnerer en boolsk verdi: enten `True` eller `False`.

Sammenligningsoperatorer

Disse brukes for å sjekke forholdet mellom to verdier. Merk spesielt at for å sjekke om to verdier er nøyaktig like, må vi bruke *to* likhetstegn (`==`), siden ett enkelt likhetstegn (`=`) allerede er reservert for variabeltilordning.

Operator	Beskrivelse	Eksempel	Resultat
<code>==</code>	Er lik	<code>10 == 10</code>	<code>True</code>
<code>!=</code>	Er ikke lik	<code>10 != 5</code>	<code>True</code>
<code>></code>	Større enn	<code>10 > 20</code>	<code>False</code>
<code><</code>	Mindre enn	<code>10 < 20</code>	<code>True</code>
<code>>=</code>	Større enn eller lik	<code>15 >= 15</code>	<code>True</code>
<code><=</code>	Mindre enn eller lik	<code>15 <= 10</code>	<code>False</code>

Table 2: Sammenligningsoperatorer i Python.

Logiske operatører (and, or, not)

Av og til holder det ikke å sjekke bare én betingelse. Kanskje vi bare vil investere *hvis* avkastningen er høy **og** risikoen er lav.

- **and**: Returnerer **True** kun dersom *begge* sidene er sanne.
- **or**: Returnerer **True** dersom *minst én* av sidene er sann.
- **not**: Snur sannhetsverdien på hodet (gjør **True** til **False** og omvendt).

Medlemskapsoperatører (in, not in)

Disse operatorene er utrolig praktiske når vi vil sjekke om en bestemt verdi befinner seg inni en samling av data, for eksempel en liste med aksjer eller i en tekststreng.

Eksempel: Evaluering av et investeringsprosjekt

```
# Vi definerer parametere for et tenkt prosjekt
forventet_avkastning = 0.08
avkastningskrav = 0.05
hoy_risiko = False
portefolje = ["Equinor", "DNB", "Telenor", "Mowi"]

# 1. Sammenligning: Er avkastningen høy nok?
krav_oppfylt = forventet_avkastning >= avkastningskrav

# 2. Logikk: Invester hvis kravet er oppfylt OG det IKKE er høy risiko
skal_investere = krav_oppfylt and not hoy_risiko

# 3. Medlemskap: Har vi DNB i porteføljen vår fra før?
har_dnb = "DNB" in portefolje

print(f"Oppfyller prosjektet avkastningskravet? {krav_oppfylt}")
print(f"Bør vi gjennomføre investeringen? {skal_investere}")
print(f"Er DNB allerede i porteføljen vår? {har_dnb}")

> Oppfyller prosjektet avkastningskravet? True
> Bør vi gjennomføre investeringen? True
> Er DNB allerede i porteføljen vår? True
```

Ta også titt en ekstra gang på eksempelet over, kan du forklare hvorfor vi får **True** som output i hver av de tre `print()`-setningene?

2.9 Caseoppgave

KAPITTELCASE 2: Variabler og datatyper

Fagansvarligs tips: Variabler er selve byggeklossene i all programmering. Å gi variablene dine gode, beskrivende navn er kanskje den mest undervurderte egenskapen til en god programmerer. I denne casen skal vi se på hvordan vi lagrer ulike typer informasjon, og hva som skjer når vi bryter reglene.

Scenariet: Lønnssystemet

Du har fått en deltidsjobb i administrasjonen til et lokalt bakeri. Hver måned må sjefen regne ut hvor mye de ansatte skal ha utbetalt etter skatt. Sjefen bruker i dag en gammel kalkulator, og ber deg om å skrive et lite Python-program som kan automatisere utregningen av en lønsslipp.

Del 1: Identifiser datatypene

For å bygge lønssystemet, må vi lagre ulike informasjon i variabler. Hvilken *datatype* (Integer, Float, eller String/Tekst) bør du bruke for å lagre følgende verdier i Python?

1. Den ansattes navn ("Kari Nordmann")
2. Antall timer jobbet (37)
3. Skattesatsen (0.25)

Del 2: Fyll inn koden

Nedenfor ser du utkastet til lønnsprogrammet. Din oppgave er å fylle inn de tomme strekene (____) slik at utregningene blir riktige, og at den avsluttende utskriften (f-stringen) fungerer.

```
# 1. Variabler for den ansatte
ansatt_navn = "Kari Nordmann"
timer_jobbet = 37
timelonn = 250
skattesats = 0.25

# 2. Matematiske utregninger
bruttolonn = _____ * _____
skattetrekk = bruttolonn * skattesats
nettolonn = bruttolonn - _____

# 3. Utskrift av lønsslipp
print(f"Lønsslipp for {_____}:")
print(f"Bruttolønn: {bruttolonn} kr")
print(f"Skattetrekk: {skattetrekk} kr")
print(f"Utbetalt (Nettolønn): {_____} kr")
```

Del 3: Finn feilen (Feilsøking)

Sjefen i bakeriet har prøvd å skrive litt Python-kode selv, men programmet hans nekter å kjøre. Han får feilmeldinger ("SyntaxError" og "TypeError") overalt.

Se på koden hans nedenfor. Kan du finne de **tre** store feilene han har gjort med variablene sine?

- *Hint:* Tenk på reglene for hva et variabelnavn kan starte med, hvilke tegn som er lov å bruke, og hva som skjer hvis man prøver å blande tekst og tall med et plusstegn.

```
# Sjefens ødelagte kode
1_ansatt = "Ola Hansen"
skatt% = 0.20
utbetalt = 5000

print("Lønnen til " + 1_ansatt + " er " + utbetalt)
```

Del 4: Refleksjon rundt ”Ryddig kode”

Du kunne teknisk sett ha kalt variablene dine for `a`, `b`, `c` og `d` i stedet for `ansatt_navn`, `timer_jobbet`, osv., og datamaskinen ville regnet ut akkurat det samme svaret.

Hvorfor er det likevel en ekstremt dårlig idé for en økonom eller dataanalytiker å bruke slike korte, ubeskrivende variabelnavn i et profesjonelt arbeidsmiljø? Hvem er det egentlig vi skriver koden for?

3 Funksjoner

Funksjoner er helt sentralt i all slags programmering. Et program uten egendefinerte funksjoner er egentlig bare en lang liste med kommandoer, og det er svært begrenset hva man kan bygge med det.

En funksjon er rett og slett en gjenbrukbar kodeblokk som tar inn noen verdier (parametere), gjør en beregning, og returnerer et resultat.

En vanlig nybegynnerfeil er å ikke utnytte muligheten til å lage egne funksjoner tilstrekkelig. Hovedregelen er at dersom du har kode som repeteres flere ganger, bør du lage en ny funksjon som håndterer dette. Dette gjør koden vår modulær, ryddig, og langt enklere å feilsøke. Et annet godt tips er at antall linjer i hver funksjon bør holdes så lavt som mulig.

3.1 Innebygde funksjoner i Python

Før vi lærer å lage egne funksjoner, er det greit å vite at Python kommer med en lang rekke *innebygde* funksjoner som allerede er programmert for oss. Slik slipper vi å finne opp kruttet på nytt.

Eksempel: Innebygde funksjoner

```
# max() og min() finner henholdsvis største og minste verdi
hoyeste_tall = max(5, 3.0, 50)
minste_tall = min(2.4, 2.0)

# abs() finner absoluttverdien av et tall
absolutt = abs(-2.4)

# len() finner antall tegn (lengden) i en tekststreng, liste etc.
lengde = len("Handelshøgskolen")

print(f"Høyeste tall: {hoyeste_tall}")
print(f"Minste tall: {minste_tall}")
print(f"Absoluttverdi: {absolutt}")
print(f"Lengde på ordet: {lengde}")

> Høyeste tall: 50
> Minste tall: 2.0
> Absoluttverdi: 2.4
> Lengde på ordet: 16
```

3.2 Hva er en funksjon?

I Python kan vi tenke på en funksjon som en liten maskin eller en fabrikk. Som figur 3 viser, består denne prosessen av tre hoveddeler:

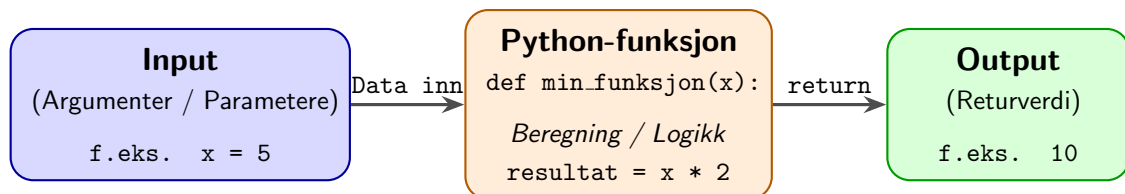
- **Input (Data inn):** For at maskinen skal kunne gjøre en jobb, må vi ofte putte noe inn i den. Dette kaller vi *argumenter* eller *parametere*. I eksempelet over mater vi

funksjonen med verdien $x = 5$.

- **Maskineriet (Beregning):** Inne i funksjonen ligger selve oppskriften eller logikken. Dette er koden som forteller nøyaktig hva som skal skje med dataene vi sendte inn. I figuren under er oppgaven enkel: funksjonen skal ta tallet den får inn og gange det med to ($x * 2$).
- **Output (Data ut):** Til slutt spytter programmet ut et ferdig resultat, som vi kaller en *returverdi*. I Python bruker vi nøkkelordet `return` for å sende dette resultatet (her tallet 10) tilbake til der funksjonen ble brukt.

Hvorfor bygger vi slike funksjoner?

Hovedgrunnen er *gjenbruk* og *ryddighet*. Når ”maskinen” først er definert, slipper du å skrive den samme koden om og om igjen. Du kan bare kalle på funksjonen og mate den med nye tall ($x = 10$, $x = 500$), så vil den gjøre jobben sin og returnere et nytt, ferdig beregnet resultat hver eneste gang.



Figur 3: Illustrasjon av en Python-funksjon. Den tar inn data (input), prosesserer dem i en kodeblokk, og sender resultatet tilbake (output) ved hjelp av `return`.

3.3 Å definere egne funksjoner (def)

For å lage en egen funksjon i Python bruker vi nøkkelordet `def` (fra engelsk *define*). Deretter gir vi funksjonen et logisk navn, etterfulgt av parenteser som inneholder *parametere* (verdiene funksjonen forventer å få inn). Til slutt må vi ha et kolon, og selve koden inni funksjonen må ha *innrykk*.

Som regel avslutter vi en funksjon med `return`. Dette sender resultatet av beregningen tilbake til der funksjonen ble kalt på, slik at vi kan lagre resultatet i en variabel og bruke det videre i programmet. For økonomer bruker vi ofte funksjoner for oppgaver som skal gjentas, for eksempel nåverdimetoden (NPV).

Eksempel: Økonomisk funksjon

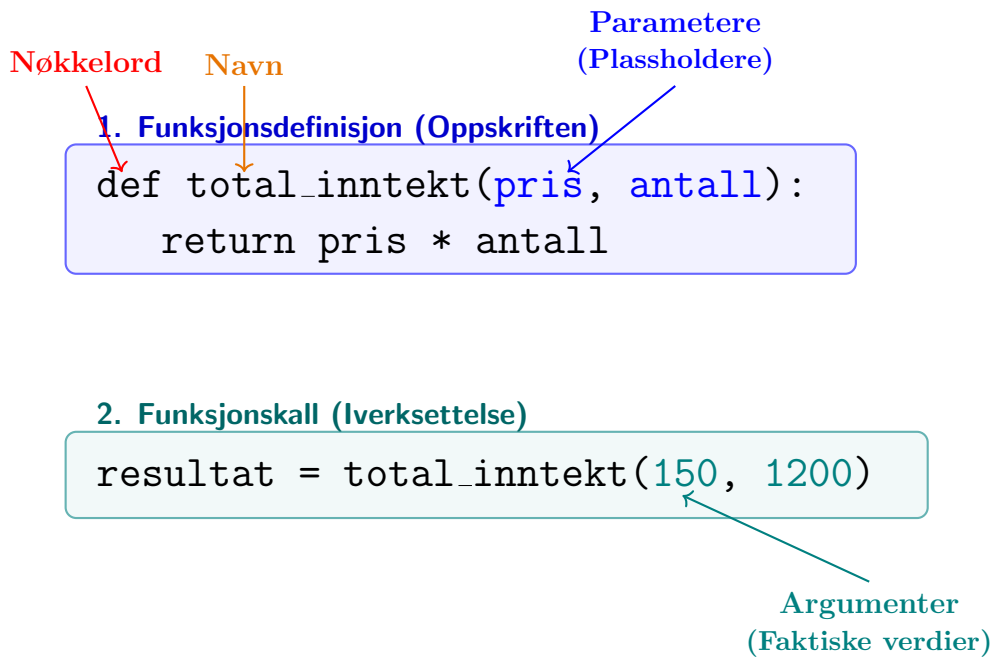
```
def beregn_fremtidsverdi(naaverdi, rente, aar):  
    """  
    Beregner fremtidsverdi av et beløp.  
    Teksten inni her kalles en 'docstring' og fungerer som  
    funksjonens innebygde bruksanvisning.  
    """  
    fremtidsverdi = naaverdi * (1 + rente)**aar  
    return fremtidsverdi  
  
fv = beregn_fremtidsverdi(10000, 0.05, 10)  
print(f"Verdien etter 10 år er {fv:.2f} kr")  
  
> Verdien etter 10 år er 16288.95 kr
```

3.4 Funksjonens anatomi: Parametere vs. Argumenter

Eksempelet over viser hvordan en funksjon er definert i Python, men la oss se litt nærmere på dette. For når man først lærer om funksjoner, er det spesielt to begreper som ofte forveksles: *parametere* og *argumenter*. Selv om de virker like, har de to helt forskjellige roller i funksjonens levetid.

- **Parametere (*Placeholders*):** Dette er variablene vi definerer i parentesene på topplinjen til funksjonen (`def`-linjen). De fungerer som midlertidige tomme bokser eller merkelapper. Funksjonen vet ikke hva som skal puttes i dem ennå, men den vet hvordan den skal bruke dem når de en gang får et innhold.
- **Argumenter (De faktiske verdiene):** Dette er de ekte verdiene (tall, tekst, lister) som du faktisk sender *inn* i funksjonen når du kaller på den senere i koden din.

Figur 4 illustrerer denne anatomen visuelt, samt hvordan verdiene flyter fra kallet og opp til definisjonen.



Figur 4: Funksjonens anatomi. Argumentene (150 og 1200) sendes inn og overtar plassen til parameterne (`pris` og `antall`) under kjøring.

3.5 Viktigheten av `return` vs. `print`

En av de aller vanligste fellene for nybegynnere er å blande sammen `return` og `print()`. Begge deler kan få det til å se ut som funksjonen gjør det samme i Jupyter Notebook, men under panseret er forskjellen fundamental.

- `print()` betyr: Vis dette resultatet på skjermen for mennesket som ser på. Så snart teksten er printet, glemmer datamaskinen tallet. Verdien er "tapt" for resten av programmet.
- `return` betyr: Send denne verdien tilbake til programmet. Datamaskinen lagrer resultatet, slik at du kan putte det inn i en ny variabel, bruke det i en ny formel, eller sende det videre til en annen funksjon.

Hvis du ikke oppgir en `return`-setning i funksjonen din, vil Python automatisk returnere en usynlig verdi kalt `None` (ingenting).

La oss se på hva som skjer hvis en økonom prøver å bruke en funksjon som mangler `return` til å beregne mva (merverdiavgift) på et overskudd:

FEIL: Funksjon uten return (Bruker print)

```
def beregn_overskudd_feil(inntekt, kostnad):  
    overskudd = inntekt - kostnad  
    print(overskudd) # Viser tallet på skjermen, men returnerer ingenting!  
  
# Vi kaller funksjonen og prøver å lagre resultatet i en variabel  
mitt_overskudd = beregn_overskudd_feil(10000, 4000)  
  
# Vi prøver å regne videre med variabelen  
mva_a_betale = mitt_overskudd * 0.25
```

6000

TypeError: unsupported operand type(s) for *: 'NoneType' and 'float'

Legg merke til feilmeldingen over. Funksjonen printet ut 6000, så det så riktig ut på skjermen. Men fordi den manglet `return`, ble variabelen `mitt_overskudd` stående tom (`NoneType`). Datamaskinen klarer naturligvis ikke å gange ingenting med 0.25, og programmet krasjer.

Her ser du den korrekte måten å gjøre det på, hvor verdien faktisk sendes tilbake og blir gjenbrukbar:

RIKTIG: Funksjon med return

```
def beregn_overskudd_riktig(inntekt, kostnad):  
    overskudd = inntekt - kostnad  
    return overskudd # Sender verdien trygt tilbake til programmet  
  
mitt_overskudd = beregn_overskudd_riktig(10000, 4000)  
  
# Nå fungerer det utmerket å regne videre!  
mva_a_betale = mitt_overskudd * 0.25  
print(f"MVA å betale blir: {mva_a_betale} kr")  
  
> MVA å betale blir: 1500.0 kr
```

En gylden huskeregel er derfor: *Bruk `print()` når du skal snakke til et menneske, og `return` når funksjonen skal snakke videre til resten av dataprogrammet.*

Prøv selv!

Se nøye på eksemplene over med `return` og `print()`. Det er lett å nikke og tro man forstår det når man leser, men den beste måten å lære det på, er å kjenne det på kroppen (eller i tastaturet).

Åpne Jupyter Notebook, skriv av begge eksemplene nøyaktig slik de står, og prøv å kjøre dem. Endre deretter på tallene (argumentene) du sender inn. Hva skjer hvis du prøver å dele resultatet på to i stedet for å gange med 0.25? Ved å gjøre feilene selv i trygge omgivelser, blir du en mye bedre problemløser!

Et ikke-økonomisk eksempel: Kroppsmasseindeks (BMI)

I biologi, helsefag og medisin er det ofte behov for å standardisere målinger for store grupper av pasienter. Et klassisk eksempel er å beregne BMI (Kroppsmasseindeks). Siden selve den matematiske formelen er lik for alle pasienter, er dette et perfekt bruksområde for en funksjon.

Eksempel: Biologi og Helse (BMI)

```
def beregn_bmi(vekt_kg, hoyde_m):  
    """  
    Beregner BMI basert på vekt (kg) og høyde (m).  
    """  
    bmi = vekt_kg / (hoyde_m ** 2)  
    return round(bmi, 1)  
  
# Beregner BMI for to pasienter  
pasient1_bmi = beregn_bmi(75, 1.80)  
pasient2_bmi = beregn_bmi(92, 1.75)  
  
print(f"Pasient 1 har BMI: {pasient1_bmi}")  
print(f"Pasient 2 har BMI: {pasient2_bmi}")
```

```
> Pasient 1 har BMI: 23.1  
> Pasient 2 har BMI: 30.0
```

Et psykologisk/statistisk eksempel: Z-score

I psykologien brukes det ofte kognitive tester (for eksempel IQ-tester eller hukommelses-tester). For å kunne sammenligne resultatene fra ulike tester, gjør psykologer og samfunnsvitere ofte rå-scorene om til en *Z-score* (standardavviksenheter). Dette forteller oss hvor mye et individ avviker fra gjennomsnittet i en populasjon.

Formelen for Z-score er: $Z = \frac{X - \mu}{\sigma}$, der X er pasientens score, μ er populasjonsgjennomsnittet, og σ er standardavviket. Ved å lage en funksjon, kan psykologen konvertere et helt datasett med testresultater på sekunder.

Eksempel: Psykologi og Statistikk (Z-score)

```
def beregn_z_score(score, snitt, standardavvik):  
    """  
    Beregner Z-score for å se hvor mange standardavvik  
    en score ligger over eller under gjennomsnittet.  
    """  
    z_score = (score - snitt) / standardavvik  
    return round(z_score, 2)  
  
# Populasjonssnittet for en standard IQ-test er 100,  
# og standardavvik er 15  
iq_score = 120  
z = beregn_z_score(score=iq_score, snitt=100, standardavvik=15)  
  
print(f"En score på {iq_score} gir en Z-score på {z}")  
  
> En score på 120 gir en Z-score på 1.33
```

3.6 Korte, anonyme funksjoner (Lambda)

Mange nybegynnere synes ordet *lambda* høres skummelt og avansert ut, men det er egentlig bare en praktisk skrivemåte for å lage bittesmå funksjoner på én enkelt linje.

Slike funksjoner kalles ofte *anonyme funksjoner* fordi de i utgangspunktet ikke har noe navn (vi bruker ikke `def`-nøkkelordet for å navngi dem). Syntaksen for en lambda-funksjon er alltid:

lambda parametere: uttrykk

Lambda-funksjoner inneholder automatisk en usynlig `return`. De er svært nyttige senere i kurset, for eksempel når vi skal gjøre en rask matematisk justering på en hel kolonne med data i Pandas, og en full `def`-blokk føles som overkill.

La oss se på forskjellen mellom en vanlig funksjon og en lambda-funksjon som gjør nøyaktig det samme (beregner 22 % skatt av en inntekt):

Eksempel: Vanlig funksjon vs. Lambda

```
# 1. Den tradisjonelle måten (tar flere linjer)
def beregn_skatt_vanlig(inntekt):
    return inntekt * 0.22

# 2. Lambda-måten (alt på én linje)
# Her velger vi å lagre den i en variabel for å kunne kalle på den
beregn_skatt_lambda = lambda inntekt: inntekt * 0.22

# Kaller på begge funksjonene for å se at resultatet blir likt
vanlig_skatt = beregn_skatt_vanlig(500000)
lambda_skatt = beregn_skatt_lambda(500000)

print(f"Skatt fra vanlig funksjon: {vanlig_skatt}")
print(f"Skatt fra lambda-funksjon: {lambda_skatt}")

> Skatt fra vanlig funksjon: 110000.0
> Skatt fra lambda-funksjon: 110000.0
```

Huskeregel: Bruk `def` når funksjonen krever flere steg med logikk, eller når du skal bruke den mange steder i programmet ditt. Bruk `lambda` når du kun trenger en enkel matematisk operasjon der og da (en såkalt ”bruk og kast”-funksjon).

3.7 Caseoppgave

KAPITTELCASE 3: Funksjoner og AI

Fagansvarligs tips: En funksjon er som en liten fabrikk: Du putter råvarer inn (parametere), fabrikken gjør en jobb, og til slutt spytter den ut et ferdig produkt (**return**). I denne casen skal du designe din egen fabrikk fra bunnen av, før vi ser på en klassisk felle mange går i når de ber kunstig intelligens om å skrive funksjoner for dem.

Scenariet: Kasssystemet i nettbutikken

Du bygger et kasssystem for en nettbutikk. Nettbutikken selger varer med ulike rabatt, og i tillegg må det legges til 25 % merverdiavgift (MVA) på slutten. Siden dette skal gjøres for tusenvis av kunder hver dag, ønsker du å lage en *funksjon* som tar inn den opprinnelige prisen og rabatten, og regner ut nøyaktig hva kunden skal betale.

Del 1: Algoritmisk tenkning (Pólyas trinn 1 & 2)

Før vi koder, må vi planlegge "fabrikken" vår.

1. **Input (Parametere):** Hvilke to verdier må funksjonen vår få vite av brukeren for å kunne regne ut sluttprisen?
2. **Logikk (Pseudokode):** Skriv ned på papir de to matematiske stegene funksjonen må utføre. (Hint: Trekk fra rabatten først, gang deretter med 1.25 for å legge til MVA).
3. **Output (return):** Hva er det endelige tallet fabrikkens skal levere tilbake til kasssystemet?

Del 2: Bygg funksjonen

Nedenfor ser du skjelettet til funksjonen. Fyll inn de tomme strekene (-----) slik at funksjonen tar inn riktige parametere, utfører logikken din, og leverer svaret tilbake.

```
# 1. Definerer fabrikken og hvilke råvarer (parametere) den trenger
def beregn_sluttpris(-----, -----):

    # 2. Utfører jobben (logikken)
    pris_etter_rabatt = pris - rabatt
    pris_med_mva = pris_etter_rabatt * 1.25

    # 3. Sender det ferdige produktet ut av fabrikken
    return -----

# Tester funksjonen (Kunden kjøper en jakke til 1000 kr, med 200 kr rabatt)
kunden_skal_betale = beregn_sluttpris(1000, 200)
print(f"Totalpris å betale: {kunden_skal_betale} kr")
```

Del 3: AI-assistanse og den store fellen

Sjefen din synes det går litt tregt, og bestemmer seg for å bruke ChatGPT for å lage en ny funksjon. Han ber ChatGPT om følgende: *Lag en Python-funksjon som tar inn en pris, legger til 25 % skatt, og viser resultatet.*

ChatGPT gir sjefen din denne koden:

```
def legg_til_skatt(pris):
    pris_med_skatt = pris * 1.25
    print(pris_med_skatt) # AI-en bruker print i stedet for return!

# Sjefen prøver å bruke funksjonen for å regne ut prisen for tre bøker:
en_bok = legg_til_skatt(200)
totalpris_for_tre_boker = en_bok * 3
```

Når sjefen kjører koden, krasjer hele programmet med feilmeldingen:

`TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'`

Din oppgave: Forklar sjefen hvorfor programmet krasjer.

- *Hint:* Hva er den fundamentale forskjellen på `print` (vise noe på skjermen for et menneske) og `return` (levere en verdi tilbake til datamaskinen)? Hva skjer med en variabel hvis funksjonen ikke returnerer noe som helst?

4 Datastrukturer og NumPy

Når vi jobber med dataanalyse eller økonomiske modeller, håndterer vi sjelden bare én og én variabel av gangen. Vi jobber med tusenvis av historiske aksjepriser, lange lister med kundenavn, eller komplekse tabeller. For å håndtere dette effektivt, trenger vi strukturer for å lagre og organisere dataene våre.

I Python har vi tre svært viktige innebygde datastrukturer: **lister**, **tupler** og **dictionaries**. I tillegg bruker vi eksterne pakker som **NumPy** for å gjøre lynrask matematikk på disse strukturene.

4.1 Lister (Lists)

En liste er den mest brukte måten å lagre flere verdier på i Python. Du kan tenke på det som en handleliste eller en portefølje. Lister defineres ved hjelp av klammeparenteser `[]`, og de er *mulige å endre på* (mutable). Det betyr at vi når som helst kan legge til, fjerne eller endre elementer i listen etter at den er opprettet.

Indeksering (Viktig!): I Python starter vi alltid å telle på null! Det første elementet i en liste bor derfor på "plass 0" (indeks 0).

En utrolig praktisk funksjon i Python er *negativ indeksering*. Hvis du vil ha tak i det aller siste elementet i en lang liste (uten å måtte telle nøyaktig hvor lang den er), kan du be om indeks `[-1]`. Det nest siste elementet er `[-2]`, og så videre.

Nyttige listemetoder: Når vi har opprettet en liste, har Python flere innebygde verktøy (metoder) for å endre på den.

- `.append(x)`: Legger til *ett* nytt element helt sist i listen.
- `.extend(liste)`: Slår sammen to lister ved å ta alle elementene fra en annen liste, og legge dem til sist i den opprinnelige.
- `.pop(indeks)`: Fjerner og "spytter ut" et element fra listen. Ligger parenteser tom (`.pop()`), fjerner den automatisk det aller siste elementet.

Eksempel: Indeksering og listemetoder

```
# Vi oppretter en liste med aksjetickere
portefolje = ["EQNR", "DNB", "YAR", "TEL"]

# --- INDEKSERING OG SLICING ---
forste_aksje = portefolje[0]
siste_aksje = portefolje[-1] # Henter siste (TEL)

# Slicing: Henter ut de tre første (fra start, til og ikke med 3)
topp_tre = portefolje[0:3]

# --- METODER FOR Å ENDRE LISTEN ---
# 1. Legger til ÉN ny aksje med .append()
portefolje.append("MOWI")

# 2. Legger til FLERE aksjer samtidig med .extend()
nye_aksjer = ["SALM", "NHY"]
portefolje.extend(nye_aksjer)

# 3. Fjerner den aller siste aksjen fra listen med .pop()
# Dette fjerner NHY, men vi kan også lagre den i en variabel!
fjernet_aksje = portefolje.pop()

print(f"Første: {forste_aksje} | Siste fra original: {siste_aksje}")
print(f"Topp tre: {topp_tre}")
print(f"Fjernet aksje: {fjernet_aksje}")
print(f"Hele porteføljen til slutt: {portefolje}")
```

```
> Første: EQNR | Siste fra original: TEL
> Topp tre: ['EQNR', 'DNB', 'YAR']
> Fjernet aksje: NHY
> Hele porteføljen til slutt: ['EQNR', 'DNB', 'YAR', 'TEL', 'MOWI',
'SALM']
```

4.2 Tupler (Tuples)

En tuple minner veldig om en liste, men med én helt avgjørende forskjell: Den er *imuterbar* (immutable). Det betyr at når en tuple først er opprettet, kan den **aldri** endres. Du kan ikke legge til, slette eller justere verdiene inni den. Tupler defineres med vanlige parenteser ().

Hvorfor bruke tupler når lister virker mer fleksible? Økonomer bruker ofte tupler for data som under ingen omstendigheter bør endres ved et uhell underveis i programmet, for eksempel faste GPS-koordinater, RGB-fargekoder for en graf, eller et fastlåst forhold mellom pris og antall fra en historisk transaksjon. I tillegg er tupler marginalt raskere for datamaskinen å lese enn lister.

Eksempel: Transaksjon lagret som en tuple

```
# Oppretter en tuple med (Pris, Antall_solgt)
transaksjon = (150.50, 2000)

pris = transaksjon[0]
print(f"Salgsprisen var: {pris}")

# Hvis vi prøver å kjøre: transaksjon[0] = 160.00
# Vil Python krasje og gi oss en TypeError fordi tupler er låst!

> Salgsprisen var: 150.5
```

4.3 Oppslagsverk (Dictionaries / Dicts)

Lister og tupler er fantastiske når dataene har en fast rekkefølge. Men hva om vi vil slå opp informasjon basert på et *navn* i stedet for et tall (indeks)? Da bruker vi en **dictionary** (ofte forkortet **dict**).

En dictionary er bygd opp av "Nøkkel-Verdi"-par (Key-Value pairs). Hvis du slår opp på en nøkkel, får du den tilhørende verdien. Vi definerer dem med krøllparenteser {}.

Eksempel: Aksjekurser i en dictionary

```
# Oppretter en dict med Ticker (Nøkkel) og Pris (Verdi)
aksjekurser = {
    "EQNR": 295.50,
    "DNB": 210.00,
    "YAR": 350.20
}

# Slår opp prisen til DNB
dnb_pris = aksjekurser["DNB"]

# Legger til et helt nytt selskap i oppslagsverket
aksjekurser["ORK"] = 85.40

print(f"Prisen på DNB-aksjen er {dnb_pris} kr")
print(aksjekurser)

> Prisen på DNB-aksjen er 210.0 kr
> {'EQNR': 295.5, 'DNB': 210.0, 'YAR': 350.2, 'ORK': 85.4}
```

4.4 NumPy - Numerisk matematikk

Selv om Python sine vanlige lister er fleksible, har de et stort problem for oss som jobber med kvantitativ analyse: De er trege, og de støtter ikke standard vektorisert matematikk (akkurat hva det betyr kommer vi tilbake til).

Hvis vi har en vanlig liste med tall og prøver å gange listen med 2, vil ikke Python doble verdiene i listen. I stedet vil Python *repetere* hele listen to ganger! Her kommer biblioteket **NumPy** (Numerical Python) inn som en reddende engel [2]. NumPy konverterer listene våre til såkalte **arrays**, som gjør det mulig å utføre matematikk på millioner av tall samtidig (vektorisering), uten å måtte skrive lange, trege løkker.

Eksempel: Vanlig liste vs. NumPy Array

```
import numpy as np

# Vi har en vanlig liste med kontantstrømmer
kontantstrommer = [-1000, 200, 300]

# Forsøk 1: Vanlig liste ganget med 2
liste_feil = kontantstrommer * 2

# Forsøk 2: Vi gjør listen om til et NumPy-array først!
cf_array = np.array(kontantstrommer)
array_riktig = cf_array * 2

print("Når vi ganger en vanlig liste med 2:")
print(liste_feil)

print("\nNår vi ganger et NumPy-array med 2:")
print(array_riktig)

# Vi kan også bruke NumPy sine innebygde, lynraske funksjoner
total_sum = cf_array.sum()
print(f"\nSummen av kontantstrømmen: {total_sum}")
```

```
> Når vi ganger en vanlig liste med 2:
> [-1000, 200, 300, -1000, 200, 300]

> Når vi ganger et NumPy-array med 2:
> [-2000  400  600]

> Summen av kontantstrømmen: -500
```

Visste du at?

NumPy er kjernen i nesten all dataanalyse og kunstig intelligens i Python. Grunnen til at det er så ekstremt raskt, er at selve regneoperasjonene under panseret egentlig ikke skrives i Python, men i programmeringsspråket **C**. Python fungerer bare som et brukervennlig "styringspanel" for den raske C-koden!

4.5 Caseoppgave

KAPITTELCASE 4: Datastrukturer og NumPy

Fagansvarligs tips: I programmering handler mye om å velge riktig verktøy for jobben. Lister er fleksible arbeidshester, dictionaries er fantastiske for raske oppslag, tuples er trygge fordi de ikke kan endres, og NumPy fungerer som en superdata-maskin for tallknusing. Her skal vi se hvordan de brukes sammen!

Scenariet: Aksjeanalytikeren

Du jobber som dataanalytiker for et meglerhus. Hver morgen får du inn enorme mengder finansiell data. Noe av informasjonen er bare tekst og merkelapper, mens andre deler består av titusenvis av historiske aksjepriser som du må gjøre raske matematiske beregninger på. For å bygge et robust program, må du fordele dataene i riktig datastruktur.

Del 1: Velg riktig verktøy

Koble riktig datastruktur (**Liste**, **Tuple**, **Dictionary** eller **NumPy Array**) til de fire behovene under:

1. Du skal koble en aksjeticker (f.eks. "EQNR") direkte opp mot det fulle selskapsnavnet ("Equinor ASA"), slik at du lynraskt kan slå opp navnet senere.
2. Du skal lagre GPS-koordinatene (breddegrad, lengdegrad) til Oslo Børs. Dette er verdier du vil "låse" slik at ingen deler av programmet kan endre dem ved et uhell.
3. Du får inn en strøm av nye sluttkurser hver dag. Du trenger en enkel struktur for å samle disse, der du lett kan legge til (**append**) nye verdier på slutten.
4. Du har 5 000 historiske dagskurser, og må trekke fra 10 kr på alle sammen, samt regne ut standardavviket. Alt dette må skje på et brøkdels sekund.

Del 2: Fra liste til tallknuser (NumPy)

Nedenfor ser du et lite skript som henter inn tre aksjepriser som en vanlig Python-liste. Du skal gjøre denne om til en NumPy-array, og deretter gange alle prisene med 1.10 for å simulere en 10 % prisøkning. Fyll inn de tomme strekene (-----).

```
# Importerer numpy med det vanlige kallenavnet
import numpy as -----

# Dataen vi får inn er en standard Python-liste
priser_liste = [100, 150, 200]

# Gjør om listen til en NumPy-array for å kunne gjøre matematikk
priser_array = np.-----(priser_liste)

# Simulerer at alle aksjene stiger med 10 %
nye_priser = priser_array * -----

print(nye_priser)
```

Del 3: Oppslag i en Dictionary

Meglerhuset bruker en *dictionary* for å holde oversikt over hvilke selskaper som tilhører hvilken ticker. Fyll inn koden for å slå opp en eksisterende verdi, og for å legge til et helt nytt selskap ("TEL" for "Telenor ASA").

```
selskaper = {"EQNR": "Equinor ASA", "DNB": "DNB Bank ASA"}

# Slår opp navnet på DNB-aksjen
bank_navn = selskaper["-----"]

# Legger til Telenor i ordboken vår
selskaper["-----"] = "Telenor ASA"
```

Del 4: Den store multiplikasjonsfellen!

Et av de vanligste (og mest frustrerende) problemene nybegynnere møter på i Python, skjer når man forveksler lister og NumPy-arrays i matematikk.

Anta at vi har to variabler:

```
vanlig_liste = [1, 2, 3]
numpy_array = np.array([1, 2, 3])
```

Hva blir resultatet hvis du printer ut:

1. `vanlig_liste * 2` ?
2. `numpy_array * 2` ?

Hint: Husker du hva som skjer hvis du ganger en *tekststreng* med 2 i Python (f.eks "Hei" * 2)? Python behandler vanlige lister på akkurat samme måte! Forklar hvorfor det er kritisk å gjøre om finansielle data til NumPy-arrays før man begynner analysen.

5 Pandas og databehandling

Når økonomer sier de jobber med Python, jobber de egentlig i svært stor grad med pakken **Pandas**. Pandas er et bibliotek spesialbygd for å håndtere paneldata og tabulær data [2]. Kjerneobjektet i Pandas er en såkalt **DataFrame**, som best kan beskrives som et overlegent, programmerbart Excel-regneark.

For å bruke Pandas, må vi først installere det (ofte via `%pip install pandas` i Jupyter), og deretter importere det øverst i koden vår. Vi gir det nesten alltid aliaset `pd`.

5.1 Anatomien til en DataFrame

En **DataFrame** består av tre hovedkomponenter: En **Indeks** (radnavn/radnummer), **Kolonner** (variabelnavn), og selve **Dataene**. Hver enkelt kolonne i en **DataFrame** kalles i Pandas-terminologi for en **Series**.

	År	BNP_vekst	Inflasjon
0	2021	3.1	2.0
1	2022	1.2	5.8
2	2023	0.8	4.5

Figur 5: Strukturen til en Pandas **DataFrame**. Den består av rader (indeks), kolonner, og under panseret er hver kolonne et NumPy-array.

5.2 Hvor bor dataene våre? (Filsystem og lagring)

Før vi kan analysere data, må vi forstå hvor dataene faktisk befinner seg.

Når du lagrer en Excel-fil eller en CSV-fil (Comma Separated Values) på datamaskinen din, skrives informasjonen fysisk inn på datamaskinens harddisk (SSD). For at operativsystemet (Windows eller macOS) skal finne tilbake til filen, organiseres den i et *filsystem* med mapper og undermapper.

Når vi skal be Python om å åpne en fil, må vi gi programmet den nøyaktige stien (filbanen) til der dataene bor:

- **Absolutt sti:** Den fulle adressen fra hjertet av harddisken din. F.eks:
`C:/Brukere/Markus/Dokumenter/data/okonomi.csv`.
- **Relativ sti:** Adressen til filen *relativt til mappen Python-skriptet ditt ligger i*. Hvis Jupyter-notatboken din ligger i samme mappe som datasettet, kan du bare skrive `"okonomi.csv"`.

5.3 Lese inn og inspisere data

I motsetning til Excel, hvor du visuelt blar gjennom tusenvis av rader, bruker vi i Pandas innebygde metoder for å få et raskt sammendrag av dataene våre [1].

Eksempel: Innlesing og inspeksjon av CSV

```
import pandas as pd

# Lese inn data fra en CSV-fil (ofte fra en url eller lokal filbane)
url = "https://raw.githubusercontent.com/uit-bed-1304-h26/.../data.csv"
df = pd.read_csv(url)

# .head() viser de 5 første radene i datasettet
print("Topp 5 rader:")
print(df.head())

# .info() forteller oss hvor mange rader, kolonner og
# hvilke datatyper (int, float, object/tekst) vi har
print("\nInformasjon om datasettet:")
df.info()

# .describe() gir oss statistiske nøkkeltall
# (snitt, min, max, standardavvik)
sammendrag = df.describe()

> Topp 5 rader:
> ... (Viser tabell) ...
> Informasjon om datasettet:
> <class 'pandas.core.frame.DataFrame'>
> RangeIndex: 100 entries, 0 to 99
> Data columns (total 3 columns): ...
```

5.4 Utvelgelse, filtrering og nye kolonner

En av de viktigste ferdighetene i Pandas er å kunne manipulere dataene. Vi kan hente ut spesifikke kolonner, opprette nye basert på matematiske utregninger (vektorisert matematikk med NumPy i bakgrunnen), og filtrere rader ved hjelp av logiske betingelser.

Eksempel: Manipulering av en DataFrame

```
# 1. Hente ut kun én kolonne (Returnerer en Series)
bnp_data = df["BNP_per_capita"]

# 2. Opprette en NY kolonne basert på matematikk fra en annen
df["BNP_i_tusen"] = df["BNP_per_capita"] / 1000

# 3. Filtring: Beholde kun land med høyere utdanning enn 12 år
# Vi putter en logisk sjekk inn i klammeparentesene!
hoy_utdanning_df = df[df["Utdanning"] > 12]

# 4. Sortering: Finne de 10 landene med høyest BNP
topp_10 = df.sort_values(by="BNP_per_capita", ascending=False).head(10)

# Dataframen er nå endret og oppdatert i minnet!
```

Original DataFrame (df) før manipulering

	Land	BNP_per_ca	Utdanning	Steg 1	BNP_per_capita
0	Norge	90000	14		90000
1	Sverige	60000	13		60000
2	Land X	20000	10		20000

↓ Steg 2: `df["BNP_i_tusen"] = ...`
bnp_data (Series)

	Land	BNP_per_ca	Utdanning	BNP_i_tusen
0	Norge	90000	14	90.0
1	Sverige	60000	13	60.0
2	Land X	20000	10	20.0

↓ Steg 3: `hoy_utdanning_df = df[df["Utdanning"] > 12]`
(Rad 2 fjernes fordi 10 ikke er større enn 12)

	Land	BNP_per_ca	Utdanning	BNP_i_tusen
0	Norge	90000	14	90.0
1	Sverige	60000	13	60.0

Ny DataFrame (hoy_utdanning_df) etter filtrering

Figur 6: Visuell fremstilling av hvordan koden i eksempelet over trekker ut, modifierer og filtrerer data. Legg spesielt merke til hvordan Pandas bevarer de opprinnelige indekssnumrene (0 og 1) i den filtrerte datarammen nederst.

5.5 Henting av data fra eksterne kilder (Databaser og API)

I eksemplene over brukte vi ferdige CSV-filer. Men i arbeidslivet, spesielt for økonomer og analytikere, kommer data sjelden i ferdige filer på e-post. Dataene ligger ofte lagret på eksterne servere, for eksempel i store relasjonsdatabaser (SQL), eller de gjøres tilgjengelige via et **API** (Application Programming Interface).

Et API er en "servitør" som lar ditt Python-program snakke med en annen bedrifts server og bestille data direkte inn i koden din.

Eksempel: Statistisk Sentralbyrå (SSB)

SSB har et åpent API. I stedet for å gå inn på nettsidene deres, klikke i menyer og laste ned en Excel-fil manuelt hver måned, kan vi be Python gjøre en nettverksforespørsel (**request**) og laste dataene direkte inn i en Pandas DataFrame. Slik kan vi bygge automatiserte makroøkonomiske dashboards!

Slik fungerer konseptet i praksis (merk at for SSBs mest avanserte tabeller brukes ofte pakken **pyjstat**, eller modulen **requests** for å sende en dataspørring, men prinsippet er at koden henter ferske tall via internett hver gang den kjøres):

Konseptuelt Eksempel: Hente ferske data via API

```
import requests
import pandas as pd

# 1. Vi definerer nettadressen (API-endepunktet) til SSB
api_url = "https://data.ssb.no/api/v0/no/table/03013"

# 2. Vi formulerer en spørring (en JSON-tekst) for å si nøyaktig
# hvilke variabler vi vil ha (f.eks. konsumprisindeksen).
query = {
    "query": [{"code": "Konsumpris", "selection": {"filter": "all",
    "values": ["*"]}]],
    "response": {"format": "json-stat2"}
}

# 3. Vi "ringer" servitøren (API-et) og poster forespørselen vår
respons = requests.post(api_url, json=query)

# (Etter dette gjøres responsen om til en Pandas DataFrame)
# Neste gang koden kjøres, får du automatisk morgendagens tall!

> Data successfully fetched from API (Status code: 200)
```

Kombinasjonen av å kunne programmere API-uttrekk automatisk med **requests**, behandle tabellene i **Pandas**, og visualisere dem med **Matplotlib**, er en nyttig *skill* å kunne anvende i mange ulike fagfelt og jobber.

5.6 Caseoppgave

KAPITTELCASE 5: Pandas og datavask

Fagansvarligs tips: I den virkelige verden er data nesten alltid rotete. Før vi kan bygge kule grafer eller kjøre avanserte simuleringer, må vi rydde opp (datavask). Pandas er selve sveitserkniven for dette arbeidet. I denne casen skal du hjelpe sjefen med å temme et kranglete datasett.

Scenariet: Det nye kassesystemet

Kaféen har fått et nytt kassesystem, og sjefen din har eksportert salgsdataene for de to siste dagene. Hun er frustrert og kaller deg inn på kontoret: *Jeg trenger å vite den totale bruttoomsetningen for hver enkelt kategori! Men systemet har registrert returer (negative antall), og på grunn av en feil mangler noen av varene pris (NaN). Hvordan fikser vi dette?*

Slik ser de rå, ubehandlede dataene ut i Pandas (df):

Dato	Kategori	Antall	Pris
01.03	Kaffe	5	35.0
01.03	Bakst	-1	45.0
02.03	Te	2	NaN
02.03	Kaffe	10	35.0

Del 1: Datavask med Pseudokode

Forklar med vanlige ord (pseudokode) hvilke tre steg vi må gjøre for å gå fra tabellen over, til en ferdig oversikt som kun viser summen av omsetningen per kategori.

- *Hint:* Du må fjerne to rader av ulike grunner, lage en ny kolonne med matematikk, og til slutt gruppere alt sammen.

Del 2: Skriv Pandas-koden

Fyll inn de tomme strekene (-----) slik at koden utfører planen din:

```
import pandas as pd

# 1. Filtrer bort returer (behold bare rader der Antall er større enn 0)
df = df[df["-----"] > 0]

# 2. Fjern rader som mangler pris (NaN)
df = df.dropna(subset=["-----"])

# 3. Lag en ny kolonne for Omsetning (Antall ganger Pris)
df["Omsetning"] = df["-----"] * df["-----"]

# 4. Grupper på Kategori og finn summen av Omsetning
kategori_salg = df.groupby("-----")["Omsetning"]._____( )

print(kategori_salg)
```

Ekstraoppgave: Les og forstå AI-koden

Litt senere på dagen ønsker sjefen å finne ut *hvilken dato* kaféen solgte for mest penger totalt. Hun ber ChatGPT om hjelp, og får koden du ser under. ChatGPT valgte en litt annen tilnærming til manglende verdier enn du gjorde i Del 2.

Din oppgave: En dataanalytiker må alltid kvalitetssikre kode skrevet av kunstig intelligens før den tas i bruk. Legg inn en kommentar (#) på **hver eneste linje** i koden under, der du forklarer nøyaktig hva Pandas gjør med dataene på akkurat den linjen.

```
df_ren = df.fillna(0)
# Forklaring:

df_ren["Omsetning"] = df_ren["Antall"] * df_ren["Pris"]
# Forklaring:

daglig_salg = df_ren.groupby("Dato")["Omsetning"].sum()
# Forklaring:

beste_dag = daglig_salg.idxmax()
# Forklaring:

print(beste_dag)
# Forklaring: Skriver ut resultatet på skjermen.
```

Refleksjon: Ser du noen problemer med ChatGPT sin tilnærming (`fillna(0)`) sammenlignet med din metode (`dropna`) i dette spesifikke tilfellet? Hva skjer hvis man erstatter en manglende *pris* med 0 kroner og ganger det med et antall? Gjør det noen skade på totalsummen?

6 Logikk og beslutninger

Frem til nå har programmene våre vært ganske ”dumme”. De har startet på linje 1, og kjørt hver eneste linje slavisk nedover til bunnen. Men for å bygge reelle økonomiske modeller eller analyseverktøy, må programmet vårt kunne ta *avgjørelser* underveis, basert på dataene det får inn.

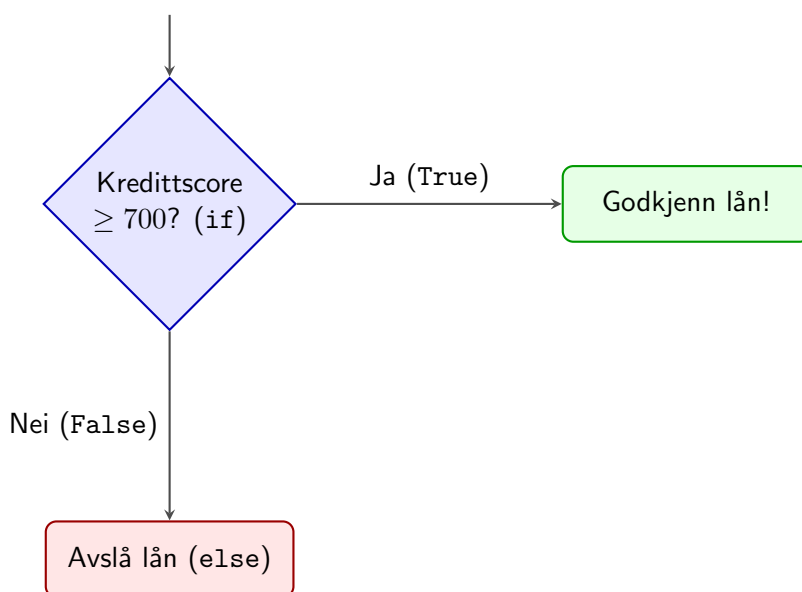
Betinget logikk handler om å la koden velge ulike veier avhengig av om en påstand er `True` (sann) eller `False` (usann).

6.1 Flytskjema: Slik tenker datamaskinen

Før vi ser på selve koden, er det svært nyttig å visualisere hvordan betinget logikk fungerer i praksis ved hjelp av flytskjemaer. Et flytskjema viser hvilken vei datamaskinen ”tenker” og beveger seg nedover i programmet. Hver diamant representerer et spørsmål (en logisk betingelse). Svaret på spørsmålet (enten `True` eller `False`) avgjør hvilken ”boks” (kodeblokk) datamaskinen velger å kjøre.

1. Enten/eller (`if` / `else`)

Den mest grunnleggende formen for beslutning er et enkelt veidele. Hvis påstanden er sann, gjør vi én handling. Hvis påstanden er usann, utfører vi *alltid* alternativet (`else`). Figur 7 viser en klassisk kredittsjekk for utstedelse av lån.



Figur 7: Flytskjema for en `if` / `else`-struktur. Datamaskinen blir tvunget til å velge én av to veier.

Definisjon: If/else-betingelse

En **if/else**-setning brukes i programmering for å utføre ulike kodeblokker avhengig av om en logisk betingelse er sann (**True**) eller falsk (**False**).

Generell syntaks i Python:

```
if betingelse:
    # kode som kjører hvis betingelsen er sann (True)
else:
    # kode som kjører hvis betingelsen er falsk (False)
```

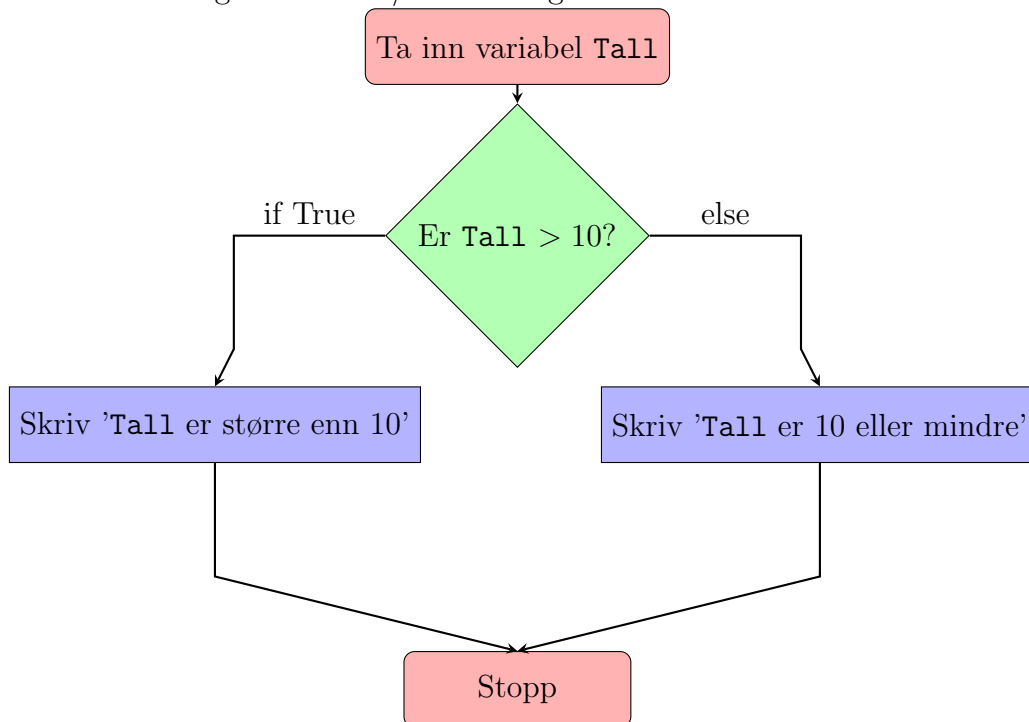
Enda et eksempel med flytskjema

La oss si at vi får følgende i oppgave:

Oppgave

Ta inn en variabel **Tall**, og sjekk om tallet er over eller under 10.

Under gir vi et enkelt eksempel, hvor vi sjekker om en variabel **Tall** er større enn 10. Dette illustrerer logikken i en if/else-setning:



Dette eksempelet kan kanskje virke unødvendig, eller at dere tenker ”*hvorfor skal vi lære dette?*”

Men faktisk kan det være ganske nyttig med enkle **if/else-setninger**.

Økonomisk eksempel

I Norge (i 2025), er det en **frikortgrense** på 100 000 kr. Det betyr at, tjener du mindre enn 100 000 kr, trenger du ikke å betale skatt. Tjener du mer enn dette, **må** du betale skatt.

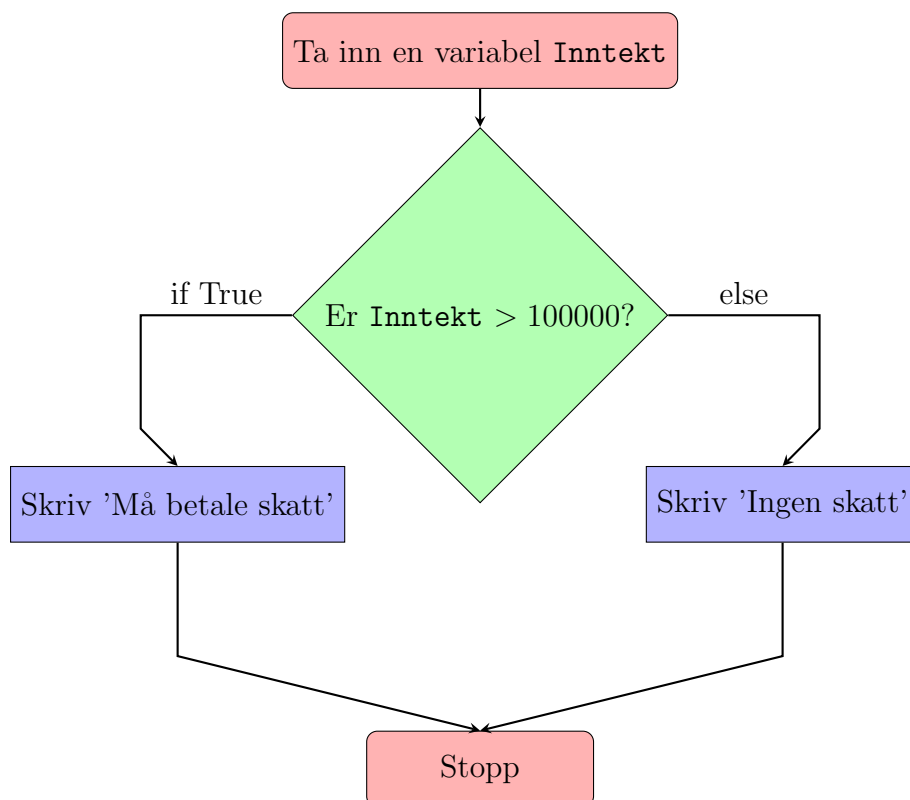
Dette er jo en **if/else-setning** i sin natur! La oss se på hvordan vi ville løst det ved hjelp av pseudokode, skisser og til slutt, hvordan vi kan skrive det som Python-kode.

Oppgave: Ta inn inntekt, og sjekk om personen må betale skatt eller ikke.

Pseudokode

- Les inn inntekt `Inntekt`
- Hvis `Inntekt > 100000`, skriv 'Må betale skatt'
- Ellers, skriv 'Ingen skatt'

Flytdiagram



Python-kode

Her er hvordan vi kan implementere eksemplet i Python:

```
# Leser inn inntekt fra bruker
inntekt = float(input("Skriv inn din inntekt: "))

# If/else-betingelse
if inntekt > 100000:
    print("Må betale skatt")
else:
    print("Ingen skatt")
```

Forklaring:

- 'input()' leser inn verdien fra brukeren.
- 'float()' gjør teksten om til tall slik at vi kan sammenligne den.
- 'if' sjekker om inntekten er over 100 000.
- 'else' håndterer tilfellene der inntekten er lik eller mindre enn 100 000.

Vi har flere måter å sjekke betingelser på

I if-setninger kan vi bruke ulike typer betingelser for å bestemme hvilke kodeblokker som skal kjøres.

Sammenligninger

Sammenligningsoperatorer brukes til å sammenligne verdier:

- == : lik
- != : ikke lik
- > : større enn
- < : mindre enn
- >= : større enn eller lik
- <= : mindre enn eller lik

Eksempel:

```
alder = 18
```

```
# Hvis alder er større eller lik 18
if alder >= 18:
    print("Stemmerett")
else:
    print("Ikke stemmerett")
```


Logiske operatorer

Logiske operatorer brukes til å kombinere flere betingelser:

- **and** : sann hvis begge betingelser er sanne
- **or** : sann hvis minst én betingelse er sann
- **not** : inverterer sannhetsverdien

Eksempel:

```
alder = 20
har_lappen = True

# Hvis alder er over eller lik 18, og har lappen
if alder >= 18 and har_lappen:
    print("Kan kjøre bil")
else:
    print("Kan ikke kjøre bil")
```

2. Flere alternativer (if / elif / else)

Ofte er ikke verden bare svart/hvitt. Hvis det første spørsmålet er usant, vil vi kanskje stille et *nytt* spørsmål før vi eventuelt gir opp. Til dette bruker vi **elif** (else if). Du kan ha så mange **elif**-diamanter du vil nedover i kjeden.

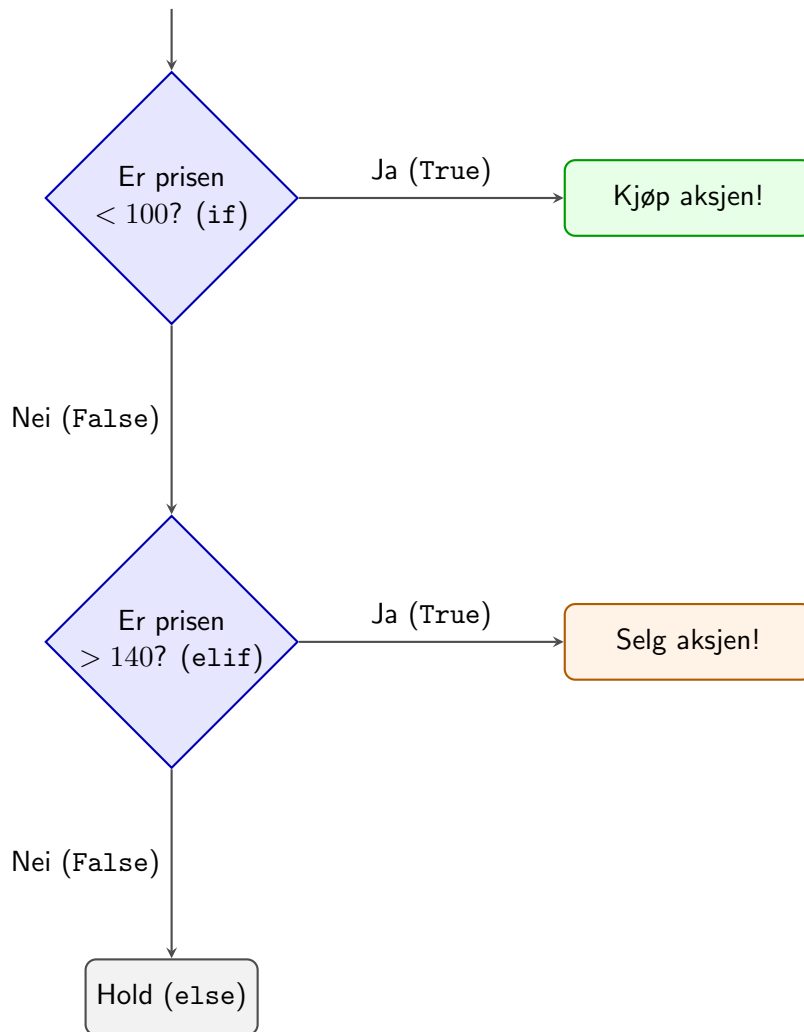
Definisjon: If/elif/else-betingelse

En **if/elif/else**-setning brukes i programmering for å utføre ulike kodeblokker, når vi har en logisk betingelse med **mer enn to utfall**.

Generell syntaks i Python:

```
if betingelse1:
    # kode som kjører hvis betingelse1 er sann
elif betingelse2:
    # kode som kjører hvis betingelse2 er sann
else:
    # kode som kjører hvis ingen av de overstående betingelsene er sanne
```

Figur 8 viser en enkel aksjrobot. Det viktigste å lære fra denne figuren er at programmet tester betingelser slavisk ovenfra og ned. **Så snart en betingelse er sann, utføres handlingen, og programmet hopper ut av hele systemet uten å sjekke resten!** **else** fungerer som en siste sikkerhetsnett som fanger opp alt som ikke var sant i trinnene over.



Figur 8: Flytskjema for en `if` / `elif` / `else`-struktur. Hvis prisen er 90, kjøper roboten. Er den 150, går vi ned til `elif`-leddet og selger. Er prisen 120, havner vi helt nederst i `else`-leddet.

6.2 Den enkle `if`-setningen og innrykk

Det enkleste vi kan gjøre, er å be Python kjøre en kodeblokk *kun dersom* en enkelt betingelse er oppfylt.

Viktig om syntaks: Når vi skriver en `if`-setning, må linjen alltid avsluttes med et kolon (:). I tillegg må all kode som "tilhører" denne `if`-setningen ha et **innrykk** (én Tab-tast eller fire mellomrom). Koden som ikke har innrykk, kjøres uansett.

Eksempel: Enkel if-setning og innrykk

```
budsjett = 500
pris = 450

if pris <= budsjett:
    # Denne koden har INNRYKK (indentation)
    # Den kjøres KUN hvis betingelsen over er True
    print("Du har råd til å kjøpe varen.")
    ny_saldo = budsjett - pris
    print(f"Din nye saldo er: {ny_saldo}")

# Denne koden har IKKE innrykk.
# Den kjøres uansett hva som skjedde i if-setningen!
print("Takk for at du brukte butikken vår.")

> Du har råd til å kjøpe varen.
> Din nye saldo er: 50
> Takk for at du brukte butikken vår.
```

6.3 Enten/eller: if og else

Svært ofte vil vi at programmet skal gjøre én ting hvis betingelsen er sann, men noe *annet* hvis den er usann. Til dette bruker vi nøkkelordet **else** (ellers). **else** skal aldri ha en egen logisk sjekk ved siden av seg, den fanger bare opp alt som if-setningen ikke fanget.

Eksempel: To veivalg med if og else

```
kredittscore = 650
krav_for_laan = 700

if kredittscore >= krav_for_laan:
    print("Søknad innvilget. Lånet utbetales.")
else:
    # Fanger opp alle situasjoner der scoren er UNDER 700
    print("Søknad avslått. Kredittscoren er for lav.")

> Søknad avslått. Kredittscoren er for lav.
```

6.4 Flere alternativer: elif

Hva om vi har mer enn to utfall? I Python bruker vi **elif** (en forkortelse for "else if"). Du kan ha så mange **elif**-blokker du vil mellom en **if** og en **else**.

OBS: Python sjekker blokkene ovenfra og ned. Så snart den finner en betingelse som er **True**, kjører den koden og hopper over resten av sjekkene!

Eksempel: Skatteklasser med if

```
inntekt = 850000

if inntekt > 1000000:
    print("Kategori: Toppskatt")
    skatt = inntekt * 0.45
elif inntekt > 600000:
    print("Kategori: Høy inntekt")
    skatt = inntekt * 0.35
elif inntekt > 300000:
    print("Kategori: Middels inntekt")
    skatt = inntekt * 0.25
else:
    print("Kategori: Lav inntekt (Frikort/Lav skatt)")
    skatt = inntekt * 0.10

print(f"Beregnet skatt: {skatt} kr")

> Kategori: Høy inntekt
> Beregnet skatt: 297500.0 kr
```

Tips: I eksempelet over, trenger vi ikke å sjekke om inntekten er under én million i `elif`-en, for hvis den hadde vært over én million, ville Python allerede stoppet på den første `if`-en!

6.5 Nøstet logikk (Kompliserte beslutninger)

Av og til er en beslutning avhengig av to eller flere variabler samtidig. Vi kan løse dette enten ved å bruke logiske operatører (`and` / `or`), eller ved å "nøste" en `if`-setning på innsiden av en annen `if`-setning. Dette krever at vi holder tunga rett i munnen med tanke på innrykk (indentation).

Eksempel: Avansert investeringslogikk

```
aksjepris = 95
portefolje_verdi = 10000
har_risikovilje = True

if aksjepris < 100:
    print("Prisen er god!")

    # Her starter en ny, innrykket "if" INNE i den første
    if portefolje_verdi > 5000 and har_risikovilje == True:
        print("-> Du har penger og risikovilje. Vi kjøper!")
    else:
        print("-> Du mangler enten kapital eller risikovilje. Kjøper ikke.")

else:
    # Denne "else"-en hører til den aller første "if"-en
    print("Prisen er for høy. Vi venter.")

> Prisen er god!
> -> Du har penger og risikovilje. Vi kjøper!
```

I eksempelet over brukte vi **and** for å kreve at *begge* betingelsene var oppfylt samtidig, i tillegg til at vi nøstet if-setningene inni hverandre.

Vi kan også bygge avansert logikk på én og samme linje ved å kombinere operatorene **or** (eller) og **not** (ikke). Dette lar oss snu på betingelser, noe som ofte gjør koden mer naturlig å lese.

La oss se på et eksempel der et selskap søker om statlig støtte. For å få støtte må selskapet **ikke** være konkurs, **og** de må enten drive med grønn teknologi **eller** være lokalisert i et distrikt. Her bruker vi parenteser for å gruppere logikken, akkurat slik du ville gjort i vanlig matematikk.

Eksempel: Logiske operatører med or og not

```
er_konkurs = False
grønn_teknologi = False
lokalisert_i_distriktet = True

# Vi bruker "not" for å sjekke at er_konkurs er usann (False),
# og parenteser for å si at minst én av de to andre må være sann (True).
if not er_konkurs and (grønn_teknologi or lokalisert_i_distriktet):
    print("Søknad innvilget: Selskapet kvalifiserer til støtte!")
else:
    print("Søknad avslått: Kravene er ikke oppfylt.")

> Søknad innvilget: Selskapet kvalifiserer til støtte!
```

Bruk av `not in` for å sjekke lister

Et annet svært vanlig og praktisk bruksområde for `not` i økonomi og finans, er når vi vil sjekke at en verdi *ikke* befinner seg i en bestemt samling av data, for eksempel en svarteliste over land eller blokkerte kundenumre.

I stedet for å skrive tungvint kode for å sjekke om landet mangler, kan vi bruke den svært lesbare kombinasjonen `not in`:

Eksempel: Sjekke lister med `not in`

```
# En liste med land selskapet vårt ikke har lov til å handle med
svartelistede_land = ["Land A", "Land B", "Land C"]

klient_land = "Norge"

if klient_land not in svartelistede_land:
    print(f"Transaksjon godkjent. {klient_land} er et tillatt marked.")
else:
    print("Transaksjon BLOKKERT: Landet er på sanksjonslisten!")

> Transaksjon godkjent. Norge er et tillatt marked.
```

For-løkker - iterasjon

Definisjon: For-løkke

En **for-løkke** brukes i programmering for å gjenta en kodeblokk et bestemt antall ganger, eller over elementene i en liste, et intervall eller en sekvens.

Generell syntaks i Python:

```
for element in sekvens:
    # kode som kjører for hvert element
```

Eksempel med oppgave

La oss si vi ønsker å skrive ut tallene fra 1 til 5.

Oppgave

Bruk en for-løkke til å skrive ut tallene 1, 2, 3, 4 og 5.

Dette kan vi skrive i Python på følgende vis.

Eksempel på en For-løkke

```
# Skriver ut tallene fra 1 til 5
for i in range(1, 6):
    print(i)
```

```
> 1
> 2
> 3
> 4
> 5
```

Forklaring:

- ‘range(1, 6)’ genererer tallene 1, 2, 3, 4, 5.
- For hver verdi av ‘i’ i sekvensen kjører koden i løkken.
- ‘print(i)’ skriver ut verdien av ‘i’ hver gang løkken kjører.
- Legg merke til at tallet 6 ikke skrives ut, fordi funksjonen `range(a, b)` teller **fra** **og med** a, og **opp til** (men ikke inkludert) b.

Tips: Prøv selv, og gjør det kjent med løkker.

For-løkke over en liste

Vi kan også iterere over elementene i en liste:

```
frukt = ["eple", "banan", "appelsin"]

for f in frukt:
    print(f)
```

Forklaring:

- Løkken kjører én gang for hvert element i listen ‘frukt’.
- Variabelen ‘f’ tar verdien til hvert element underveis i løkken.
- Her vil den printe følgende:
 - *eple*
 - *banan*
 - *appelsin*

While-løkker - grundigere forklart

Definisjon: While-løkke

En **while-løkke** brukes i programmering for å gjenta en kodeblokk så lenge en gitt betingelse er sann (**True**). Løkken fortsetter å kjøre inntil betingelsen blir falsk (**False**).

Generell syntaks i Python:

```
while betingelse:  
    # kode som kjører så lenge betingelsen er sann
```

Eksempel med oppgave

La oss si vi ønsker å skrive ut tallene fra 1 til 5 ved hjelp av en while-løkke.

Oppgave

Bruk en while-løkke til å skrive ut tallene 1, 2, 3, 4 og 5.

Python-kode

Eksempel på en For-løkke

```
# Skriver ut tallene fra 1 til 5  
i = 1  
while i <= 5:  
    print(i)  
    i += 1 # øk i med 1 hver gang løkken kjører
```

```
> 1  
> 2  
> 3  
> 4  
> 5
```

Forklaring:

- Vi starter med å sette `i = 1`.
- Løkken kjører så lenge `i <= 5` er sann.
- `print(i)` skriver ut verdien av `i`.
- `i += 1` øker `i` med 1 for hver iterasjon. Hvis vi ikke gjør dette, vil `while`-løkka kjøre for alltid...
- Når `i` blir 6, blir betingelsen `i <= 5` falsk, og løkken stopper.

While-løkke med liste

Vi kan også bruke while-løkker til å iterere over en liste:

Eksempel på en For-løkke

```
frukt = ["eple", "banan", "appelsin"]
i = 0

while i < len(frukt):
    print(frukt[i])
    i += 1
```

```
> 1
> 2
> 3
> 4
> 5
```

Forklaring:

- Vi bruker en indeks *i* for å holde styr på posisjonen i listen.
- `len(frukt)` gir lengden på listen.
- Løkken kjører så lenge *i* er mindre enn lengden på listen.
- `frukt[i]` henter elementet på posisjon *i*, og `i += 1` flytter til neste element.
- Dette vil printe:
 - *eple*
 - *banan*
 - *appelsin*

Økonomisk eksempel: Akkumulere inntekt med while-løkke

Oppgave: Vi ønsker å finne ut hvor mange måneder det tar å oppnå en total inntekt på minst 30 000 kr, gitt månedlige inntekter:

[3000, 3200, 3500, 2800, 3000, 3100, 3300, 3400, 3600, 3700, 3500, 3400]

Det kan vi løse i Python ved hjelp av en while-løkke.

Eksempel på en For-løkke

```
monthly_income = [3000, 3200, 3500, 2800, 3000, 3100,
                  3300, 3400, 3600, 3700, 3500, 3400]
total_income = 0
index = 0

while total_income < 30000 and index < len(monthly_income):
    # Legger til inntekten for den gjeldende måneden
    total_income += monthly_income[index]
    index += 1      # Går til neste måned

print(f"Total inntekt nådd: {total_income} kr etter {index} måneder.")
```

Total inntekt nådd: 32600 kr etter 10 måneder.

Forklaring av koden

- **monthly_income**: Liste med månedlig inntekt for 12 måneder.
- **total_income**: Starter på 0, akkumulerer inntektene.
- **index**: Holder styr på hvilken måned vi er på i listen.
 - Legg merke til at vi må starte med en verdi av **index** (tredje kodelinje: **index** = 0), og den defineres **før** while-løkken starter.

While-løkke:

- **while total_income < 30000 and index < len(monthly_income)**: Løkken kjører så lenge **total_income** er mindre enn 30 000 kr og vi ikke har gått gjennom alle månedene.
- Inne i løkken:
 - **total_income += monthly_income[index]**: Legger til inntekten for den gjeldende måneden.
 - **index += 1**: Øker indeksen for å gå videre til neste måned.

Utskrift: Etter løkken er ferdig, altså når betingelsen går fra **True** til **False**, skriver vi ut den totale inntekten som er oppnådd og etter hvor mange måneder.

Komponenter i while-løkker

Komponent	Beskrivelse	Eksempel
while	Starter løkken og definerer betingelsen for å fortsette.	<code>while total_income < 30000:</code>
Betingelse	Uttrykk som evalueres til True eller False .	<code>total_income < 30000</code>
Løkkeblokk	Koden som kjøres så lenge betingelsen er sann.	<code>total_income += monthly_income[index]</code>
break	Avslutter løkken når det ikke er nødvendig å fortsette.	<code>if total_income >= 30000: break</code>
continue	Hopper over resten av løkkens kode for den nåværende iterasjonen.	<code>continue</code>
index	Indeks, for å gi while -løkka et startpunkt og sikre at løkka ikke kjører for alltid.	<code>index = 0</code> (før løkka) <code>index +=1</code> (inni løkka)

6.6 Caseoppgave

KAPITTELCASE: Logikk og beslutninger

Fagansvarligs tips: En datamaskin er ekstremt lydig, men blottet for sunn fornuft. Den gjør nøyaktig det du ber den om, i nøyaktig den rekkefølgen du skriver det. I denne casen skal du bygge logikken for et pris- og fraktsystem, og du vil raskt se hvorfor rekkefølgen på spørsmålene dine (`if` / `elif`) er helt avgjørende.

Scenariet: Grossistens rabattsystem

Du utvikler et automatisk bestillingssystem for en grossistbedrift. Når en ny ordre kommer inn, skal programmet sjekke tre ting før fakturaen genereres:

1. **Sikkerhet:** Bedriften har ikke lov til å selge til land på en spesifikk sanksjonsliste.
2. **Volumrabatt:** Ordre over 10 000 kr får 15 % rabatt. Ordre over 5 000 kr får 10 % rabatt. Alle andre får 0 % rabatt.
3. **Frakt:** Frakten koster normalt 500 kr. Men hvis kunden er "VIP", **eller** ordreverdien er over 20 000 kr, er frakten gratis.

Del 1: Algoritmisk tenkning (Pólyas trinn 1 & 2)

Bruk penn og papir for å visualisere beslutningstreet.

- Hva bør være den *aller første* tingen programmet sjekker (den ytterste `if`-setningen) for å unngå å beregne rabatt på en ulovlig ordre?
 - I rabatt-beregningen, spiller det noen rolle om vi sjekker 10 000-grensen eller 5 000-grensen først? Hvorfor / hvorfor ikke?
-

Del 2: Fyll inn koden

Skjelettet til programmet er klart. Fyll inn de tomme strekene (-----) slik at systemet følger reglene beskrevet i scenariet. Pass på at du bruker riktige logiske operatører (or, not in, etc.).

```
sanksjonsliste = ["Land X", "Land Y", "Land Z"]

# Input for denne spesifikke ordren
kunde_land = "Norge"
ordre_verdi = 12000
er_vip = False

# 1. Sikkerhetssjekk
if kunde_land ----- sanksjonsliste:

    # 2. Volumrabatt
    if ordre_verdi > 10000:
        rabatt_prosent = 0.15
    ----- ordre_verdi > 5000:
        rabatt_prosent = 0.10
    -----:
        rabatt_prosent = 0.0

    # 3. Beregn frakt
    if er_vip == True ----- ordre_verdi > 20000:
        frakt = 0
    else:
        frakt = 500

    # Sluttberegning (kjøres kun hvis ordren er lovlig)
    total = (ordre_verdi * (1 - rabatt_prosent)) + frakt
    print(f"Ordre godkjent! Totalpris: {total} kr")

else:
    print("AVBRUTT: Kan ikke levere til dette landet.")
```

Del 3: Logisk kortslutning (Finn feilen!)

En av dine medstudenter har forsøkt å skrive rabatt-logikken på egen hånd, men kundene klager. En kunde som handlet for hele 15 000 kr klager over at hun bare fikk 10 % rabatt, ikke 15 %.

Se på koden til medstudenten din under:

```
ordre_verdi = 15000

if ordre_verdi > 5000:
    rabatt = 0.10
elif ordre_verdi > 10000:
    rabatt = 0.15
else:
    rabatt = 0.0
```

Din oppgave: Forklar nøyaktig hva som skjer inni datamaskinen når denne koden kjøres med en `ordre_verdi` på 15 000. Hvorfor kommer programmet *aldri* til å dele ut en rabatt på 15 %, uansett hvor mye kunden handler for?

Del 4: Rydd opp i AI-kode med "and"

Du ber ChatGPT skrive en logisk sjekk for å se om en ordre er kvalifisert for "Gull-status". AI-en gir deg en nøstet logikk som fungerer, men som er fryktelig stygg og unødvendig lang:

```
if kunde_aar > 5:
    if ordre_verdi > 50000:
        print("Kunde får Gull-status!")
    else:
        print("Ikke Gull-status.")
else:
    print("Ikke Gull-status.")
```

Hvordan kan du skrive om denne koden til å bli **én enkelt if / else-blokk** ved å bruke den logiske operatoren **and**? Skriv ned den forenklete løsningen din.

7 Datavisualisering med Matplotlib

Som mennesker er vi svært dårlige til å finne mønstre i store tabeller med tusenvis av tall, men vi fanger umiddelbart opp trender når vi ser en graf. Datavisualisering er derfor en kritisk ferdighet, både for å forstå dine egne data, og for å kunne kommunisere funnene dine til andre.

I Python er `Matplotlib` det desiderte standardbiblioteket for grafer og plotting. Mens vi bruker `Pandas` til å vaske og strukturere dataene, og `NumPy` for å gjøre raske beregninger, fungerer `Matplotlib` som "penselen" som maler resultatene våre på skjermen.

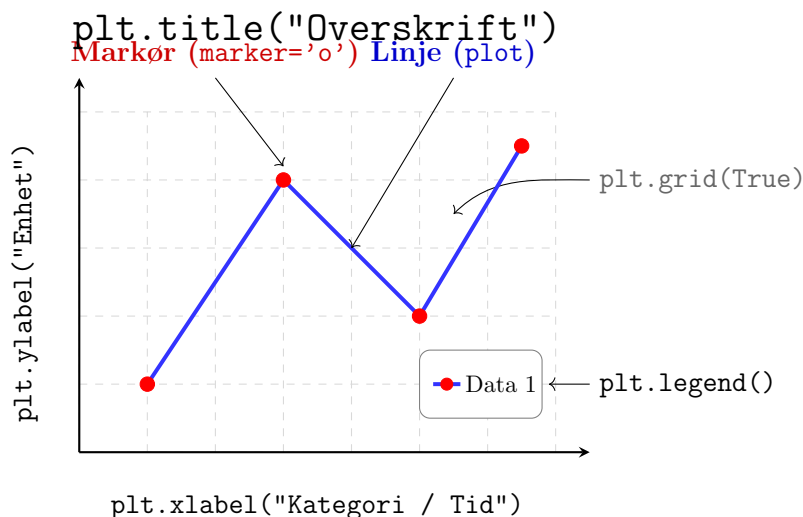
7.1 En god figur skal være selvforklarende

En gylden regel i all analyse – enten det gjelder finans, medisin eller biologi – er at en figur skal kunne stå helt på egne ben. Hvis noen finner figuren din uten å lese teksten rundt, skal de likevel forstå nøyaktig hva den viser.

For å oppnå dette, **må** figuren din inneholde følgende elementer:

- **Tittel (`title`):** Forteller kort og presist hva plottet viser.
- **Aksetitler (`xlabel` og `ylabel`):** Hva måler aksene, og i hvilken enhet? (For eksempel "Mill. NOK" eller "Vekt i kg").
- **Tegnforklaring (`legend`):** Helt nødvendig dersom du har flere linjer eller grupper av data i samme figur, slik at leseren vet hvilken farge som betyr hva.

I Figur 9 kan du se anatomien til et `Matplotlib`-plot, og hva de ulike delene kalles i koden.



Figur 9: Anatomien til en figur i `Matplotlib` og de tilhørende Python-kommandoene.

7.2 Linjeplot: Utvikling over tid (Økonomi)

Et linjeplot (`plt.plot()`) er best egnet når vi vil vise hvordan noe utvikler seg over tid (tidsserier). Et klassisk eksempel i økonomien er å plote profitt, aksjekurser eller BNP.

Eksempel: Linjeplot av profittutvikling

```
import matplotlib.pyplot as plt

# Definerer dataene våre i lister
aar = [2020, 2021, 2022, 2023]
profitt = [100, 150, 130, 200]

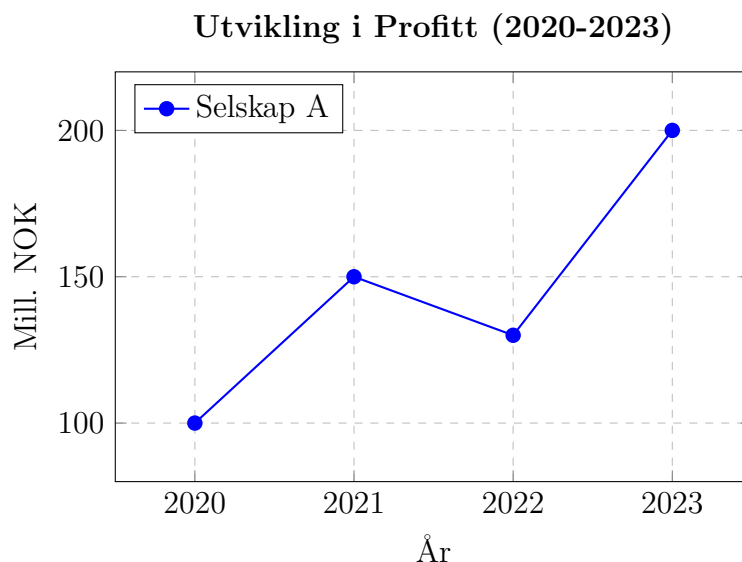
# Selve plottet. Vi ber om fargen blå, og en 'o' som markør på punktene
plt.plot(aar, profitt, marker='o', color='blue', label='Selskap A')

# Legger til tekst og struktur
plt.title("Utvikling i Profitt (2020-2023)")
plt.xlabel("År")
plt.ylabel("Mill. NOK")
plt.legend() # Viser boksen som forklarer at blå linje er "Selskap A"
plt.grid(True) # Slår på rutenettet i bakgrunnen

# Viser figuren på skjermen (svært viktig i mange editorer!)
plt.show()

> [Grafen genereres og vises som et bilde i output]
```

Slik vil grafen se ut når programmet kjøres. Legg spesielt merke til hvordan rutenett (grid), tegnforklaring (legend), og aksetitler gjør grafen lett å lese for mottakeren:



Figur 10: Resultatet av Matplotlib-koden over. Dette er et standard, vellaget linjeplot som oppfyller kravene til å være selvforklarende.

7.3 Punktplo (Scatter): Sammenhenger mellom to variabler (Biologi)

Hva om vi ikke har data over tid, men vi ønsker å se om det er en statistisk sammenheng (korrelasjon) mellom to helt forskjellige egenskaper? Da bruker vi et *scatter-plot* (spredningsplot).

La oss se på et tverrfaglig eksempel fra biologien. Vi har samlet inn data fra seks pingviner, og vi lurer på om pingviner som veier mye også har lengre vinger. Her gir det ingen mening å trekke en strek mellom punktene, så vi bruker `plt.scatter()` i stedet for `plt.plot()`.

Eksempel: Punktplo (Scatter) av biometriske data

```
import matplotlib.pyplot as plt

# Data fra måling av pingviner
vekt_kg = [3.2, 4.1, 3.8, 5.2, 4.8, 3.5]
vinge_cm = [18.5, 20.0, 19.2, 21.5, 20.8, 18.9]

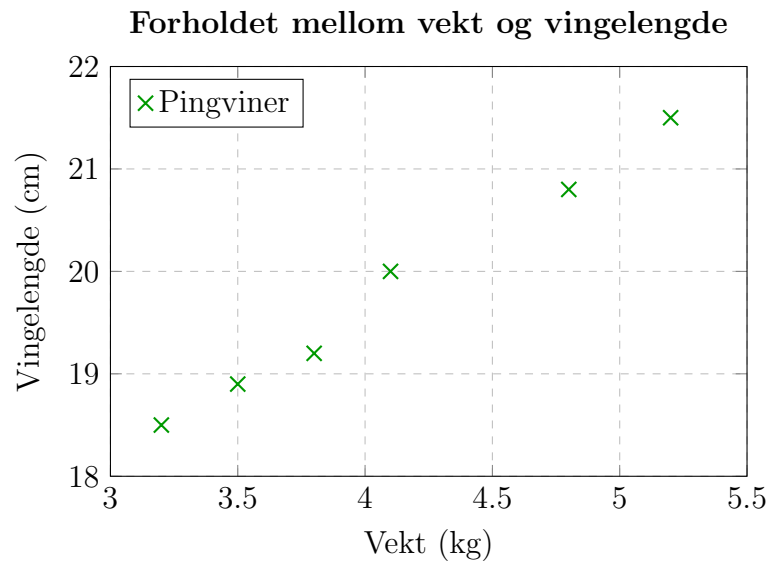
# Lager et punktplo. Endrer farge og bruker et kryss ('x') som markør
plt.scatter(vekt_kg, vinge_cm, color='green', marker='x', label='Pingviner')

# Gjør figuren selvforklarende
plt.title("Forholdet mellom vekt og vingelengde")
plt.xlabel("Vekt (kg)")
plt.ylabel("Vingelengde (cm)")
plt.legend()

plt.show()

> [Et punktplo vises, hvor man tydelig kan se at økt vekt gir økt vingelengde]
```

Slik vil punktplottet se ut når koden kjøres. Her ser vi tydelig verdien av å bruke markører (x) uten å trekke linjer, noe som gjør det lett å observere den positive sammenhengen mellom vekt og vingelengde:



Figur 11: Resultatet av `plt.scatter()`-koden over. Punktplottet gjør det enkelt å visuelt bekrefte hypotesen om at tyngre pingviner generelt har lengre vinger.

Tips: Stilisering av figurer

Matplotlib er ekstremt tilpasningsdyktig. Prøv å kjøre koden over i din egen Jupyter Notebook, men legg til linjen `plt.style.use('ggplot')` eller `plt.style.use('seaborn-v0_8-darkgrid')` helt øverst (etter importen). Du vil umiddelbart se at grafene dine får et svært profesjonelt og moderne fargetema, helt automatisk!

7.4 Caseoppgave

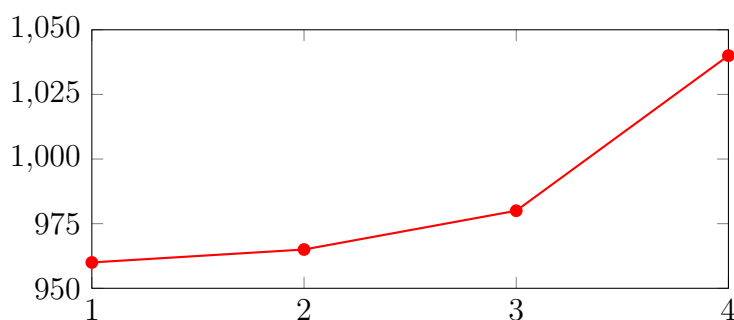
KAPITTELCASE 6: Matplotlib og Etikk

Fagansvarligs tips: Å lage en graf i Python er lett, men å lage en *god* og *ærlig* graf krever forståelse. Datavisualisering kan brukes til å lyse opp sannheten, men det er like ofte brukt til å manipulere den. I denne casen skal du rydde opp i et forsøk på å pynte på virkeligheten!

Scenariet: Den villedende pitchen

Du har nylig blitt ansatt som dataanalytiker i oppstartsselskapet *AppVekst AS*. Sjefen din skal ha et avgjørende møte med investorer i morgen, og viser deg stolt grafen han har tenkt å bruke på storskjermen for å bevise ”den eksplorative veksten vår”.

Han har tegnet opp grafen slik:



Del 1: Finn feilene (Kritisk analyse)

Du innser umiddelbart at hvis sjefen viser denne grafen til erfarne investorer, vil han bli ledd ut av rommet. Grafen bryter nesten alle regler for god datavisualisering. Finn frem penn og papir, og list opp **minst fire** alvorlige feil eller mangler med denne figuren.

- *Hint:* Se på figurens ”anatomi” fra teorikapitlet. Hva mangler for at den skal være selvforklarende?
- *Hint 2 (Etikk):* Se nøye på tallene på y-aksen. Hvis sjefen hevder ”eksplorative vekst”, hvor stor er den faktiske økningen i prosent fra punkt 1 til punkt 4? Er grafens *visuelle stigning* ærlig?

Del 2: Fiks grafen med kode

Du tar over tastaturet og bestemmer deg for å lage en ærlig og profesjonell versjon med Matplotlib. Fyll inn de tomme strekene (____) slik at grafen blir komplett, får et rutenett, og aller viktigst: tvinger y-aksen til å starte på 0!

```
import matplotlib.pyplot as plt

# Dataene til sjefen
maaneder = [1, 2, 3, 4]
brukere = [960, 965, 980, 1040]

plt.figure(figsize=(10, 6))

# Tegner opp selve linjen
plt._____(maaneder, brukere, marker='o', color='blue',
          label='Aktive brukere')

# Legger til nødvendig tekst (Gjør grafen selvforklarende!)
plt._____("Utvikling i antall aktive brukere (Q1)")
plt._____("Måned")
plt._____("Antall brukere")

# Fikser sjefens "juks" ved å tvinge y-aksen til å gå fra 0 til 1200
plt._____(0, 1200)

# Slår på tegnforklaring og rutenett
plt._____( )
plt._____(True)

plt.show()
```

Del 3: Flere linjer i samme graf

Investorene vil garantert spørre om hvordan det går med konkurrenten deres, *Hy-peApp*. Du har dataene for konkurrenten, som startet på 500 brukere og vokste til 1100 brukere i samme periode: `konkurrent = [500, 700, 900, 1100]`

Hvordan ville du lagt inn denne konkurrenten i den samme grafen din, før `plt.show()` kalles? Skriv ned den ene linjen med Python-kode du trenger for å tegne opp denne andre linjen, gjerne med fargen rød og en passende "label".

8 Symbolsk matematikk med SymPy

Mens vi bruker NumPy til *numerisk* tallknusing (hvor alt blir regnet ut og gjort om til desimaltall), brukes biblioteket **SymPy** til *symbolsk* matematikk.

SymPy fungerer akkurat slik du gjør når du regner med penn og papir. I stedet for å runde av $\frac{1}{3}$ til 0.3333, beholder SymPy det som en nøyaktig brøk. Med SymPy kan datamaskinen manipulere algebraiske uttrykk, løse komplekse ligninger, derivere og integrere – helt eksakt. Dette er ekstremt nyttig i fag som samfunnsøkonomi, matematikk og statistikk.

8.1 Symboler og algebra

Før vi kan bruke bokstaver som variabler i matematiske uttrykk, må vi fortelle Python at de er nettopp matematiske symboler, og ikke vanlige tekststrenger eller tomme bokser. Dette gjør vi med `sp.Symbol()`.

SymPy kan også hjelpe oss med å rydde opp i uttrykk, for eksempel ved å gange ut parenteser (*ekspandere*) eller trekke sammen uttrykk (*faktorisere*).

Eksempel: Symboler og utvidelse av parenteser

```
import sympy as sp

# 1. Definerer x og y som matematiske symboler
x = sp.Symbol('x')
y = sp.Symbol('y')

# 2. Oppretter et matematisk uttrykk: (x + y)^2
uttrykk = (x + y)**2

# 3. Ganger ut parentesene
ekspandert = sp.expand(uttrykk)

# 4. Faktorerer tilbake igjen
faktorisert = sp.factor(ekspandert)

print(f"Opprinnelig: {uttrykk}")
print(f"Ekspandert: {ekspandert}")
print(f"Faktorisert: {faktorisert}")

> Opprinnelig: (x + y)**2
> Ekspandert: x**2 + 2*x*y + y**2
> Faktorisert: (x + y)**2
```

Legg merke til at her har vi et uttrykk $(x+y)^2$ som etter ekspansjon, og faktorisering, blir til det samme uttrykket igjen.

Visste du at?

Hvis du trenger mange matematiske symboler i en oppgave, kan det bli mye unødvendig skriving å definere dem én og én med `sp.Symbol()`.

Heldigvis har SymPy noe kalt `sp.symbols()` (legg merke til s-en på slutten). Med denne kan du opprette flere symboler på én og samme linje ved å separere dem med et mellomrom i tekststrengen:

```
x, y, z = sp.symbols('x y z')
```

8.2 Løse ligninger og markedslikevekt

Når vi har definert symbolene våre, kan vi bruke funksjonen `sp.solve()` for å la Python løse ligninger for oss. Vi setter rett og slett opp ligningen slik at den er lik null, og mater den inn i funksjonen.

I mikroøkonomi er det klassiske eksempelet å finne markedslikevekt, altså den prisen og mengden der tilbudet er nøyaktig lik etterspørselen.

Eksempel: Løse for markedslikevekt

```
import sympy as sp

Q = sp.Symbol('Q')

# Definerer inverse tilbuds- og etterspørselsfunksjoner
# (Pris som funksjon av Mengde)
tilbud = 20 + Q
etterspørsel = 100 - Q

# Løser for mengden (Q) der tilbud = etterspørsel
# Vi flytter alt til én side: tilbud - etterspørsel = 0
likevekts_mengde = sp.solve(tilbud - etterspørsel, Q)[0]

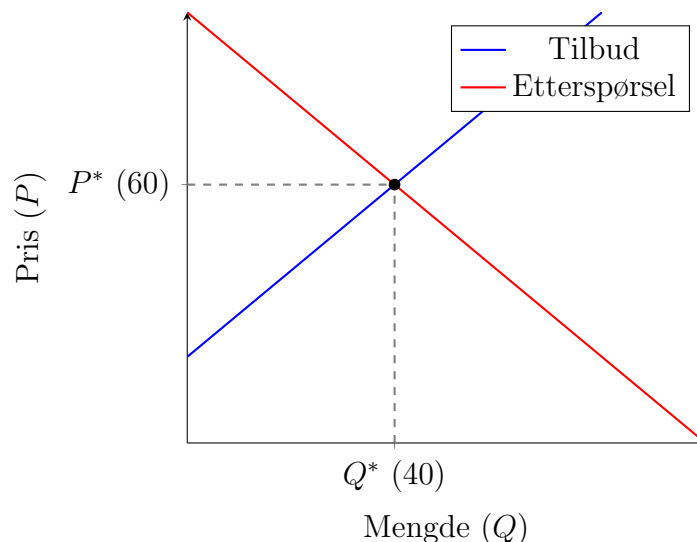
# Setter mengden inn igjen for å finne prisen (P)
likevekts_pris = tilbud.subs(Q, likevekts_mengde)

print(f"Likevektsmengde (Q*): {likevekts_mengde}")
print(f"Likevektspris (P*): {likevekts_pris}")

> Likevektsmengde (Q*): 40
> Likevektspris (P*): 60
```

Visuell fremstilling av markedslikevekten

Selv om Python finner tallet for oss, er det viktig å forstå den grafiske tolkningen av dette. Her har vi tegnet situasjonen fra koden over ved hjelp av et tradisjonelt kryss-diagram.

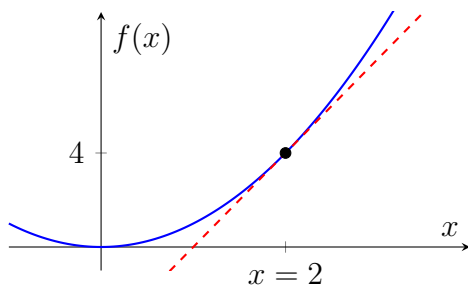


Figur 12: Markedslikevekt: Skjæringspunktet mellom tilbuds- og etterspørselskurven gir oss verdiene Python regnet ut for oss.

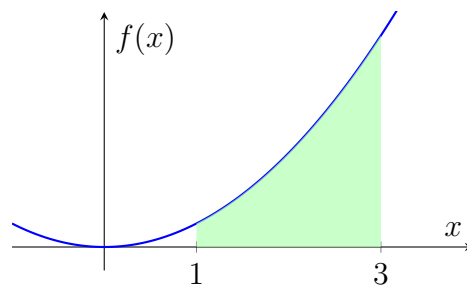
8.3 Kalkulus: Derivasjon og Integrasjon

I økonomi er vi ofte opptatt av endringer på marginen. For å finne en bedrifts profittmaksimum må vi vite hvor grenseinntekten er lik grensekostnaden. For å finne disse grenseverdiene må vi *derivere*.

Derivasjon handler om å finne stigningsmålet til en funksjon i et gitt punkt (tangenten). Integrasjon er det motsatte; det brukes for å regne ut *arealet* under en funksjon, for eksempel for å finne konsumentoverskuddet (arealet under etterspørselskurven og over prisen).



Figur 13: Derivasjon: Finner stigningen til den røde tangenten.



Figur 14: Integrasjon: Finner størrelsen på det grønne arealet.

Med SymPy slipper vi å huske alle brøk- og kjernereglene fra mattekurset, datamaskinen tar seg av derivasjonen ved hjelp av funksjonen `sp.diff()`, og integrasjonen med `sp.integrate()`.

Eksempel: Derivasjon og Integrasjon med SymPy

```
import sympy as sp

x = sp.Symbol('x')

# Definerer en matematisk funksjon:  $f(x) = x^3 + 2x$ 
f_x = x**3 + 2*x

# --- DERIVASJON ---
# Vi deriverer funksjonen med hensyn på x
derivert = sp.diff(f_x, x)

# --- INTEGRASJON ---
# 1. Ubestemt integral (antiderivert)
integral = sp.integrate(derivert, x)

# 2. Bestemt integral (Finner arealet under 'derivert'-kurven fra x=0 til x=5)
# Merk syntaksen: (variabel, start, slutt)
areal = sp.integrate(derivert, (x, 0, 5))

print(f"Opprinnelig funksjon f(x): {f_x}")
print(f"Den deriverte f'(x): {derivert}")
print(f"Integralet av den deriverte: {integral} (Tilbake til start!)")
print(f"Areal under f'(x) fra 0-5: {areal}")

> Opprinnelig funksjon f(x): x**3 + 2*x
> Den deriverte f'(x): 3*x**2 + 2
> Integralet av den deriverte: x**3 + 2*x (Tilbake til start!)
> Areal under f'(x) fra 0-5: 135
```

La oss sjekke dette matematisk!

For å bygge tillit til at Python og SymPy faktisk gjør det den skal, er det en veldig god øvelse å bekrefte utregningen med penn og papir (også kjekt for forståelsen, og repetisjon av mattekunnskaper).

I koden over ba vi SymPy om å regne ut det bestemte integralet av den deriverte funksjonen, $f'(x) = 3x^2 + 2$, med nedre grense $x = 0$ og øvre grense $x = 5$. Fra matematikken vet vi at vi løser dette ved å finne den antideriverte (som tar oss tilbake til $x^3 + 2x$), og deretter sette inn grensene:

$$\begin{aligned}\int_0^5 (3x^2 + 2) dx &= [x^3 + 2x]_0^5 \\ &= (5^3 + 2 \cdot 5) - (0^3 + 2 \cdot 0) \\ &= (125 + 10) - 0 \\ &= 135\end{aligned}$$

Vi ser at vår egen håndregning stemmer på med verdien SymPy ga oss i terminalen! Forskjellen er bare at når du senere møter på mer komplekse funksjoner i mikroøkonomien, vil SymPy fortsatt kunne raskt gi deg svaret, uten fare for regnefeil.

8.4 Caseoppgave

KAPITTELCASE 7: SymPy og profittmaksimering

Fagansvarligs tips: I økonomi leter vi nesten alltid etter et ”optimalt” punkt. For en bedrift er dette punktet der profitten er på sitt aller høyeste. Fra matematikken vet vi at vi finner toppen av en funksjon ved å derivere den og sette den lik null (eller sette grenseinntekt lik grensekostnad). I denne casen skal vi la SymPy ta seg av matematikken for oss.

Scenariet: Monopolisten

Du er analytiker for en bedrift som har monopol på produksjon av en spesiell type el-sykler. Du har estimert følgende funksjoner for markedet:

- **Etterspørsel (Pris):** $P = 100 - Q$
- **Totale kostnader:** $TC = 200 + 10Q + 0.5Q^2$

(Hvor Q er antall produserte sykler, og pris/kostnad er i tusen kroner).

Sjefen vil vite nøyaktig hvor mange sykler (Q^*) dere bør produsere for å maksimere profitten, og hvilken pris (P^*) dere skal ta for dem.

Del 1: Sett opp det matematiske problemet

Før vi kan kode, må vi definere hva profitt egentlig er, og hvordan vi maksimerer den. Finn frem penn og papir:

1. Profitt (π) er Totale Inntekter (TR) minus Totale Kostnader (TC). Hvordan regner man ut Totale Inntekter når man kjenner Pris (P) og Mengde (Q)?
2. Skriv ned uttrykket for TR basert på etterspørselsfunksjonen i scenariet.
3. Den gylne regelen for profittmaksimering er at grenseinntekt (MR) må være lik grensekostnad (MC). Hva er den matematiske sammenhengen mellom TR og MR (og TC og MC)?

Del 2: Fyll inn SymPy-koden

Vi skal nå la Python finne det optimale produksjonsnivået. Fyll inn de tomme strekene (-----) for å definere funksjonene, derivere dem, og løse $MR = MC$.

```
import sympy as sp

# 1. Definerer mengden Q som et matematisk symbol
Q = sp.-----('Q')

# 2. Legger inn funksjonene fra scenariet
P = 100 - Q
TC = 200 + 10*Q + 0.5*Q**2

# 3. Regner ut Totale Inntekter (TR = Pris * Mengde)
TR = P * -----

# 4. Deriverer for å finne grenseverdiene (MR og MC)
MR = sp.-----(TR, Q)
MC = sp.diff(-----, Q)

# 5. Løser ligningen MR = MC for å finne optimal mengde
# Vi flytter alt til én side av likhetstegnet: MR - MC = 0
optimal_Q = sp.-----(MR - MC, Q)[0]

# 6. Finner optimal pris ved å sette Q inn i etterspørselsfunksjonen
optimal_P = P.subs(Q, optimal_Q)

print(f"Optimal produksjon (Q*): {optimal_Q}")
print(f"Optimal pris (P*): {optimal_P}")
```

Del 3: Sjekk med penn og papir

For å bygge tillit til koden, skal vi sjekke at datamaskinen har regnet riktig.

1. Regn ut MR ved å derivere uttrykket ditt for TR fra Del 1 med hensyn på Q .
2. Regn ut MC ved å derivere uttrykket for TC fra scenariet med hensyn på Q .
3. Sett $MR = MC$ og løs ligningen for Q . Får du $Q = 30$?
4. Regn ut profitten bedriften får når de produserer dette optimale antallet.

9 Simulering og usikkerhet

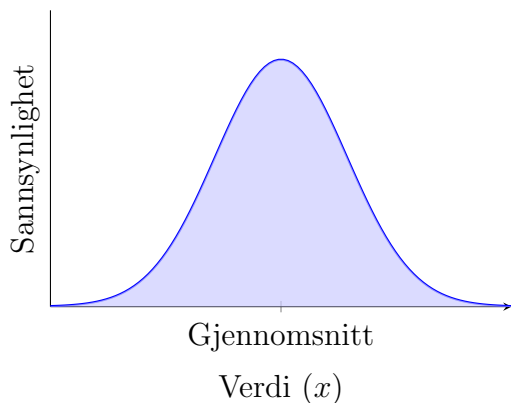
Vi har nå kommet til det punktet i kurset hvor vi skal kombinere nesten alt vi har lært: variabler, funksjoner, lister, løkker, NumPy og Matplotlib.

I virkeligheten vet vi sjelden nøyaktig hva som kommer til å skje i fremtiden. Hva blir inflasjonen neste år? Hvor mange kunder kommer innom butikken i morgen? Hva blir aksjeprisen om seks måneder? I stedet for å gjette på ett enkelt tall, bruker vi **simulering** for å modellere usikkerheten. Ved å la datamaskinen ”kaste terning” tusenvis av ganger basert på statistiske sannsynligheter, kan vi utforske et enormt utvalg av mulige fremtidsutfall.

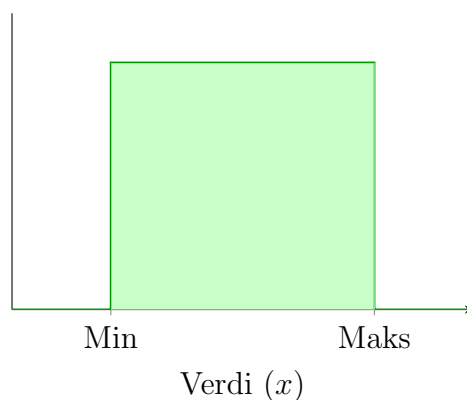
9.1 Tilfeldighet med NumPy

For å kunne simulere usikkerhet, må vi kunne be datamaskinen om å trekke tilfeldige tall. Til dette bruker vi under-modulen `np.random`. De to vanligste fordelingene vi trekker fra er:

- **Normalfordeling** (`np.random.normal`): Trekker tall som klumper seg rundt et gjennomsnitt (typisk en bjellekurve). Nyttig for å simulere for eksempel aksjeavkastning, høyde på mennesker eller testresultater.
- **Uniform fordeling** (`np.random.uniform`): Trekker desimaltall hvor alle verdier i et intervall har nøyaktig like stor sjanse for å bli trukket.



Figur 15: Normalfordeling (`normal`): Utfallene klumper seg rundt midten. Typisk for aksjeavkastning.



Figur 16: Uniform fordeling (`uniform`): Alle verdier innenfor intervallet har akkurat like stor sjanse.

Viktig: Replikerbarhet med `seed()`

Datamaskiner kan egentlig ikke lage *ekte* tilfeldige tall. De bruker kompliserte matematiske formler som bare *ser* tilfeldige ut. Hvis du vil at programmet ditt skal trekke nøyaktig de samme ”tilfeldige” tallene hver gang du kjører koden (noe som er ekstremt nyttig når du skal feilsøke, eller levere en eksamensbesvarelse), bruker vi `np.random.seed(et_tall)`.

Et klassisk eksempel: Terningkast (Diskret uniform fordeling)

Mens en kontinuerlig uniform fordeling trekker tilfeldige desimaltall, vil et terningkast trekke *heltall* fra 1 til 6. Siden alle de seks sidene på en rettferdig terning har nøyaktig like stor sjanse for å havne opp ($1/6$), kaller vi dette en *diskret uniform fordeling*.

For å trekke tilfeldige heltall i NumPy, bruker vi funksjonen `np.random.randint()`.

Når vi simulerer terningkast, får vi også oppleve **de store talls lov** i praksis: Hvis vi kaster terningen bare 10 ganger, vil fordelingen se skjev og rar ut. Men hvis vi lar data-maskinen kaste terningen 10 000 ganger, vil vi se at alle tallene trekkes nesten nøyaktig like mange ganger (ca. 1666 ganger hver), og grafen blir helt flat på toppen!

For å visualisere slike frekvenser, bruker vi et **histogram** (`plt.hist()`).

Eksempel: Simulering av 10 000 terningkast

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Simulerer 10 000 terningkast.
# OBS: I randint(start, stopp) er stopp-tallet IKKE inkludert.
# Vi må derfor skrive 1 til 7 for å få tallene 1, 2, 3, 4, 5, 6.
antall_kast = 10000
terningkast = np.random.randint(1, 7, size=antall_kast)

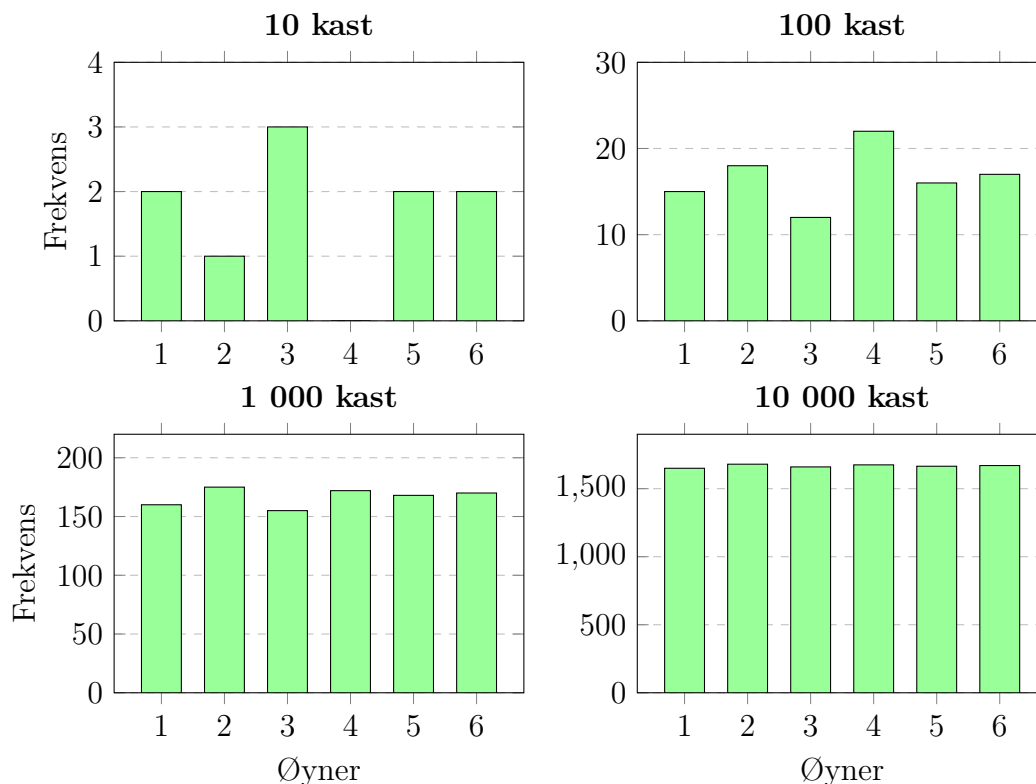
# Visualiserer fordelingen med et histogram
# 'bins' bestemmer grensene for hver søyle
plt.figure(figsize=(8,5))
plt.hist(terningkast, bins=[0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5],
        color='green', edgecolor='black', alpha=0.7)

plt.title(f"Fordeling ved {antall_kast} terningkast")
plt.xlabel("Øyner på terningen")
plt.ylabel("Antall ganger trukket")
plt.xticks([1, 2, 3, 4, 5, 6]) # Tvinger x-aksen til å bare vise heltall
plt.show()
```

```
> [Et histogram vises med seks like brede søyler (fra 1 til 6).
Alle søylene er nesten nøyaktig like høye og stopper på ca. 1660 på
y-aksen, noe som bekrefter at trekningen er uniform og rettferdig.]
```

For å virkelig forstå hvorfor vi trenger tusenvis av simuleringer for å fjerne støy, kan vi se på hva som skjer med fordelingen når vi gradvis øker antall terningkast (N). Dette fenomenet kalles (som nevnt over) **de store talls lov** i statistikken.

Når vi kaster terningen få ganger, vil tilfeldigheter spille en enorm rolle. Men etter hvert som N blir større, jevnes støyen ut, og vi nærmer oss den teoretiske, sanne fordelingen (som er at hver side skal trekkes nøyaktig $1/6$ av gangene).



Figur 17: De store talls lov i praksis. Legg merke til at for bare 10 kast er resultatet kaotisk (terningen landet for eksempel aldri på 4). Ved 10 000 kast er vi svært nær den teoretiske sannsynligheten, og alle søylene er praktisk talt like høye.

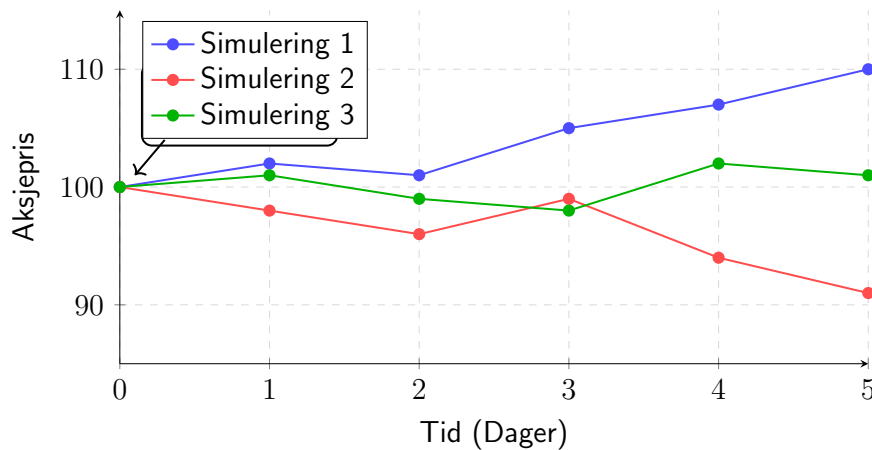
Prøv selv!

Kopier koden over inn i Jupyter Notebook. Prøv å endre `antall_kast` fra 10000 til 10. Kjør koden flere ganger (husk å fjerne `np.random.seed()`). Ser fordelingen fortsatt rettferdig ut? Dette forklarer hvorfor vi i finans og økonomi ofte må kjøre titusenvis av simuleringer for å få pålitelige svar!

9.2 Monte Carlo-simulering av aksjepriser

En av de mest kjente simuleringsmetodene i finans er *Monte Carlo-simulering* (oppkalt etter kasinoet i Monaco). Hvis vi ikke vet hva avkastningen til en aksje blir i morgen, men vi vet at den historisk sett har en forventet daglig vekst (snitt) og en viss volatilitet (standardavvik), kan vi simulere mange parallelle virkeligheter (prisbaner).

Figur 18 illustrerer konseptet visuelt: Vi starter på samme punkt i dag, men for hver dag som går, "slår" tilfeldighetene inn og skaper helt unike baner.



Figur 18: Konseptet bak Monte Carlo-simulering. Ved å legge til tilfeldig støy for hver tidsperiode, skaper vi et vifteformet utfallsrom som gir oss et inntrykk av risikoen fremover.

Når vi skal programmere dette i Python, krever det at vi bruker en **nøstet løkke** (en for-løkke inni en annen for-løkke). Den ytterste løkken bestemmer hvilken simuleringrunde vi er på (for eksempel simulering nr. 1 av 10). Den innerste løkken går gjennom hver enkelt dag i året og regner ut den nye prisen basert på gårsdagens pris og dagens tilfeldige avkastning.

Eksempel: Monte Carlo-simulering av aksjepriser

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Sett parametere (Variabler)
start_pris = 100
dager = 252          # Antall handelsdager i et år
simuleringer = 10     # Hvor mange parallelle virkeligheter vi vil lage
forventet_avk = 0.0005 # Ganske nøyaktig 0 % per dag (0.05%)
volatilitet = 0.02     # Risiko (Standardavvik på 2%)

# Låser tilfeldigheten slik at alle får samme graf!
np.random.seed(42)

plt.figure(figsize=(10,6))

# 2. Den ytterste løkken (kjører én gang per prisbane)
for i in range(simuleringer):

    # Trekker 252 tilfeldige avkastningstall fra en normalfordeling
    avkastning = np.random.normal(loc=forventet_avk,
                                   scale=volatilitet,
                                   size=dager)

    prisbane = [start_pris]

    # 3. Den innerste løkken (regner ut prisen dag for dag)
    for avk in avkastning:
        # Henter gårsdagens pris (siste element) og ganger med veksten
        ny_pris = prisbane[-1] * (1 + avk)
        prisbane.append(ny_pris)

    # Tegner opp akkurat denne prisbanen før vi starter på neste
    plt.plot(prisbane)

# 4. Gjør grafen selvforklarende
plt.title("Monte Carlo Simulering: 10 mulige fremtidsbaner")
plt.xlabel("Handelsdager")
plt.ylabel("Aksjepris (NOK)")
plt.grid(True)
plt.show()
```

> [En graf genereres som viser 10 ulike, "frynsete" prisbaner som alle starter på 100, men sprer seg utover som en vifte mot høyre.]

Det virkelige potensialet med Monte Carlo ligger ikke i å se på grafen med 10 streker, men å sette antall simuleringer til for eksempel 10 000 (ikke prøv å plote alle disse på

skjermen samtidig!). Deretter kan vi bruke NumPy til å trekke ut sluttprisene fra hver bane og spørre Python: *I hvor mange av disse 10 000 parallelluniversene endte porteføljen vår opp med å gå konkurs?* Dette gir oss svært presise og empiriske risikomål.

9.3 Caseoppgave

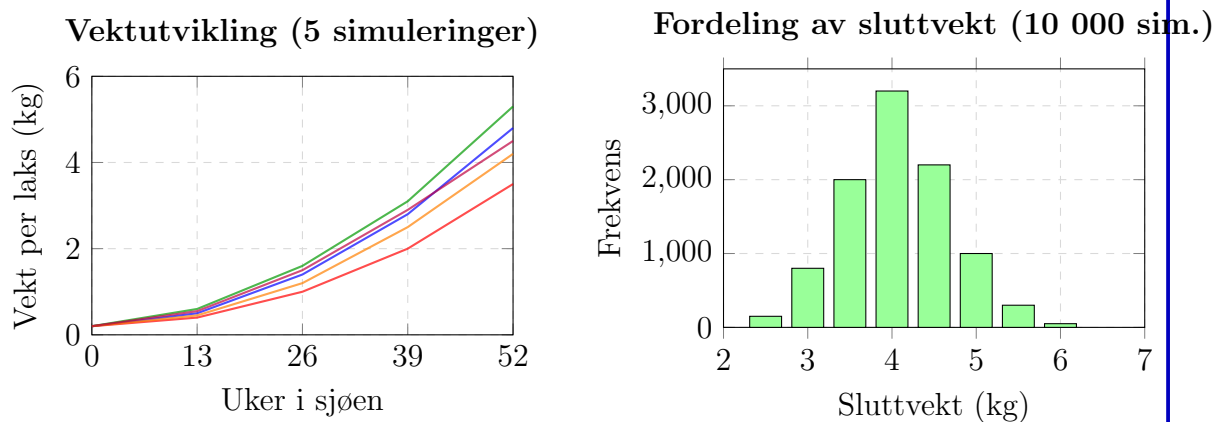
KAPITTELCASE 8: Simulering i havbruksnæringen

Fagansvarligs tips: I dette kapitlet har du lært å modellere usikkerhet. Norge er en av verdens største eksportører av laks, men oppdrett er en næring full av biologisk og økonomisk risiko. I denne casen skal du bygge en Monte Carlo-simulator som hjelper et oppdrettsselskap med å forutsi fremtidige inntekter basert på usikker vekst i laksemerden.

Scenariet: Laksemerden

Du jobber for oppdrettsselskapet *FjordLaks AS*. Dere har akkurat satt ut 100 000 smolt (smålaks) i en merd. Hver smolt veier i snitt 0.2 kg. Laksen skal stå i sjøen i nøyaktig 52 uker før den slaktes.

Problemet er at fisken ikke vokser like mye hver uke. Avhengig av temperatur og fôring, er den ukentlige veksten normalfordelt med et snitt på 6 % og et standardavvik på 1 %. Sjefen lurar på hva den forventede *sluttvekten* blir, og hvor mye penger dere kan forvente å tjene hvis lakseprisen ved slakting er 80 kr/kg (og vi antar at 95 000 laks overlever).



Figur 19: Venstre: 5 mulige vekstbaner for laksen. Høyre: Et histogram av sluttvekten etter at vi har latt datamaskinen simulere 10 000 ulike utfall.

Del 1: Algoritmisk tenkning

Bruk figurene og teksten over for å planlegge koden.

1. I en nøstet løkke har vi en *ytterløkke* og en *innerløkke*. Hva er det den innerste løkken i dette programmet skal telle ned? (Er det antall simuleringer, eller antall uker laksen er i sjøen?)
2. Hvilket tall vil du putte inn i `np.random.normal(loc=???, scale=???)` basert på scenariet over?

Del 2: Bygg Monte Carlo-simulatoren

Fyll inn de tomme strekene (-----) slik at vi kan regne ut gjennomsnittlig omsetning etter 1 000 simuleringer.

```
import numpy as np

antall_simuleringer = 1000
uker = 52
start_vekt = 0.2

# Her skal vi samle opp sluttvekten fra alle de 1000 simuleringene
alle_sluttvekter = []

# Den ytterste løkken (kjører én gang per simulerte fremtid)
for i in range(-----):
    vekt = start_vekt

    # Den innerste løkken (simulerer ukene som går for denne ene fisken)
    for uke in range(-----):
        # Trekker en tilfeldig vekst fra normalfordelingen
        ukentlig_vekst = np.random.-----(loc=0.06, scale=0.01)

        # Legger veksten til dagens vekt
        vekt = vekt * (1 + ukentlig_vekst)

    # Når de 52 ukene er ferdige, lagrer vi sluttvekten i listen vår
    alle_sluttvekter.-----(vekt)

# --- ANALYSE AV RESULTATENE ---
# Vi gjør listen om til en NumPy-array for å kunne regne på den
sluttvekter_array = np.array(alle_sluttvekter)

# Beregner forventet inntekt i NOK (95 000 overlevende laks * 80 kr/kg)
inntekter = sluttvekter_array * 95000 * 80

print(f"Forventet gjennomsnittsinntekt: {np.mean(inntekter):.0f} kr")
print(f"I verste fall (minimum): {np.min(inntekter):.0f} kr")
```

Del 3: Svarte svaner og svakheter i modellen

Koden du akkurat har bygget er mye brukt, men i biologi (og finans) har `np.random.normal()` en farlig blindsoner. En normalfordeling forutsetter at de fleste utfall havner nær snittet, og at ekstreme utfall (som en vekst på -20%) i praksis er umulig.

Men laksen er sårbar for sykdomsutbrudd, lakselus og giftige algeoppblomstringer. Dette kalles "fete haler" i statistikken, eller "Svarte svaner" i økonomien (ekstreme hendelser som sjelden inntreffer, men som har enorm påvirkning).

Refleksjon: Hvordan kunne du endret logikken inni den innerste løkken (ved hjelp av en `if`-setning og `np.random.uniform`) for å simulere at det er 2% sjanse *hver uke* for at laksen rammes av en algeoppblomstring som reduserer vekten / dreper biomasse med 30% umiddelbart? Skriv ned pseudokode for denne tanken!

Del II: Repetisjon, oppgaver og løsningsforslag

Denne delen av kompendiet er bygd opp som en samling *repetisjonsark*, ett for hvert hovedtema vi har gått gjennom. Materialet er hentet fra og videreutviklet basert på øvingsarkene i **BED-1304 Python-Lab**. Hvert kapittel følger samme oppskrift:

1. **Repetisjon og teori** av temaet.
2. **Flervalgsoppgaver** (ett riktig svar per oppgave).
3. **Kort svar** / ”Hva blir output?” som tester kodeforståelse.
4. **Praktiske programmeringsoppgaver**.
5. **Løsningsforslag** til alt sammen.

Et godt råd før du starter: Forsøk alltid å løse oppgavene *selv* før du ser på fasiten. For de praktiske oppgavene finnes det som regel mange veier til målet. At din løsning ser annerledes ut enn løsningsforslaget, betyr ikke at den er feil! Husk Pólyas fjerde steg: *se over svaret, og spør deg selv om du tror på det*.

Repetisjon: Python Basics

Repetisjon og teori

Hva er et programmeringsspråk?

Et programmeringsspråk er et formelt språk som brukes til å gi instruksjoner til en datamaskin. Språket gir oss muligheten til å beskrive algoritmer og løse problemer på en strukturert måte.

Eksempel: Høynivåspråk (Python)

Python er et høynivåspråk. Koden er enkel å lese og forstå, og minner mye om menneskelig språk. For eksempel kan vi skrive et lite program som regner ut summen av tallene fra 1 til 5:

Eksempel: Summere tall ved hjelp av en for-løkke

```
total = 0
for i in range(1, 6):
    total += i

print(f"Summen av tallene fra 1 til 5 er {total}")
```

Eksempel: Lavnivåspråk (Assembly)

Assembly er et lavnivåspråk, som ligger mye nærmere maskinkoden. Her gjør vi det samme som i Python-eksemplet (summen av tallene 1 til 5), men koden er mye mer detaljert og vanskeligere å lese:

```
MOV CX, 5      ; Teller (antall tall)
MOV AX, 0      ; Akkumulator = 0
MOV BX, 1      ; Startverdi = 1

LOOP_START:
ADD AX, BX     ; Legg til BX i AX
INC BX         ; Øk BX med 1
LOOP LOOP_START

; Resultatet (summen) ligger nå i AX
```

Hva er Python?

Python er et av verdens mest brukte programmeringsspråk. Det er kjent for å være lett å lese og lære, samtidig som det er kraftig nok til å brukes i avanserte prosjekter innen dataanalyse, maskinlæring, webutvikling, automatisering og mye mer.

Gode programmeringsvaner

- **Pen kode er bedre enn stygg kode.** Bruk innrykk, mellomrom og linjeskift for å gjøre koden ryddig.
- **Kommenter koden.** Skriv korte forklaringer slik at både du og andre forstår hensikten.
- **Gi gode navn til variabler.** Bruk `alder` i stedet for `x` dersom variabelen faktisk representerer en alder.
- **Unngå duplisering.** Ikke skriv samme kode flere ganger, lag heller funksjoner.
- **Test koden ofte.** Kjør korte tester underveis for å fange opp feil tidlig.

Variabeltyper

- **int** (heltall): `x = 5`
- **float** (desimaltall): `y = 3.14`
- **bool** (sann/usann): `z = True`
- **string** (tekst): `navn = "Anna"`

Utskrift med `print()`

Vi kan skrive ut data med funksjonen `print()`:

```
print("Hei verden")
```

Vi kan også bruke f-strenger for å formatere tekst:

```
navn = "Anna"
alder = 22
print(f"Hei, jeg heter {navn} og jeg er {alder} år gammel.")
```

Operatorer i Python

I Python finnes det mange ulike operatorer. Disse brukes til å utføre beregninger, sammenligninger og tilordninger. Her følger en oversikt over de vanligste:

- **Aritmetiske operatorer:**
 - `+` : addisjon (f.eks. `2 + 3 = 5`)
 - `-` : subtraksjon (f.eks. `5 - 2 = 3`)
 - `*` : multiplikasjon (f.eks. `4 * 3 = 12`)
 - `/` : divisjon (flyttall, f.eks. `7 / 2 = 3.5`)
 - `//` : heltallsdivisjon (avrunder ned, f.eks. `7 // 2 = 3`)
 - `%` : modulus (resten etter divisjon, f.eks. `7 % 2 = 1`)

– `**` : eksponent (opphøyd i, f.eks. `2 ** 3 = 8`)

- **Tilordningsoperatorer:**

– `=` : tilordning (lagrer en verdi i en variabel, f.eks. `x = 5`)

– `+=`, `-=`, `*=`, `/=` : kombinerer operasjon og tilordning. Eksempel:

```
x = 5
x += 3    # tilsvarer x = x + 3
print(x) # Output: 8
```

- **Sammenligningsoperatorer:**

– `==` : er lik (f.eks. `5 == 5` gir `True`)

– `!=` : ikke lik (f.eks. `5 != 3` gir `True`)

– `>` : større enn

– `<` : mindre enn

– `>=` : større enn eller lik

– `<=` : mindre enn eller lik

Eksempel som viser flere operatorer i bruk

```
a = 7
b = 2

print(a + b)    # 9
print(a / b)    # 3.5
print(a // b)   # 3
print(a % b)    # 1
print(a ** b)   # 49

print(a == b)   # False
print(a > b)    # True
```

Eksempel ved bruk av `//` og `%` i Python

- `//` er **heltallsdivisjon**. Det gir resultatet av divisjonen, men avrundet ned til nærmeste heltall.

```
print(7 // 3) # Output: 2    (fordi 7 / 3 = 2.333..., avrundet ned til 2)
```

- `%` er **modulus-operatoren**, som gir resten etter heltallsdivisjon.

```
print(7 % 3)  # Output: 1    (fordi 7 = 3*2 + 1)
```

Disse to brukes ofte sammen, f.eks. når man skal finne ut om et tall er partall eller oddetall:

```
tall = 10
if tall % 2 == 0:
    print("Partall")
else:
    print("Oddetall")
```

Del 1: Flervalgsoppgaver

1. Hva brukes Python hovedsakelig til?
 - (a) Matlaging
 - (b) Dataanalyse, programmering og automatisering
 - (c) Tegning
 - (d) Ingen av de over
2. Hva er output fra `print(2+3*4)`?
 - (a) 20 b) 14 c) 24 d) 18
3. Hva blir resultatet av `5 ** 2`?
 - (a) 7 b) 10 c) 25 d) 52
4. Hvilken datatype har `3.14`?
 - (a) int b) float c) bool d) string
5. Hvilken operator brukes for divisjon?
 - (a) * b) + c) - d) /
6. Hva er output fra `print(True and False)`?
 - (a) True b) False c) 1 d) 0
7. Hva er riktig måte å lage en streng på?
 - (a) `navn = '01a'` b) `navn = "01a"` c) Begge d) Ingen
8. Hva er resultatet av `10 // 3`?
 - (a) 3.33 b) 3 c) 4 d) 30
9. Hva gjør operatoren `%`?
 - (a) Potens b) Rest ved divisjon c) Multiplikasjon d) Negasjon
10. Hva er resultatet av `2 * (3+4)`?
 - (a) 14 b) 10 c) 12 d) 20
11. Hva vil `print(type(True))` gi?

- (a) int b) bool c) string d) float
12. Hva skjer om du skriver `navn = 10; navn = navn + 5`?
- (a) Feil b) navn blir 15 c) navn blir "105" d) navn blir 5
13. Hva blir output fra `print(2**3)`?
- (a) 6 b) 8 c) 9 d) 12
14. Hvordan skriver man en kommentar i Python?
- (a) `//` kommentar b) `/*` kommentar `*/` c) `#` kommentar d) `-` kommentar
15. Hva er riktig utskrift, gitt `navn = "Markus"`?
- (a) `print(f"Hei {navn}")`
(b) `print("Hei {navn}")`
(c) Begge virker likt
(d) Ingen virker

Del 2: Kortsvarsoppgaver

- `x = 10; y = 3; print(x % y)`
- `a = 2; b = 5; print(a * b + 3)`
- `print("2" + "3")`
- `print("2" * 3)`
- `print(10 // 3)`
- `print(3 > 5)`
- `navn = "Ola"; alder = 20; print(f"{navn} er {alder} år")`
- `x = 2; y = x; x = 5; print(y)`
- Skriv pseudokode for et program som spør om navn og alder, og skriver ut "Hei NAVN, du er ALDER år gammel."
- Skriv pseudokode for et program som regner ut arealet av et rektangel.

Del 3: Praktiske oppgaver

- Lag et program som ber om to tall og skriver ut summen.
- Lag et program som beregner arealet av en sirkel gitt radius.
- Lag et program som spør om et tall og skriver ut kvadratet.
- Lag et program som konverterer temperatur fra Celsius til Fahrenheit.
- Lag et program som spør etter et navn og skriver ut en personlig hilsen.

Løsningsforslag

Del 1: Flervalg

1: b 2: b 3: c 4: b 5: d 6: b 7: c 8: b 9: b 10: a 11: b 12: b
13: b 14: c 15: a

Del 2: Kortsvar

1: 1 2: 13 3: "23" 4: "222" 5: 3 6: False 7: Ola er 20 år 8: 2

Oppgave 9 (pseudokode):

```
START
    INPUT navn
    INPUT alder
    PRINT "Hei " + navn + ", du er " + alder + " år gammel."
STOP
```

Mulig Python-kode:

```
navn = input("Hva heter du? ")
alder = input("Hvor gammel er du? ")
print(f"Hei {navn}, du er {alder} år gammel.")
```

Oppgave 10 (pseudokode):

```
START
    INPUT lengde
    INPUT bredde
    area = lengde * bredde
    PRINT "Arealet er " + area
STOP
```

Mulig Python-kode:

```
lengde = float(input("Oppgi lengden: "))
bredde = float(input("Oppgi bredden: "))
areal = lengde * bredde
print(f"Arealet er {areal}")
```

Del 3 - praktiske oppgaver. Husk, det er mange måter å løse ulike problem. Har dere noe annet enn det som står her, betyr ikke det at dere nødvendigvis har gjort noe galt!

```
# Oppgave 1: Brukeren skriver inn to heltall, a og b
a = int(input("Skriv et tall: "))
b = int(input("Skriv et tall til: "))
print("Summen er", a + b)
```

```
# Oppgave 2: Areal av sirkel
# importerer numpy som np, slik at vi kan bruke numpy-biblioteket - for tallet pi
import numpy as np
```

```

r = 4
areal = np.pi * r**2
print("Areal er", areal)
# Alternativt, kan vi bruke en f-streng
print(f"Areal er {np.pi * r**2}.")

```

Her vil løsningen gi et svar med flere desimaler. Vurder å bruke funksjonen `round(areal, 2)` – hva gjør `round()`? Hva gjør tallet "2"?

```

# Oppgave 3: Kvadratet av et tall
# Ber brukeren om et tall, og sikrer at Python tolker det som en "float".
tall = float(input("Skriv inn et tall: "))
print(f"Kvadratet av {tall} er {tall**2}!")

# Oppgave 4: Konverterer temperatur fra Celsius til Fahrenheit
celsius = float(input("Skriv inn temperatur i Celsius: "))
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius} grader Celsius tilsvarer {fahrenheit} grader Fahrenheit.")

# Oppgave 5: Tar inn et navn og skriver ut en personlig hilsen
navn = input("Hva heter du? ")
print(f"Heisann {navn}, hyggelig! Vi kan bli venner :)")

```

Repetisjon: Funksjoner

Repetisjon og teori

Hva er en funksjon?

En funksjon er en gjenbrukbar blokk med kode som utfører en bestemt oppgave. Funksjoner gjør koden mer ryddig og modulær, og lar deg unngå duplisering.

Innebygde funksjoner i Python

Python har mange innebygde funksjoner, blant annet:

- `print()` – skriver ut tekst eller variabler.
- `len()` – returnerer lengden av en sekvens (f.eks. liste, string).
- `max()` – finner største verdi.
- `min()` – finner minste verdi.
- `sum()` – summerer verdier i en sekvens.
- `type()` – viser datatypen til en variabel.

Hvordan definere en funksjon

For å definere en funksjon bruker vi `def`-nøkkelordet:

```
def funksjonsnavn(param1, param2):  
    # innrykk er viktig!  
    resultat = param1 + param2  
    return resultat
```

Viktig: Alt som hører til funksjonen må ha innrykk (typisk 4 mellomrom). Funksjonen kan returnere en verdi med `return` eller gjøre en handling uten returverdi.

Parametere og argumenter

- **Parametere** er variablene som defineres i funksjonen (`param1, param2`).
- **Argumenter** er verdiene du sender inn når du kaller funksjonen (`funksjonsnavn(3, 5)`).

Eksempel: funksjon med returverdi

```
# Her er x vår parameter  
def kvadrat(x):  
    return x**2  
  
# Nå er tallet 5 vårt argument  
print(kvadrat(5)) # Output: 25
```

Eksempel: funksjon som skriver ut noe

```
def hils(navn):  
    print(f"Heisann {navn}!")  
  
hils("Ola")  # Output: Heisann Ola!
```

Funksjoner med flere parametere

```
def summer(a, b, c):  
    return a + b + c  
  
print(summer(2, 3, 4))  # Output: 9
```

Del 1: Flervalgsoppgaver

1. Hva brukes en funksjon til?

- (a) Gjenbrukbar kodeblokk
- (b) Skrive kommentarer
- (c) Endre variabeltyper
- (d) Tegning

2. Hvordan definerer man en funksjon?

- (a) `function navn():`
- (b) `def navn():`
- (c) `fun navn():`
- (d) `define navn():`

3. Hva er en parameter?

- (a) Verdien som sendes inn
- (b) Variabelen som defineres i funksjonen
- (c) Navnet på funksjonen
- (d) En kommentar

4. Hva er et argument?

- (a) Verdien som sendes inn når funksjonen kalles
- (b) Variabelen som defineres i funksjonen
- (c) Navnet på funksjonen
- (d) Returverdien

5. Hvilken funksjon gir lengden på en liste?

- (a) `size()`
- (b) `len()`
- (c) `length()`
- (d) `count()`

6. Hva gjør `return`?

- (a) Stopper funksjonen og returnerer en verdi
- (b) Printer en verdi
- (c) Definerer en parameter

- (d) Endrer en variabel
7. Hva vil `print(max(3,5,2))` gi?
- (a) 2 b) 3 c) 5 d) Feil
8. Hva vil `print(min([4,7,1]))` gi?
- (a) 4 b) 1 c) 7 d) [4, 7, 1]
9. Hva gjør `sum([1,2,3])`?
- (a) 6 b) 123 c) [1,2,3] d) Feil
10. Hva skjer hvis du glemmer innrykk i en funksjon?
- (a) Ingenting b) Python gir en `IndentationError` c) Funksjonen kjører som vanlig d) Kommentarer ignoreres
11. Hva skjer hvis du glemmer `return` i en funksjon som skal gi en verdi?
- (a) Returnerer automatisk 0
(b) Returnerer `None`
(c) Funksjonen krasjer
(d) Python antar `return 1`
12. Hva er forskjellen på parameter og argument?
- (a) Parametere er verdier, argumenter er variabler
(b) Parametere defineres i funksjonen, argumenter sendes inn ved kall
(c) Ingen forskjell, synonymer
(d) Parametere er alltid tall, argumenter alltid tekst
13. Hva gjør `len("Python")`?
- (a) Returnerer antall bokstaver
(b) Printer ordet baklengs
(c) Gir en feilmelding
(d) Multipliserer antall bokstaver med 2
14. Hva er en fordel med funksjoner?
- (a) Man slipper variabler
(b) Koden blir ryddig og gjenbrukbar
(c) Python kjører raskere
(d) Eneste måten å bruke `if` på
15. Hva skriver denne koden ut?

```
def test(x, y):  
    return x + y  
print(test(3, 7))
```

- (a) 10 b) 37 c) x+y d) Feilmelding

Del 2: Kortsvarsoppgaver

1. Hva blir output?

```
def kvadrat(x):  
    return x**2  
print(kvadrat(4))
```

2. Hva vil dette skrive ut?

```
def hils(navn):  
    print(f"Hei {navn}!")  
hils("Anna")
```

3. Hva blir resultatet?

```
def summer(a, b, c):  
    return a+b+c  
print(summer(1,2,3))
```

4. Skriv pseudokode for et program som lager en funksjon som regner kvadratet av et tall.
5. Skriv pseudokode for en funksjon som tar inn to tall og returnerer den største.

Del 3: Praktiske oppgaver

Når dere løser disse oppgavene, kan det være lurt å starte med å skrive en pseudokode med penn og papir, selv om du tror du vet hvordan du vil løse det. Det er en god øvelse!

1. Lag `hils()` som tar et navn og printer en personlig hilsen.
2. Lag `areal_rektangel(lengde, bredde)` som returnerer arealet.
3. Lag `faktorielle(n)` som returnerer faktorialen. (**Utfordring!**)
4. Bruk innebygde funksjoner `max()`, `min()` og `sum()` på en liste med tall.
5. Lag en funksjon som tar to tall og returnerer det største.

Løsningsforslag

Del 1: Flervalg

1: a 2: b 3: b 4: a 5: b 6: a 7: c 8: b 9: a 10: b 11: b 12: b
13: a 14: b 15: a

Del 2: Kortsvar

1: 16 2: Hei Anna! 3: 6

Oppgave 4 (pseudokode):

```
START
    FUNKSJON kvadrat(x)
        RETURN x * x
STOP
```

Oppgave 5 (pseudokode):

```
START
    FUNKSJON maks(a, b)
        HVIS a > b
            RETURN a
        ELSE
            RETURN b
STOP
```

Del 3 - praktiske oppgaver:

```
# Oppgave 1
def hils(navn):
    print(f"Heisann {navn}!")
hils("Ola")

# Oppgave 2
def areal_rektangel(lengde, bredde):
    return lengde * bredde
print(areal_rektangel(4, 5)) # Output: 20

# Oppgave 3
def faktorielle(n):
    resultat = 1
    for i in range(1, n + 1):
        resultat *= i
    return resultat
print(faktorielle(5)) # Output: 120

# Oppgave 4
tall_liste = [3, 7, 1, 5]
print(max(tall_liste)) # 7
print(min(tall_liste)) # 1
print(sum(tall_liste)) # 16
```



```
# Oppgave 5
def maks(a, b):
    if a > b:
        return a
    else:
        return b
print(maks(5, 9))  # Output: 9
```

Repetisjon: Lister, Tuple, Dictionary og NumPy

Repetisjon og teori

Lister

En liste er en samling av elementer som kan endres (mutable). Lister defineres med hakeparenteser `[]`.

```
frukt = ["eple", "banan", "appelsin"]
print(frukt[0])  # Output: eple

frukt.append("granateple")
print(frukt)  # ["eple", "banan", "appelsin", "granateple"]
```

Viktige metoder for lister:

- `append(x)` – legger til et element på slutten.

```
liste = [1, 2, 3]
liste.append(4)
print(liste)  # [1, 2, 3, 4]
```

- `extend(iterable)` – legger til alle elementer fra en *iterable*.

```
liste = [1, 2, 3]
liste.extend([4, 5])
print(liste)  # [1, 2, 3, 4, 5]
```

- `insert(i, x)` – setter inn element `x` på indeks `i`.

```
liste = [1, 3, 4]
liste.insert(1, 2)
print(liste)  # [1, 2, 3, 4]
```

- `remove(x)` – fjerner første element som er lik `x`.

```
liste = [1, 2, 2, 3]
liste.remove(2)
print(liste)  # [1, 2, 3]
```

- `pop(i)` – fjerner og returnerer element på gitt indeks (eller siste).

```
liste = [1, 2, 3]
liste.pop()      # Fjerner 3
liste.pop(0)     # Fjerner 1
print(liste)     # [2]
```

- `clear()` – tømmer hele listen.
- `index(x)` – returnerer posisjonen til første forekomst av `x`.

- `count(x)` – teller antall ganger `x` finnes i listen.
- `sort()` – sorterer listen i stigende rekkefølge. *Hva om lista inneholder ord, og ikke heltall? Prøv da vel!*
- `reverse()` – reverserer rekkefølgen.
- `copy()` – returnerer en kopi av listen.

Slicing av lister

Syntaksen er `liste[start:slutt:steg]`, der `start` er inkludert, `slutt` er ekskludert, og `steg` angir hvor mye vi hopper (standard 1).

```
tall = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(tall[2:6])      # [2, 3, 4, 5]
print(tall[:4])       # [0, 1, 2, 3]
print(tall[5:])       # [5, 6, 7, 8, 9]
print(tall[::2])      # [0, 2, 4, 6, 8]
print(tall[::-1])     # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Tips: Negativ indeks teller fra slutten (-1 er siste element), og `liste[:]` er en enkel måte å lage en kopi av en liste på.

Tuple

En **tuple** ligner på en liste, men den er **immutable**, dvs. at den ikke kan endres etter opprettelse. Man oppretter en tuple med parenteser `()`.

```
koordinat = (10, 20)
print(koordinat[0])  # Output: 10
```

Hvorfor bruke tuple i stedet for liste?

- Når dataene ikke skal endres (f.eks. koordinater, konstanter eller faste innstillinger).
- Tupler er **raskere** å opprette og bruke enn lister.
- Tupler kan brukes som **nøkler i dictionaries**, noe lister ikke kan.
- Det gjør tydelig for andre kodere at dataene er ”konstante”.

```
# Ulike måter å opprette tuples:
t1 = (1, 2, 3)           # Vanlig tuple
t2 = ("a", "b", "c")     # Med strenger
t3 = (1, "a", True)      # Kan blande ulike datatyper
t4 = (42,)               # En tuple med kun ett element - merk kommaet!
t5 = tuple([1, 2, 3, 4]) # Lage tuple fra en liste
```

Tilgang, slicing og unpacking:

```
t = (10, 20, 30, 40)
print(t[1])      # 20 (Python er null-indeksert)
print(t[-1])     # 40 (siste element)
print(t[1:3])    # (20, 30) - slicing gir også en tuple
```

```
person = ("Anna", 25, "Oslo")
navn, alder, by = person # unpacking
print(navn)             # Anna
```

NB: Unpacking kan du gjøre med lister også! Prøv selv :)

Bruk lister når data skal endres (eller du ønsker muligheten), og tupler når data skal være uforanderlige. Dette gir mer effektiv kode og tydeligere hensikt.

Dictionary

Et **dictionary** ("ordbok" på norsk) lagrer data i **key:value**-par. Nøkler må være **immutable** (f.eks. `str`, `int`, `tuple`), mens verdiene kan være av hvilken som helst datatype.

```
alder = {"Ola": 25, "Kari": 30, "Per": 22}
print(alder["Ola"]) # Output: 25

alder["Kari"] = 31    # Oppdatering av verdi
alder["Nina"] = 28    # Legge til nytt element
print(alder)
# {'Ola': 25, 'Kari': 31, 'Per': 22, 'Nina': 28}
```

NB: Python er *case-sensitive*. Skriver du `alder["ola"]` (med liten o), får du en feilmelding fordi nøkkelen er `Ola` med stor O.

Nyttige metoder:

- `keys()` – returnerer alle nøkler.
- `values()` – returnerer alle verdier.
- `items()` – returnerer par av (key, value).
- `get(key, value)` – henter verdien til en nøkkel, returnerer `value` hvis nøkkelen ikke finnes (valgfritt; ellers `None`).
- `pop(key)` – fjerner og returnerer verdien til en nøkkel.

```
student = {"navn": "Lise", "alder": 21, "studie": "Okonomi"}
print(student.keys())    # dict_keys(['navn', 'alder', 'studie'])
print(student.values())  # dict_values(['Lise', 21, 'Okonomi'])
print(student.items())   # dict_items([('navn', 'Lise'), ...])
print(student.get("klasse", "Ukjent")) # Output: Ukjent
student.pop("alder")
print(student)           # {'navn': 'Lise', 'studie': 'Okonomi'}
```

NumPy

NumPy er et kraftig Python-bibliotek for numeriske beregninger. Det er spesielt viktig for effektiv håndtering av store datasett, vektorer og matriser, og er mye raskere enn vanlige Python-lister fordi det er bygget oppå optimalisert C-kode.

NumPy importeres vanligvis slik (med alias `np`):

```
import numpy as np
```

OBS: Dette fungerer hvis, og bare hvis, man har lastet ned numpy først (ved hjelp av `pip`).

En- og to-dimensjonale arrays:

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)           # [1 2 3 4 5]
print(arr1[0])        # Forste element: 1
print(arr1.shape)     # Dimensjon: (5,)
```

```
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])
print(arr2[0, 1])     # Element i forste rad, andre kolonne: 2
print(arr2.shape)     # Dimensjon: (2, 3)
print(arr2.flatten()) # [1 2 3 4 5 6]
```

Nyttige operasjoner: `arr2.flatten()` gjør 2D-array om til 1D, og `arr2.shape` viser dimensjonene.

Vektoriserte operasjoner og matematiske funksjoner:

```
arr = np.array([1, 2, 3, 4])
print(arr * 2)  # [2 4 6 8] (alle elementene ganges)
```

```
print(np.pi)      # 3.14159...
print(np.exp(1))   # 2.718...
print(np.sqrt(16)) # 4.0
print(np.log(np.e)) # 1.0
```

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b)  # [5 7 9]
print(a * b)  # [ 4 10 18]
```

Dette gjør at vi slipper løkker, og koden blir både raskere og enklere.

Økonomisk eksempel: fremtidig verdi (FV)

Formelen er $FV = PV \cdot (1 + r)^t$. NumPy kan regne ut FV for mange rentesatser på en gang:

```
PV = 1000    # nåverdi
r = 0.05     # rente (5 %)
t = 10       # antall perioder
FV = PV * (1 + r)**t
print(FV)    # 1628.894626777442

renter = np.array([0.03, 0.04, 0.05])
FVs = PV * (1 + renter)**t
print(FVs)   # [1343.916 1480.244 1628.895]
```

Så her ser vi hva fremtidig verdi blir for tre ulike renteverdier. Smart, ikke sant?

Del 1: Flervalgsoppgaver

1. Hva er hovedforskjellen på en liste og en tuple?
 - (a) Lister er mutable, tupler er immutable
 - (b) Tupler er raskere å iterere over
 - (c) Ingen forskjell
 - (d) Tupler kan bare inneholde tall
2. Hvilken metode legger til ett element i en liste?
 - (a) `add()`
 - (b) `insert()`
 - (c) `append()`
 - (d) `extend()`
3. Hvordan definerer man et dictionary?
 - (a) `[]`
 - (b) `()`
 - (c) `{}`
 - (d) `dict[]`
4. Hva gir `np.array([10,20,30]).mean()`?
 - (a) 20
 - (b) 30
 - (c) Feil
 - (d) 60
5. Hva returnerer `len((1,2,3))`?
 - (a) 2
 - (b) 3
 - (c) (1,2,3)
 - (d) Feil

Del 2: Kortsvarsoppgaver

1. Hva blir output?

```
frukt = ["eple", "banan"]
frukt.append("appelsin")
print(frukt)
```

2. Hva blir resultatet?

```
koordinat = (4, 5)
print(koordinat[1])
```

3. Hvilken verdi får man her?

```
alder = {"Ola": 25, "Kari": 30}
print(alder["Kari"])
```

4. Hva vil dette gi?

```
import numpy as np
arr = np.array([2,4,6])
print(arr * 3)
```

5. Forklar med egne ord forskjellen på en liste og en tuple.

Del 3: Praktiske oppgaver

1. Lag en funksjon som tar en liste med tall og returnerer gjennomsnittet (uten NumPy).
2. Lag en tuple med koordinatene (x, y) , og en funksjon som returnerer avstanden til origo. **Hint:** $\text{avstand} = \sqrt{x^2 + y^2}$ (bruk `math.sqrt()`).
3. Lag et dictionary med studenter og poengsum. Lag en funksjon som finner studenten med høyest poengsum.
4. Bruk NumPy til å lage en array fra 1 til 10 og regn ut summen.
5. Lag en liste med tall, gjør den om til en NumPy-array, og gang alle tall med 2.
6. **Utfordring:** Lag et program som lagrer navn og favorittfarge i et dictionary. Programmet skal spørre om et navn, sjekke om det finnes, skrive ut fargen om det gjør, ellers be om fargen og legge den til. **Hint:** `if/else` og `in`.

Løsningsforslag

Del 1: Flervalg

1: a 2: c 3: c 4: a 5: b

Del 2: Kortsvar

1: ['eple', 'banan', 'appelsin'] 2: 5 3: 30 4: [6 12 18] 5: Lister kan endres, tupler kan ikke endres.

Del 3 - praktiske oppgaver:

```
# Oppgave 1
def gjennomsnitt(liste):
    return sum(liste) / len(liste)
print(gjennomsnitt([2, 4, 6])) # Output: 4.0
```

Gjennomsnittet finner vi ved å legge sammen alle tallene (`sum(liste)`) delt på antall elementer (`len(liste)`).

```
# Oppgave 2
import math # Importerer pakken math
def avstand(punkt):
    return math.sqrt(punkt[0]**2 + punkt[1]**2)
print(avstand((3, 4))) # Output: 5.0
```

```
# Oppgave 3
def topp_student(resultater):
    return max(resultater, key=resultater.get)
studenter = {"Ola": 80, "Kari": 95, "Per": 70}
print(topp_student(studenter)) # Output: Kari
```

- `resultater` er en **dictionary** med studentnavn som nøkler og poengsummer som verdier.
- `max(resultater, key=resultater.get)` finner nøkkelen med høyest verdi.

```
# Oppgave 4
import numpy as np
arr = np.arange(1, 11)
print(arr.sum()) # 55
```

```
# Oppgave 5
liste = [1, 2, 3, 4]
arr = np.array(liste)
print(arr * 2) # Output: [2 4 6 8]
```

```
# Oppgave 6 - Utfordring
favorittfarger = {"Ola": "Gul", "Kari": "Oransj"}
navn = input("Skriv navn: ")
if navn in favorittfarger:
    print(f"Favorittfargen til {navn} er {favorittfarger[navn]}")
else:
    farge = input("Oppgi favorittfargen: ")
    favorittfarger[navn] = farge
    print("Fargen er lagt til!")
```


Repetisjon: Pandas og databehandling

Repetisjon og teori

Pandas

Pandas er et av de mest brukte bibliotekene i Python for dataanalyse og behandling av tabulære data. Det bygger på NumPy og gjør det enkelt å jobbe med datasett i tabellform (ligner på Excel, men mye kraftigere).

Installasjon og import

For å installere Pandas kan du kjøre følgende i en Jupyter Notebook:

Nedlastning av Pandas biblioteket

```
# Enten kan man bruke "!", som laster ned pakken via terminal
!pip install pandas

# Eller kan man bruke "%" som betyr "magic command"
# som vil laste inn pakken i samme miljø som notebooken din
%pip install pandas
```

For å importere pakken (standard praksis er alias `pd`), og det gjøres slik:

Import av pakken

```
import pandas as pd
```

Lese inn data fra fil

For å lese inn en CSV-fil bruker vi funksjonen `read_csv`. På Windows kan det være lurt å bruke `r` foran filbanen, slik at spesialtegn (som `\n`) ikke feiltolkes.

Lese fil inn i Python

```
data = pd.read_csv(r'DERES_PATH/schooling-gdp.csv')
data.head()
```

OBS: Får du feilmelding om at du mangler `openpyxl`, kjør `%pip install openpyxl`. Får du en annen feilmelding, kommer den nok fra **filbanen** dere oppga.

Eksempel på data

Datasettet kan inneholde kolonner som:

Land	Landkode	BNP_per_capita	Utdanning	Befolkning
Norway	NOR	73262.680	11.65	4886000
Kuwait	KWT	67029.523	6.39	2992000
Luxembourg	LUX	57882.809	11.33	508000
Switzerland	CHE	55688.020	12.92	7809000
United States	USA	49500.629	13.24	309011008

Her har vi for eksempel land, landkode, BNP per innbygger, utdanningsnivå og befolkning. Dataene kan analyseres videre med Pandas. Dette datasettet er det vi skal se på i kompendiet (Kilde: Our World In Data).

Denne filen inneholder data om BNP (bruttonasjonalprodukt) og utdanningsnivå for ulike land i 2010.

Hva er en DataFrame?

En **DataFrame** er hovedstrukturen i Pandas. Den kan sees som en tabell (lignende Excel) hvor radene har indekser og kolonnene har navn. Det gir fleksibel tilgang til data, enkel filtrering, gruppering og statistiske beregninger.

Operasjoner på DataFrames

Vi antar at vi har kalt datarammen `data` (et annet vanlig navn er `df`).

- **Vise kolonner:** `data.columns`
- **Velge kolonne:** `data['kolonnenavn']`
- **Filtrere rader** (f.eks. utdanning > 10 år): `data[data['Utdanning'] > 10]`
- **Statistisk oversikt:** `data.describe()`
- **Sortere data:** `data.sort_values('kolonnenavn')`
- **Sjekk manglende verdier:** `data.isnull().sum()`
- **Lage ny kolonne:** `data['BNP_per_innb'] = data['BNP_per_capita'] / data['Befolkning']`
- **Se datatyper:** `data.dtypes`
- **Informasjon om DataFrame:** `data.info()`

Eksempler på bruk

```
import pandas as pd

data = pd.read_csv("schooling-gdp.csv") # Last inn CSV-fil
print(data.head())                     # Vis de første 5 radene
print(data["BNP_per_capita"])           # Velg en enkelt kolonne
print(data[data["Utdanning"] > 10])     # Filtrere rader

# Lage en ny kolonne basert på beregning
data["BNP_per_utdanning"] = data["BNP_per_capita"] / data["Utdanning"]
print(data.head())
```

Forklaring av eksempelet over:

- `pd.read_csv(...)`: Leser en datafil (CSV-format).
- `.head()`: Viser de første 5 radene.

- `data["BNP_per_capita"]`: Henter en bestemt kolonne.
- `data[data["Utdanning"] > 10]`: Filtrerer rader basert på en betingelse.

```
print(data.describe()) # Gir en statistisk oversikt
# Sortere land etter høyest BNP per capita
print(data.sort_values("BNP_per_capita", ascending=False).head(5))
```

Pandas gir oss dermed verktøy til å lese og lagre datafiler, utforske og filtrere data, gjøre beregninger og lage nye variabler, få statistiske oppsummeringer, og sortere og analysere datasett raskt og effektivt.

Del 1: Flervalgsoppgaver

1. Hvilken metode viser de første radene?
 - (a) `data.top()`
 - (b) `data.head()`
 - (c) `data.start()`
 - (d) `data.show()`
2. Hvilken metode gir statistisk oppsummering?
 - (a) `data.summary()`
 - (b) `data.stats()`
 - (c) `data.describe()`
 - (d) `data.info()`
3. Hvordan henter man kolonnen A?
 - (a) `data[A]`
 - (b) `data["A"]`
 - (c) `data.get("A")`
 - (d) b) og c)
4. Hva gjør `data.isnull().sum()`?
 - (a) Sletter tomme rader
 - (b) Teller manglende verdier per kolonne
 - (c) Returnerer True/False per celle
 - (d) Gir feil
5. Hvilken metode sorterer en DataFrame?
 - (a) `data.order()`
 - (b) `data.sort('kol')`
 - (c) `data.sort_values('kol')`
 - (d) `data.arrange('kol')`
6. Hva gir `data.shape`?
 - (a) Antall rader
 - (b) Antall kolonner
 - (c) Tuple med (antall rader, antall kolonner)
 - (d) Datatypene
7. Hvordan lager man kolonnen $C = A + B$?
 - (a) `data["C"] = A+B`
 - (b) `data.C = data.A + data.B`

- (c) `data["C"] = data["A"] + data["B"]`
 - (d) `data.newcol("C", A+B)`
8. Hva gjør `data.dtypes`?
- (a) Viser antall rader
 - (b) Viser antall kolonner
 - (c) Viser datatype per kolonne
 - (d) Konverterer alt til tekst
9. Hvordan henter man de siste 5 radene?
- (a) `data.last()`
 - (b) `data.tail()`
 - (c) `data.bottom()`
 - (d) `data.end()`
10. Hva gjør `data["A"].mean()`?
- (a) Regner gjennomsnittet av kolonnen A
 - (b) Viser første rad
 - (c) Teller antall elementer
 - (d) Sletter NaN-verdier

Del 2: Hva blir output?

1.

```
import pandas as pd
data = pd.DataFrame({"A": [1,2,3], "B": [4,5,6]})
print(data.head())
```
2.

```
print(data["A"])
```
3.

```
print(data.shape)
```
4.

```
print(data["A"].mean())
```
5.

```
print(data.describe())
```
6.

```
print(data["B"].max())
```
7.

```
print(data.isnull().sum())
```
8.

```
data["C"] = data["A"] * data["B"]
print(data)
```

Del 3: Praktiske oppgaver

OBS: Før dere kan laste inn filer, må dere ha lastet ned biblioteket `openpyxl` (!pip install openpyxl) og kjørt `import pandas as pd`.

1. Les inn `schooling-gdp.csv` i en `DataFrame` og vis de første 5 radene.
2. Sjekk hvor mange rader og kolonner `DataFrame`-en har.
3. Legg til en ny kolonne `GDP_thousands` (BNP per capita delt på 1000).
4. Finn gjennomsnittet av `Utdanning`-kolonnen.
5. Sorter etter `BNP_per_capita` og vis de 10 landene med høyest BNP.

Del 4: Utfordring. Kjør kodeblokken under:

```
import pandas as pd
url = ("https://raw.githubusercontent.com/uit-bed-1304-h25/uit-bed-1304-h26.github.io")
df = pd.read_csv(url)
df # Se på dataene
```

1. Forklar med ord hva du tror dette datasettet handler om.
2. Finn ut hvilke datatyper de ulike kolonnene er.
3. Sjekk om noen rader mangler verdier. *Hint:* `df.isnull().sum()`.
4. Sjekk hvilke verdier kolonnen `School_Type` består av. *Hint:* `df['School_Type'].value_counts()`
5. Finn hvor stor prosent som kommer fra privat skole, og hvor mange prosent fra offentlig skole.

Løsningsforslag

Del 1: Flervalg

1: b	2: c	3: d	4: b	5: c	6: c	7: c	8: c	9: b	10: a
------	------	------	------	------	------	------	------	------	-------

Del 2 - Hva blir output?

1:

```
   A  B
0  1  4
1  2  5
2  3  6
```

2:

```
   1
1  2
2  3
```

Name: A, dtype: int64

3: (3, 2) 4: 2.0 6: 6 7: A 0 / B 0 (dtype: int64)
 5: describe() gir count, mean, std, min, 25%, 50%, 75%, max for A og B (mean 2.0 for A og 5.0 for B, std 1.0 for begge). 8:

	A	B	C
0	1	4	4
1	2	5	10
2	3	6	18

Del 3 - praktiske oppgaver:

```
# Oppgave 1
import pandas as pd
data = pd.read_csv("schooling-gdp.csv") # bytt ut med din lokale filbane
print(data.head())
```

OBS: Her må dere laste ned fila fra hjemmesiden til kurset, lokalt på PC/Mac-en deres, og erstatte "schooling-gdp.csv" med lokasjonen dere har lagret fila på.

```
# Oppgave 2
print(data.shape) # (rader, kolonner) - skal bli en tuple (106, 5)
```

```
# Oppgave 3
data["GDP_thousands"] = data["BNP_per_capita"] / 1000
print(data.head())
```

```
# Oppgave 4
print(data["Utdanning"].mean())
```

```
# Oppgave 5
print(data.sort_values("BNP_per_capita", ascending=False).head(10))
```

Del 4 - Utfordring:

```
df.info() # Oppg. 2: se kolonnen "Dtype"
df["School_Type"].value_counts() # Oppg. 4: Public vs Private
df["School_Type"].value_counts(normalize=True) * 100 # Oppg. 5: prosent
```

Når vi kjører dette får vi et datasett om hvor mye studenter studerer, oppmøte, søvn osv., samt kolonnen Exam_Score (0–100). Med value_counts() ser vi at 4598 rader er Public og 2009 er Private. Still deg selv: Hva gjør normalize=True? Hva skjer med normalize=False? Hvorfor ganger vi med 100?

Repetisjon: Logikk og løkker

Repetisjon og teori

I programmering er **logikk** og **løkker** grunnleggende byggesteiner. De lar oss ta beslutninger basert på betingelser, og gjenta handlinger mange ganger uten å måtte skrive den samme koden flere ganger.

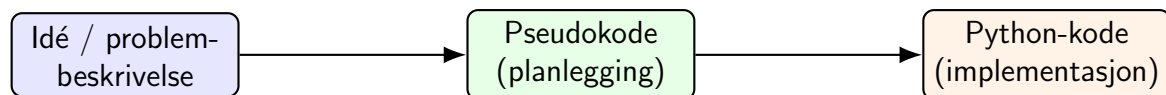
Pseudokode

Vi har tidligere i kurset diskutert hvordan vi går frem nå vi skal løse programmeringsoppgaver, og at *pseudokode* kan være en god hjelper.

Definisjon

Pseudokode: er en forenklet og uformell måte å beskrive et dataprogram eller en algoritme på. Den skrives ofte i et språk som ligner på ekte programkode, men uten å følge de strenge syntaksreglene som et programmeringsspråk krever. Pseudokode brukes gjerne i planleggings- og designfasen sammen med flytskjemaer og andre metoder, og er ment som et hjelpemiddel for mennesker – ikke noe som kan kjøres direkte på en datamaskin.

Kort sagt fungerer pseudokode som et **bindeledd mellom idé og ferdig Python-kode**, og er derfor et nyttig verktøy både i prosessen av å lære seg programmering, og drive med utvikling.



Figur 20: Pseudokode som bindeledd mellom idé og ferdig Python-kode.

Pseudokode - eksempel fra økonomi

Oppgavebeskrivelse: Vi ønsker å definere en funksjon som beregner hvordan en startkapital vokser med årlig rente over et gitt antall år. Formelen for sluttverdi er:

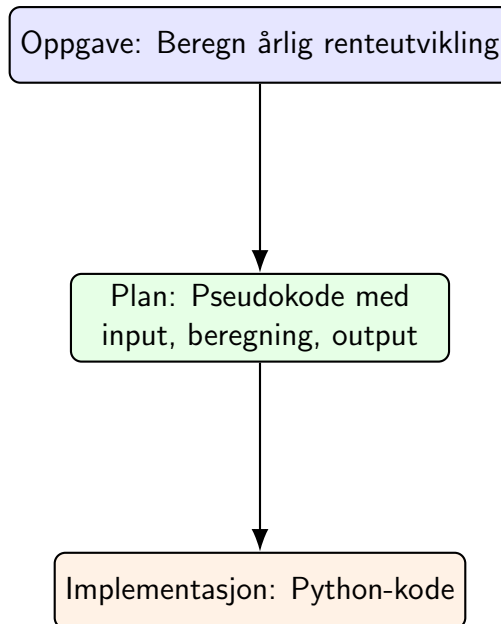
$$FV = K \cdot (1 + r)^n$$

der

- K = startkapital
- r = rente
- n = antall år
- FV = sluttkapital

Funksjonen skal ta inn disse parametrene og skrive ut resultatet i en f-string.

Skisse (prosess): Vi skisserer opp, *hva skal vi gjøre?*



Pseudokode: Vi fortsetter med å skrive en pseudokode, hvor vi stiller oss spørsmålet *hva er det vi skal kode?*

Pseudokode

- Definer en funksjon `future_value` med input: `capital`, `rate`, `years`.
- Beregn sluttverdi: `FV = capital * (1 + rate)**years`.
- Print ut en f-string: "Etter {years} år er verdien {FV}".

Nå har vi skissert opp hva vi skal gjøre, og hvordan vi har tenkt å kode dette. Da gjenstår det å skrive Python koden - og en mulig løsning er i kodeboksen under.

Python-kode

```
# Definerer funksjonen
def future_value(capital, rate, years):
    # Regner ut FV slik oppgaven beskriver
    FV = capital * (1 + rate)**years

    # Printer outputtet
    print(f"Etter {years} år er verdien {FV:.2f} kroner")

# Eksempelbruk av funksjonen med K = 10000 kr, r = 4% og years = 5 år
future_value(10000, 0.04, 5)
```

```
> Etter 5 år er veriden 12166.53 kroner
```

If/else-setning

Med `if/else` kan vi sjekke en betingelse og utføre kode kun hvis betingelsen er sann. Dersom betingelsen ikke er sann, kan vi bruke `else` til å kjøre en alternativ handling.

```
x = 10
if x > 5:
    print("x er større enn 5")
else:
    print("x er 5 eller mindre")
```

For-løkker

En `for`-løkke lar oss gjenta en blokk kode et bestemt antall ganger eller iterere over elementene i en samling (f.eks. en liste).

```
for i in range(1, 5):
    print(f"Tallet er {i}.")
```

While-løkker

En `while`-løkke kjører så lenge en betingelse er sann. Dette er nyttig når vi ikke vet hvor mange ganger koden skal gjentas.

```
x = 0
while x < 5:
    print(f"x er nå {x}")
    x += 1  # Samme som å skrive x = x + 1
```

Oppsummering: `if/else` brukes til å ta valg, `for`-løkker til å gjenta en handling et fast antall ganger eller gjennom en liste, og `while`-løkker når vi ønsker gjentakelse inntil en betingelse ikke lenger er oppfylt.

Måter å sjekke betingelser på

Sammenligningsoperatore: == (lik), != (ikke lik), >, <, >=, <=.

```
alder = 18
if alder >= 18:
    print("Stemmerett")
else:
    print("Ikke stemmerett")
```

Logiske operatore: and (sann hvis begge er sanne), or (sann hvis minst én er sann), not (inverterer sannhetsverdien).

```
alder = 20
har_lappen = True
if alder >= 18 and har_lappen:
    print("Kan kjøre bil")
else:
    print("Kan ikke kjøre bil")
```

Når if/else ikke er nok – elif!

Når vi har en logisk betingelse med **mer enn to utfall**, bruker vi **elif** (else if). **elif** lar oss lage flere grener før vi eventuelt bruker en **else**-blokk til å håndtere alt som ikke matcher tidligere betingelser.

```
alder = int(input("Skriv inn din alder: "))
if alder <= 12:
    print("Barn")
elif alder <= 17:
    print("Ungdom")
elif alder <= 64:
    print("Voksen")
else:
    print("Honnoer")
```

int() konverterer tekst til heltall slik at vi kan sammenligne alder. Python sjekker betingelsene ovenfra og ned, og stopper ved første sanne.

If/else med lister

Vi kan bruke **in** til å sjekke om en verdi finnes i en liste:

```
skattepliktig = ['lønn', 'bonus', 'kapitalinntekt']
skattefri = ['gave', 'stipend', 'frikort']
inntekt = 'bonus'
if inntekt in skattepliktig:
    print(f"{inntekt} er skattepliktig")
elif inntekt in skattefri:
    print(f"{inntekt} er skattefri")
else:
    print(f"Kategorien til {inntekt} er ukjent")
```

For-løkke – grundigere

```
# Skriver ut tallene fra 1 til 5
for i in range(1, 6):
    print(i)
```

range(1, 6) genererer tallene 1, 2, 3, 4, 5. Legg merke til at 6 ikke skrives ut, fordi range(a, b) teller **fra og med a** og **opp til** (men ikke inkludert) b. Vi kan også iterere over en liste:

```
frukt = ["eple", "banan", "appelsin"]
for f in frukt:
    print(f)
```

While-løkke – grundigere

```
# Skriver ut tallene fra 1 til 5
i = 1
while i <= 5:
    print(i)
    i += 1 # øk i med 1 hver gang løkka kjøerer
```

Hvis vi ikke øker i, vil while-løkka kjøre for alltid. Når i blir 6, blir betingelsen i <= 5 falsk, og løkken stopper. Vi kan også iterere over en liste med en indeks:

```
frukt = ["eple", "banan", "appelsin"]
i = 0
while i < len(frukt):
    print(frukt[i])
    i += 1
```

Komponenter i while-løkker

while starter løkken og definerer betingelsen; **betingelsen** evalueres til True/False; **løkkeblokken** er koden som kjøres; **break** avslutter løkken; **continue** hopper over resten av iterasjonen; og en **index** (definert **før** løkka, oppdatert **inni**) sikrer at løkka ikke kjører for evig.

Del 1: Flervalgsoppgaver

1. Hva printer x = 7; if x > 5: print("Større enn 5") else: print("5 eller mindre")?
(a) Større enn 5 b) 5 eller mindre c) Ingenting d) Feilmelding
2. Hva er sant om en for-løkke?
(a) Kan kun iterere over tall
(b) Kan iterere over lister, strenger og range

- (c) Kjører alltid kun én gang
 - (d) Fungerer bare med if/else
3. Hva gjør `for f in frukt: print(f)` (der `frukt` er en liste)?
- (a) Printer navnet på listen
 - (b) Printer hvert element
 - (c) Teller antall elementer
 - (d) Ingenting
4. Hvilken er riktig while-løkke som teller fra 1 til 3?
- (a) `i = 1; while i <= 3: print(i); i += 1`
 - (b) `while i in range(1,4): print(i)`
 - (c) `for i in range(1,4): print(i)`
 - (d) `while i < 3: print(i)`
5. Hvilken operator kombinerer to betingelser?
- (a) + b) and c) % d) ==
6. Hva printer dette (`alder = 16`)?
- ```
if alder >= 18: print("Voksen")
elif alder >= 13: print("Ungdom")
else: print("Barn")
```
- (a) Voksen    b) Ungdom    c) Barn    d) Feilmelding
7. Hva gjør `if x in liste:`?
- (a) Sjekker om x finnes i listen
  - (b) Legger til x i listen
  - (c) Fjerner x fra listen
  - (d) Gir alltid True

## Del 2: Hva blir output?

1. `x = 3`  

```
if x > 0 and x < 5:
 print("Innenfor intervallet")
else:
 print("Utenfor intervallet")
```
2. `for f in ["eple", "banan", "appelsin"]:`  

```
 print(f.upper())
```

```

1. i = 0
 while i < 3:
 print(i)
 i += 1

2. alder = 20
 har_lappen = False

 if alder >= 18 and har_lappen:
 print("Kan kjøre bil")
 else:
 print("Kan ikke kjøre bil")

3. inntekt = 120000

 if inntekt > 100000:
 print("Må betale skatt")
 else:
 print("Ingen skatt")

```

### Del 3: Praktiske oppgaver

1. Lag `klassifiser_alder(alder)` som returnerer 'Barn' (0–12), 'Ungdom' (13–17), 'Voksen' (18–64) eller 'Honnør' (65+).
2. Bruk en for-løkke til å printe kun tallene  $> 5$  i `[2,5,8,11,14]`.
3. Lag en while-løkke som teller ned fra 10 til 1.
4. Bruk en for-løkke med if/else på `[50000,120000,95000,200000]` og skriv ut om hver inntekt må betale skatt (frikortgrense 100000).
5. Lag `summer_liste(liste)` som bruker en while-løkke til å summere elementene.

**Del 4 (utfordring):** Anta startkapital 50 000 kr og investering over 12 måneder. Hver måned er den prosentvise avkastningen tilfeldig mellom  $-5\%$  og  $+10\%$ , og du har faste utgifter på 3 000 kr/mnd. Bruk `if/else` til å sjekke om kapitalen blir negativ (advarsel om konkurs), og en `while`-løkke til å summere kapitalen måned for måned inntil målkapital 70 000 kr er nådd eller pengene tar slutt. **Tips:** bruk `random.uniform(-0.05, 0.10)` (husk `import random`).

### Del 5 (vanskelig!)

Lag et program som hjelper en student med å holde oversikt over økonomien. Studenten har en månedlig inntekt, faste utgifter (dictionary), og tilfeldige månedlige ekstraavgifter (liste). Bruk en `for`- eller `while`-løkke til å simulere flere måneder, `if/else` til å sjekke om kontoen går i minus, og lag en **funksjon** som returnerer hvor mye som er spart (eller tapt) etter alle månedene. (Ekstra: bruk NumPy til gjennomsnittlig sparing, og pandas til å lage en tabell.)

## Løsningsforslag

### Del 1: Flervalg

1: a    2: b    3: b    4: a    5: b    6: b    7: a

### Del 2: Hva blir output?

1: Innenfor intervallet    2: EPLE BANAN APPELSIN (på hver sin linje)    3: 0 1 2  
(på hver sin linje)    4: Kan ikke kjøre bil    5: Må betale skatt

### Del 3 - praktiske oppgaver:

#### Python-kode

```
Oppgave 1
def klassifiser_alder(alder):
 if alder <= 12:
 return "Barn"
 elif alder <= 17:
 return "Ungdom"
 elif alder <= 64:
 return "Voksen"
 else:
 return "Honnoer"
print(klassifiser_alder(20)) # Voksen

Oppgave 2
tall = [2, 5, 8, 11, 14]
for t in tall:
 if t > 5:
 print(t)
 # Hvis t <= 5 gjoer vi ingenting

Oppgave 3
i = 10
while i > 0:
 print(i) # printer i
 i -= 1 # teller ned 1 steg fra forrige i-verdi
```

#### Python-kode

```
Oppgave 4
inntekter = [50000, 120000, 95000, 200000]
for inntekt in inntekter:
 if inntekt > 100000:
 print(f"{inntekt} kr: Må betale skatt")
 else:
 print(f"{inntekt} kr: Ingen skatt")

Oppgave 5
def summer_liste(liste):
 i = 0
 total = 0
 while i < len(liste):
 total += liste[i]
 i += 1
 return total
print(summer_liste([1, 2, 3, 4])) # 10
```

#### Del 4 - Utfordring (ett mulig løsningsforslag):

#### Python-kode

```
import random
capital = 50000
monthly_expense = 3000
goal = 70000
month = 0

print("Starter simuleringen...")
while capital > 0 and capital < goal and month <= 12:
 month += 1
 monthly_return = random.uniform(-0.05, 0.10) # tilfeldig avkastning
 capital = capital * (1 + monthly_return) - monthly_expense
 if capital <= 0:
 print(f"Konkurs etter {month} måneder!")
 break
 print(f"Måned {month}: Kapital = {capital:.2f} kr")

if capital >= goal:
 print(f"Målkapital på {goal} kr nådd etter {month} måneder!")
```

Fjerner du `monthly_expense` i beregningen, vil du oftere nå kapital-målet, siden du har kvittet deg med 3 000 kr i utgifter.

## Del 5 - enkel versjon (uten pandas/numpy):

### Python-kode

```
def simulate_budget(income, fixed_expenses, extra_expenses):
 savings = 0
 month = 0
 total_fixed = sum(fixed_expenses.values())
 for i in extra_expenses:
 month += 1
 total_expenses = total_fixed + i
 balance = income - total_expenses
 savings += balance
 if balance >= 0:
 print(f"Måned {month}: Sparte {balance} kr")
 else:
 print(f"Måned {month}: Gikk {abs(balance)} kr i minus")
 print(f"\nTotalt spart etter {month} måneder: {savings} kr")
 return savings

income = 15000
fixed_expenses = {"husleie": 6000, "mat": 3000, "stroem": 800}
extra_expenses = [500, 1200, 0, 800, 400, 3000]
simulate_budget(income, fixed_expenses, extra_expenses)
```



## Del 5 - avansert versjon (med NumPy og pandas):

### Python-kode

```
import numpy as np
import pandas as pd

def simulate_budget_pandas(income, fixed_expenses, extra_expenses):
 total_fixed = sum(fixed_expenses.values())
 months = len(extra_expenses)
 data = {"Måned": [], "Inntekt": [], "Utgifter": [], "Sparing": []}
 total_savings = 0
 for i in range(months):
 expenses = total_fixed + extra_expenses[i]
 saving = income - expenses
 total_savings += saving
 data["Måned"].append(i + 1)
 data["Inntekt"].append(income)
 data["Utgifter"].append(expenses)
 data["Sparing"].append(saving)
 df = pd.DataFrame(data)
 print(df)
 print("\nGjennomsnittlig sparing per måned:", np.mean(df["Sparing"]))
 print("Total sparing:", total_savings)
 return df

income = 15000
fixed_expenses = {"husleie": 6000, "mat": 3000, "stroem": 800}
extra_expenses = [500, 1200, 0, 800, 400, 0, 700, 6000]
simulate_budget_pandas(income, fixed_expenses, extra_expenses)
```

# Repetisjon: Matplotlib og visualisering

## Repetisjon og teori

I Python kan vi bruke `matplotlib` til å visualisere data i form av grafer og diagrammer. Dette gjør det enklere å forstå sammenhenger og se mønstre i tallmateriale. Som alltid må pakken først lastes ned:

```
!pip install matplotlib
```

**NB:** Noen må kanskje skrive `%` i stedet for `!`. Dette handler om hvor vi ”tvinger” pakken til å lastes ned.

## Eksempel på linjeplott

Med `plt.plot()` kan vi tegne grafer. Vi sender inn to lister: én med  $x$ -verdier og én med  $y$ -verdier.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

plt.plot(x, y, marker="o")
plt.xlabel("x-akse")
plt.ylabel("y-akse")
plt.title("Eksempel på plotting")
plt.show()
```

## Figurkomponenter

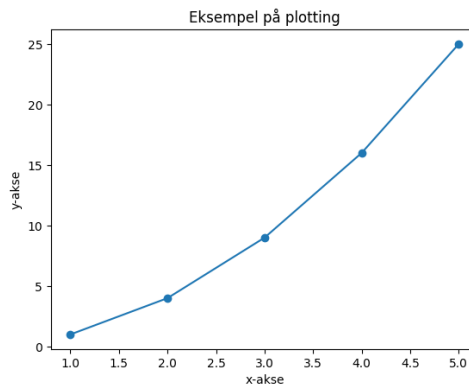
Hver figur i Matplotlib består av flere elementer: *figure* (hele tegneflaten), *axes* (plot-tområdet), *axis* (x- og y-akser), samt titler og etiketter.

### Python-kode

```
import matplotlib.pyplot as plt

x = [0, 2, 4, 6, 8]
y = [0, 4, 16, 36, 64]

fig, ax = plt.subplots() # Lager figur og akser
ax.plot(x, y, marker='o', label="Datapunkter")
ax.set_title("Komponenter i en Matplotlib-figur")
ax.set_xlabel("X-akse")
ax.set_ylabel("Y-akse")
plt.legend() # viser forklaringsboksen
plt.show()
```



Figur 21: Plot til kodeblokken over.

## Ulike diagramtyper

Vi bruker datasettet `StudentPerformanceFactors.csv` for å vise ulike måter å visualisere data på. Datasettet inneholder hvor mye studenter har studert, hvor mye foreldre er involvert, resultat på eksamen og så videre.

**1. Histogram** (fordeling av timer studert): Et histogram gir en oversikt over fordelingen av hvor mange timer studentene studerer, koden under generer plottet vi kan se i Figur 22.

### Python-kode for histogram

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("StudentPerformanceFactors.csv")
plt.hist(df["Hours_Studied"], bins=10, color="skyblue", edgecolor="black")
plt.xlabel("Hours Studied")
plt.ylabel("Number of Students")
plt.title("Histogram over timer studert")
plt.show()
```

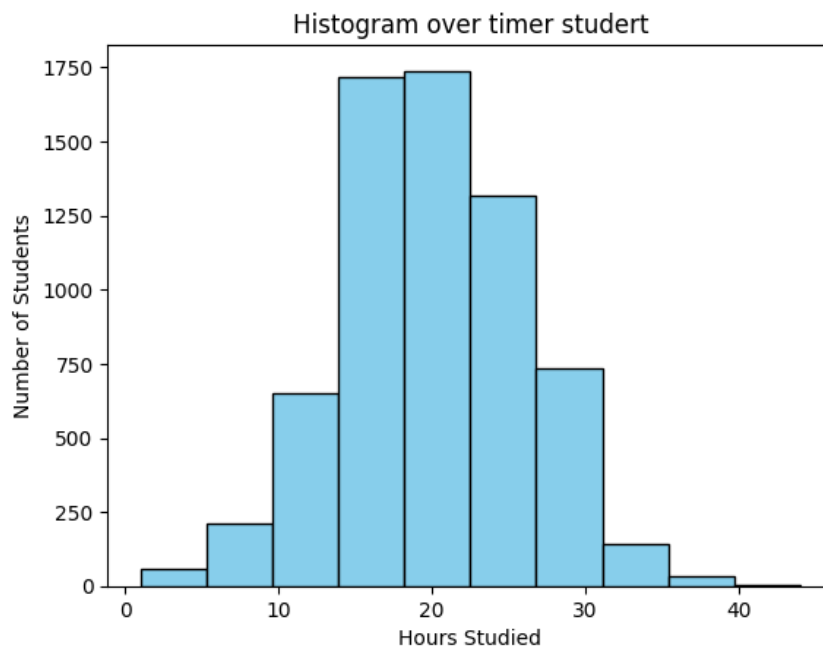
**Tips:** Prøv å gjenskap koden selv, og se hva hver enkelt linje gjør.

**2. Scatterplot** (timer studert vs. eksamensresultat): Et scatterplot kan vise sammenhengen mellom antall timer studert og `Exam_Score`, plottet ser vi i 23.

### Python-kode for scatterplott

```
plt.scatter(df["Hours_Studied"], df["Exam_Score"], color="green")
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.title("Scatterplot: Hours Studied vs Exam Score")
plt.show()
```

**Tenk selv:** Still deg selv spørsmålet, *hva ser vi egentlig her?* Hva sier dette plottet oss?



Figur 22: Histogram som viser hvor mye studentene studerer.

### 3. Kakediagram (fordeling av foreldreinvolvering).

La oss si vi er interessert i å se på hvor stor grad foreldre involverer seg i barnas skolegang (i datasettet `df`). Først, så inspiserer vi kolonnen `Parental_Involvement` i datasettet vårt `df`.

Python-kode for kakediagram

```
df["Parental_Involvement"].unique()
array(['Low', 'Medium', 'High'], dtype=object)
```

```
array(['Low', 'Medium', 'High'])
```

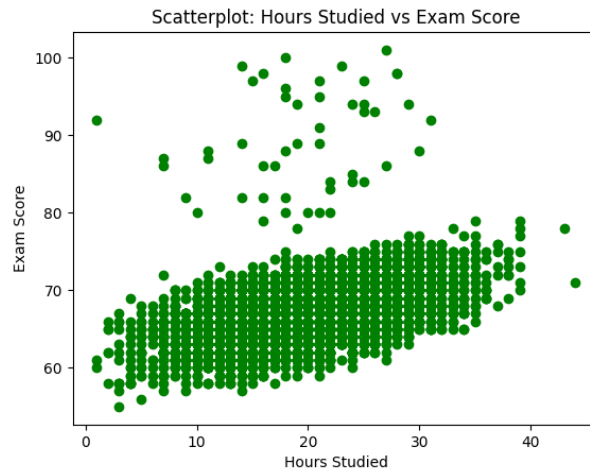
Med andre ord, det er en **kategorisk** variabel, med tre ulike verdier. Et kakediagram (pie chart) viser hvordan foreldreinvolvering er fordelt blant studentene. Diagrammet kan gi en rask oversikt over hvor mange som har lav, medium eller høy involvering (Figur 24).

Python-kode for kakediagram

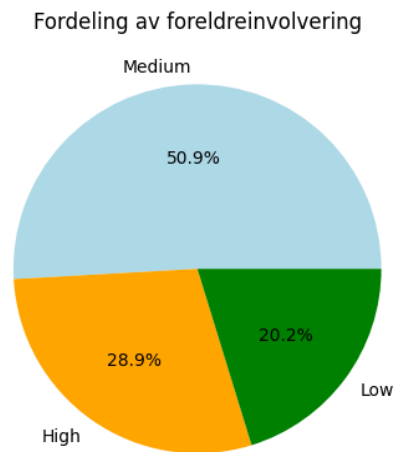
```
plt.pie(parent_counts, labels=parent_counts.index, autopct="%1.1f%%",
 colors=["lightblue", "orange", "green"])
plt.title("Fordeling av foreldreinvolvering")
plt.show()
```

### Økonomisk anvendelse: tilbud og etterspørsel

Tilbudet øker gjerne med prisen, mens etterspørselen reduseres når prisen stiger. Vi definerer funksjoner og finner likevektspunktet visuelt.



Figur 23: Histogram som viser hvor mye studentene studerer.



Figur 24: Kakediagram som viser fordelingen av foreldreinvolvering blant studentene.

Definerer funksjoner

```
def supply(x):
 """Tilbudskurve: pris øker med kvadratet av antall enheter"""
 return (x**2) * (1/250)

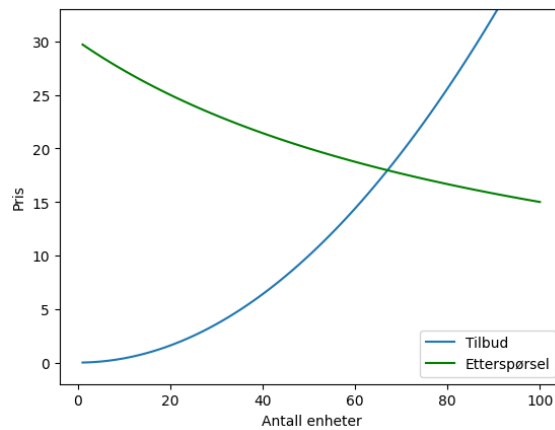
def demand(x):
 """Etterspørselskurve: pris reduseres med økende antall enheter"""
 return 3000 / (100 + x)
```

Vi kan visualisere disse funksjonene for å se hvor de krysser hverandre (likevekten):

## Visualisering av tilbud- og etterspørsel

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(1, 100, 100) # np.linspace(fra, til, antall tall)
plt.plot(x, supply(x), label='Tilbud')
plt.plot(x, demand(x), color='green', label='Etterspørsel')
plt.plot(67, supply(67), marker='o', color='red', label='Likevekt')
plt.legend(loc='lower right')
plt.xlabel('Antall enheter')
plt.ylabel('Pris')
plt.ylim(-2, 33)
plt.show()
```



Figur 25: Tilbud- og etterspørselskurven.

Som vi ser i Figur 25 så er likevekten (altså, hvor linjene krysses) rundt antall enheter er lik 67, og til en pris rundt 18.

Hvordan vi løser dette, helt nøyaktig, skal vi lære om senere i kurset ved hjelp av pakken `SymPy`. Men, vi kan likevel ”highlighte” hvor likevekten er ved hjelp av `matplotlib`.

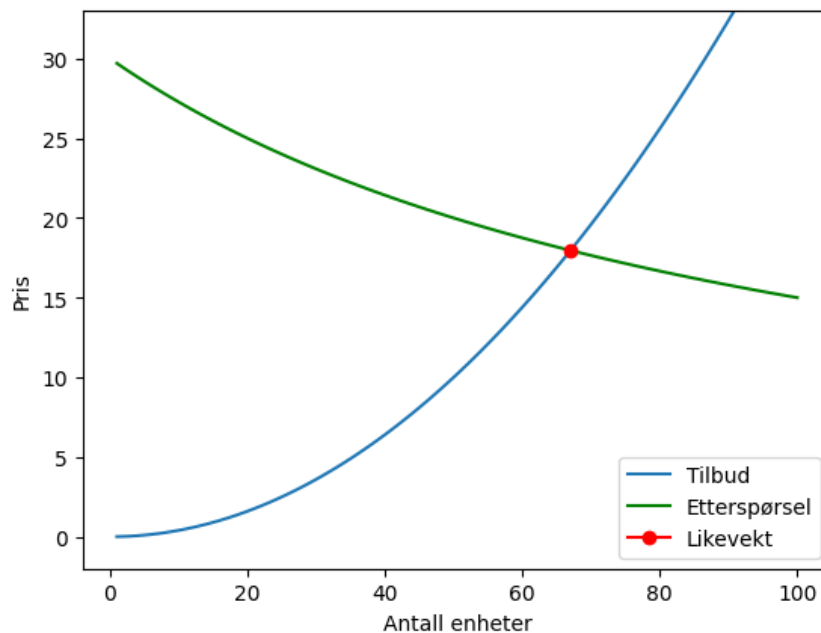
## Visualisering av tilbud- og etterspørsel

```
Plotter tilbudskurven
plt.plot(x, supply(x), label='Tilbud')
Plotter etterspørselskurven
plt.plot(x, demand(x), color='green', label='Etterspørsel')
Plotter likevekten (der det ser ut som den er), rundt 67.
Prøv andre verdier selv!
plt.plot(67, supply(67), marker='o', color='red', label='Likevekt')

Legger til forklaring, i nedre høyre hjørne
plt.legend(loc='lower right')

Setter akselabels
plt.xlabel('Antall enheter')
plt.ylabel('Pris')

Her kan du legge til plt.show() når du kjører koden
plt.show()
```



Figur 26: Tilbud- og etterspørselskurven.

Her ser vi at likevekten ser ut til å være cirka ved **Antall enheter**  $\approx 67$ . Dette gjør det tydelig hvor markedet *balanseres*, og viser hvordan matematikk og økonomi henger sammen på en visuell måte.

## Del 1: Flervalgsoppgaver

1. Hva printer `print(df["Poeng"].mean())` for poeng [10,15,12]?  
(a) 10   b) 12.33   c) 15   d) Feilmelding
2. Hvilken funksjon lager et linjediagram?  
(a) `plt.bar()`   b) `plt.plot()`   c) `plt.scatter()`   d) `plt.hist()`
3. Hva gjør `plt.scatter(x, y); plt.show()`?  
(a) Linjediagram   b) Scatterplot   c) Histogram   d) Kakediagram
4. Hva gir `np.arange(0, 10, 2)`?  
(a) [0, 1, ..., 9]   b) [0, 2, 4, 6, 8]   c) [2, 4, 6, 8, 10]   d) Feilmelding
5. Riktig måte å lage kakediagram fra en dictionary?  
(a) `plt.pie(data.values(), labels=data.keys())`  
(b) `plt.plot(data)`  
(c) `plt.bar(data)`  
(d) `plt.scatter(data)`
6. Hva printer `def dobbel(x): return x*2 - print(dobbel(5))`?  
(a) 5   b) 10   c) 'dobbel'   d) None
7. Hva gjør `plt.bar(range(len(verdier)), verdier)`?  
(a) Linjediagram   b) Stolpediagram   c) Scatterplot   d) Kakediagram
8. Hva gjør `for f in frukter: print(f.upper())`?  
(a) Printer fruktnavnene med store bokstaver  
(b) Teller antall frukter  
(c) Legger til fruktnavn i en liste  
(d) Ingenting

## Del 2: Hva blir output?

1. `plt.plot([1,2,3,4],[2,4,6,8]); plt.title("Ukjent tittel"); plt.show()`
2. `a = np.array([1,2,3]); print(a*3)`
3. `def kvadrat(x): return x**2; print(kvadrat(6))`
4. `plt.hist([5,2,7,10], bins=8, color="purple"); plt.show()`



## Del 3: Sammensatt oppgave (boligpriser Oslo)

Bruk datasettet `Median_totalpris_boliger_Oslo.csv` (SSB):

1. Skriv en funksjon som laster inn CSV-filen med pandas.
2. Lag en funksjon som finner gjennomsnittlig boligpris per år (kolonnevis) og per bydel (radvis).
3. Lag funksjoner som (a) plotter prisutviklingen for en valgt bydel 2018–2023, og (b) lager et stolpediagram som sammenligner alle bydeler i et valgt år.
4. **Ekstra:** Finn bydelen med størst prosentvis vekst 2018–2023, og visualiser den.

## Løsningsforslag

### Del 1: Flervalg

1: b    2: b    3: b    4: b    5: a    6: b    7: b    8: a

### Del 2: Hva blir output?

1: Linjediagram gjennom (1,2),(2,4),(3,6),(4,8) med tittel "Ukjent tittel".    2: [3 6 9]    3: 36    4: Histogram med 8 bins (lilla) av verdiene [5, 2, 7, 10].

## Del 3 - praktiske oppgaver:

```
Oppgave 1
import pandas as pd
def last_data(filbane):
 data = pd.read_csv(filbane)
 return data
df = last_data("data/Median_totalpris_boliger_Oslo.csv")
df.head()

Oppgave 2
Gjennomsnittlig boligpris per år (kolonnevis)
gjennomsnitt_per_år = df.mean(numeric_only=True).to_frame(name="Gjennomsnittpris")
gjennomsnitt_per_år.index.name = "år"

Gjennomsnittlig boligpris per bydel (radvis)
gjennomsnitt_per_bydel = (df.set_index("Geografi")
 .mean(axis=1, numeric_only=True)
 .to_frame(name="Gjennomsnittpris"))
gjennomsnitt_per_bydel.index.name = "Bydel"

Oppgave 3a: prisutvikling for en bydel
import matplotlib.pyplot as plt
def plot_bydel(bydel):
 priser = df[df["Geografi"] == bydel].iloc[0, 1:] # priser for valgt bydel
```

```

priser.index = priser.index.astype(int) # kolonneindeks til int
plt.figure(figsize=(10, 5))
plt.plot(priser.index, priser.values, marker='o')
plt.title(f"Prisutvikling i {bydel}")
plt.xlabel("år")
plt.ylabel("Median totalpris (MNOK)")
plt.grid(True)
plt.show()

plot_bydel("Bydel Gamle Oslo")

Oppgave 3b: stolpediagram for alle bydeler i et valgt år
def plott_bydelspriser(df, år):
 if år not in df.columns:
 raise ValueError(f"året {aar} finnes ikke. Tilgjengelig: {list(df.columns[1:])}")
 data = df[["Geografi", aar]].set_index("Geografi")
 data = data.drop("Oslo i alt", errors="ignore")
 plt.figure(figsize=(12, 6))
 data[aar].plot(kind="bar", color="skyblue", edgecolor="black")
 plt.title(f"Median boligpris per bydel i Oslo ({år})")
 plt.ylabel("Pris (MNOK)")
 plt.xlabel("Bydel")
 plt.xticks(rotation=75, ha="right")
 plt.tight_layout()
 plt.show()

plott_bydelspriser(df, "2023")

```

# Repetisjon: SymPy

## Repetisjon og teori

### Hva er SymPy?

SymPy er et bibliotek i Python for **symbolsk matematikk**. I motsetning til NumPy, som regner numerisk, kan SymPy manipulere matematiske uttrykk nøyaktig slik vi gjør på papir. Det betyr at vi kan løse ligninger, faktorisere uttrykk, derivere og integrere funksjoner, og mye mer. Pakken installeres med `!pip install sympy` (eller `%`) og importeres med `import sympy as sp`.

### Enkelt eksempel

La oss faktorisere ligningen  $x^2 = 9$ :

Python-kode for å faktorisere ligninger

```
import sympy as sp

x = sp.Symbol('x') # Definerer symbolet
solution = sp.solve(sp.Eq(x**2, 9), x) # Løser ligningen $x^2 = 9$
print(solution)
```

Output blir:

$$x = -3 \quad \text{eller} \quad x = 3$$

Dette viser hvordan SymPy kan brukes til å finne **eksakte løsninger** på ligninger.

### Faktorisering og derivasjon

Derrivasjon og faktorisering i Python

```
faktorisering
faktor = sp.factor(x**2 - 9) # $(x - 3)(x + 3)$

f = x**2 - 9
derivasjon
df = sp.diff(f, x) # $2x$
```

Output:

$$x^2 - 9 = (x - 3)(x + 3)$$

Output:

$$\frac{d}{dx}(x^2 - 9) = 2x$$

### Løsning av ligninger

Vi kan kombinere disse metodene for å finne nullpunkter. Det koden under her gjør er;

$$f(x) = 0$$

$$x^2 - 9 = 0$$

og deretter finner vi nullpunktene.

#### Løse ligninger

```
nullpunkter = sp.solve(sp.Eq(f, 0), x)
print(nullpunkter)
```

Output:

$$x = -3 \quad \text{eller} \quad x = 3$$

### Evaluering med subs() og integrasjon

#### Evaluering av funksjoner

```
verdi = f.subs(x, 2)
```

Her bruker vi SymPy funksjonen `subs()` for å erstatte vår `x` med tallet 2. Da får vi output:

$$f(2) = 2^2 - 9 = -5$$

#### Ubestemt og bestemt integral

```
F = sp.integrate(3*x**2, x) # Ubestemt integral: x^3
bestemt = sp.integrate(x**2, (x, 0, 2)) # Bestemt integral fra 0 til 2: 8/3
```

Output:

$$\int 3x^2 dx = x^3$$

Output:

$$\int_0^2 x^2 dx = \frac{8}{3}$$

Dermed kan vi bruke SymPy både til **ubestemte** og **bestemte integraler**, samt integraler over uendelige grenser.

### Hvordan skrive matematiske uttrykk i SymPy

- Multiplikasjon skrives eksplisitt med `*` ( $2x \rightarrow 2*x$ ).
- Potenser med `**` ( $x^2 \rightarrow x**2$ ).
- Brøk med `/` ( $\frac{1}{x} \rightarrow 1/x$ ).
- Funksjoner som  $\sin(x)$  og  $\ln(x)$ : `sp.sin(x)`, `sp.log(x)`.

## Likhet i SymPy

I Python betyr = alltid **tilordning**. For å sette opp matematiske ligninger trenger vi andre verktøy:

- `==` tester **strukturell likhet**, ikke matematisk. F.eks. gir `(x + 1)**2 == x**2 + 2*x + 1` verdien `False`, selv om uttrykkene er matematisk like.
- `sp.Eq(venstre, hoyre)` lager en **symbolsk ligning**.
- `sp.simplify(uttrykk)` tester **matematisk likhet**: trekk det ene fra det andre og forenkle. Gir resultatet 0, er uttrykkene like.

## Symboler og maks/min med førsteordensbetingelser

Vi definerer symboler med `sp.Symbol()` (ett) eller `sp.symbols()` (ett eller flere). For å finne maksimum/minimum deriverer vi og setter den deriverte lik null, og sjekker fortegnet til den andrederiverte:

Finner kritiske verdier

```
f = -x**2 + 4*x
f_prime = sp.diff(f, x)
critical_points = sp.solve(sp.Eq(f_prime, 0), x)
print(critical_points)
```

```
> 2
```

Her finner vi det kritiske punktet:

$$f'(x) = -2x + 4 = 0 \quad \Rightarrow \quad x = 2$$

For å sjekke om dette er et maksimum eller minimum, kan vi se på den andrederiverte:

Andrederiverte

```
f_doubleprime = sp.diff(f_prime, x)
print(f_doubleprime.subs(x, 2)) # -2 < 0, altså et maksimum
```

```
> -2
```

Her får vi  $f''(2) = -2 < 0$ , altså et maksimum.

Nå kan vi visualisere funksjonen, for å se om dette stemmer. Da bruker vi det vi har lært om `matplotlib`.

### Visualiserer $f(x)$

```
import numpy as np

x = np.linspace(-10, 10, 100)

def f_x(x):
 return -x**2 + 4*x

Tegner funksjonen
plt.plot(x, f_x(x), color='blue', label="$f(x) = -x^2 + 4x$")

Highlighter punktet (2, f(2))
plt.scatter(2, f_x(2), color='red', s=50, zorder=5)

Aksetitler
plt.xlabel("x-akse")
plt.ylabel("f(x)")

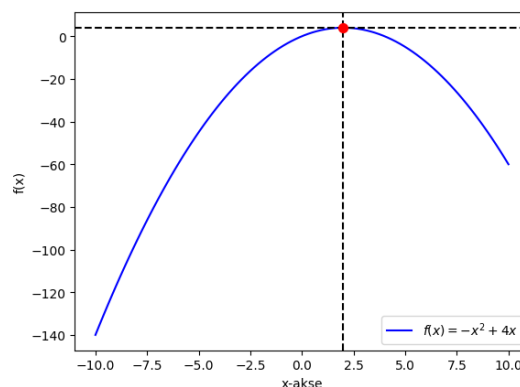
Horizontal linje i y=f(2)
plt.axhline(y=f_x(2), color='black', linestyle='--')

Vertikal linje i x=2
plt.axvline(x=2, color='black', linestyle='--')

plt.legend()

plt.show()
```

Koden resulterer i plottet fra Figur 27.



Figur 27: Plott av funksjonen  $f(x) = -x^2 + 4x$

Hvor det ser ut til å stemme at punktet  $x = 2$  gir topp-punktet til funksjonen  $f(x)$ .

## Bruk av solve() sammen med subs()

Vi kan kombinere `sp.solve()` og `subs()` for å finne både kritiske punkter og tilhørende funksjonsverdier. Vi ser på funksjonen

$$f(x) = -x^2 + 4x$$

Vi deriverer funksjonen og setter den deriverte lik null for å finne det kritiske punktet.

### Kritisk punkt og funksjonsverdi

```
Finn kritisk punkt for f(x) = -x**2 + 4*x

f = -x**2 + 4*x

f_prime = sp.diff(f, x)
critical_point = sp.solve(sp.Eq(f_prime, 0), x)

Setter inn i f(x) for å få funksjonsverdien

max_value = f.subs(x, critical_point[0])

print("Kritisk punkt:", critical_point[0])
print("Funksjonsverdi:", max_value)
```

```
> Kritisk punkt: 2
> Funksjonsverdi: 4
```

Dermed finner vi at funksjonen har sitt maksimum ved

$$x^* = 2, \quad f(x^*) = 4$$

**NB:** Legg merke til at eksempelet over finner **ett kritisk punkt**. Dette skyldes at vi henter ut det første elementet i listen med

### Henter første element

```
max_value = f.subs(x, critical_point[0])
```

Her betyr indeksen `[0]` at vi velger det første elementet i listen.

Men hva om funksjonen har flere kritiske punkter? Betrakt for eksempel funksjonen

$$f(x) = x^3 - 3x$$

som har to kritiske punkter, nemlig  $(x = -1)$  og  $(x = 1)$ .

Vi kan finne alle kritiske punkter og evaluere funksjonen i hvert av dem ved hjelp av en `for`-løkke.

### Flere kritiske punkter

```
import sympy as sp

x = sp.Symbol('x')

Eksempel: $f(x) = x^3 - 3x$

f = x**3 - 3*x

Deriver funksjonen

f_prime = sp.diff(f, x)

Finn alle kritiske punkt

critical_points = sp.solve(sp.Eq(f_prime, 0), x)

print("Kritiske punkt:", critical_points)

Evaluer funksjonen i hvert kritisk punkt

for cp in critical_points:
 value = f.subs(x, cp)
 print(f"f({cp}) = {value}")

> Kritiske punkt: [-1, 1]
> f(-1) = 2
> f(1) = -2
```

Vi ser dermed at funksjonen har de kritiske punktene

$$[(-1, 2) \quad \text{og} \quad (1, -2)]$$

Disse punktene kan visualiseres ved hjelp av NumPy og matplotlib.



### Visualisering av kritiske punkter

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-3, 3, 500)
y = x**3 - 3*x

plt.plot(x, y, label="$f(x)=x^3-3x$")

Markerer kritiske punkt

crit_points = [(-1, 2), (1, -2)]

for cp in crit_points:
 plt.scatter(cp[0], cp[1], s=50, zorder=5)
 plt.text(cp[0], cp[1] + 0.5,
 f"({cp[0]}, {cp[1]})",
 ha="center")

plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Plot av funksjonen $f(x)=x^3-3x$")

plt.axhline(0, linewidth=0.8)
plt.axvline(0, linewidth=0.8)

plt.legend()
plt.show()
```

Koden resulterer i plottet fra Figur 28, hvor de kritiske punktene er markert.

### Kjerneregelen med SymPy

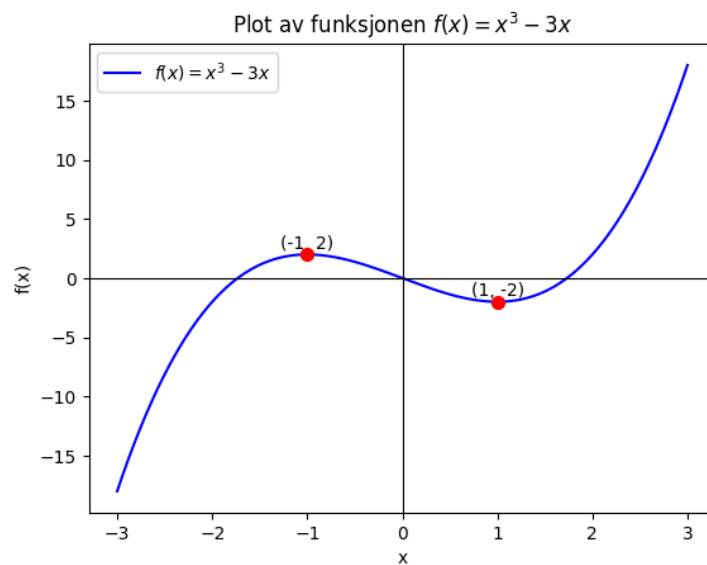
Et nyttig triks i SymPy er å lage egne funksjoner som implementerer kjente regler fra kalkulus. Et eksempel er **kjerneregelen** (chain rule), som sier at dersom

$$h(x) = f(g(x))$$

så er den deriverte gitt ved

$$h'(x) = f'(g(x)) \cdot g'(x)$$

Vi kan implementere denne regelen i SymPy på følgende måte:



Figur 28: Plott av funksjonen  $f(x) = x^3 - 3x$ , med sine kritiske punkter.

#### Implementasjon av kjerneregelen

```
def f_deriv_chain(f, g):
 df = sp.diff(f) # Deriverer f
 dg = sp.diff(g) # Deriverer g

 return df.subs({'y': g}) * dg
```

Her antar vi at funksjonen ( $f$ ) er skrevet som en funksjon av ( $y$ ), mens ( $g$ ) er en funksjon av ( $x$ ).

La oss undersøke eksemplet

$$h(x) = (x^3)^2$$

#### Eksempel på kjerneregelen

```
x, y = sp.symbols('x y')

f = y**2
g = x**3

result = f_deriv_chain(f, g)

print(result)

> 6*x**5
```

Resultatet tilsvarer

$$h'(x) = 2(x^3) \cdot 3x^2 = 6x^5$$

som er den korrekte deriverte av funksjonen

$$h(x) = x^6$$

### Liten oppsummering av kjekke SymPy-funksjoner

1. `sp.symbols()` definerer symboler
2. `sp.Eq()` lager ligninger
3. `sp.solve()` løser dem
4. `sp.diff()` deriverer
5. `sp.integrate()` integrerer
6. `.subs()` evaluerer ved å sette inn verdier
7. `sp.simplify()` tester matematisk likhet

## Del 1: Flervalgsoppgaver

1. Hva gjør `sp.diff(-x**2 + 4*x, x)`?
  - (a) Printer  $-x^2 + 4x$
  - (b) Printer  $-2x + 4$
  - (c) Printer  $f(2)$
  - (d) Error
2. Hvilken kommando setter opp en ligning?
  - (a) `sp.Eq()`
  - (b) `sp.solve()`
  - (c) `sp.subs()`
  - (d) `sp.Symbol()`
3. `sp.solve(sp.Eq(100-2*x, 20+3*x), x)` gjør hva?
  - (a) Løser tilbud alene
  - (b) Finner likevektsmengden  $x^*$
  - (c) Finner mengde og pris
  - (d) Feilmelding
4. `(x**2).subs(x, 3)` gir?
  - (a) Symbolet  $x$
  - (b) 9
  - (c) 3
  - (d) Feilmelding
5. Riktig prosedyre for maks/min?
  - (a) Sett den andrederiverte lik null
  - (b) Sett funksjonen lik null
  - (c) Sett den førstederiverte lik null og sjekk fortegnet til den andrederiverte
  - (d) Sett inn tall tilfeldig

## Del 2: Hva blir output?

1. `sp.solve(sp.Eq(100-2*x, 20+3*x), x)`
2. `f = -x**2 + 4*x; k = sp.solve(sp.Eq(sp.diff(f,x),0), x); f.subs(x, k[0])`
3. `f = y**2; g = sp.exp(x); sp.diff(f).subs({y:g}) * sp.diff(g)`

## Del 3: SymPy-case (markedslikevekt)

Gitt  $Q_d = 100 - 2p$  og  $Q_s = 20 + 3p$ :

1. Finn likevektspris og -mengde med SymPy.
2. Lag en funksjon som generaliserer: gitt etterspørsel og tilbud, finn likevekten.
3. Visualiser kurvene og marker likevektspunktet med Matplotlib.
4. **Ekstra:** Innfør en skatt på 5 per enhet, endre tilbudsfunksjonen, og finn ny likevekt. Visualiser endringen.

## Løsningsforslag

### Del 1: Flervalg

1: b    2: a    3: b    4: b    5: c

### Del 2: Hva blir output?

1:  $p = 16$ .    2: Kritisk punkt  $x = 2$ , funksjonsverdi  $f(2) = 4$ .    3:  $2e^{2x}$  (kjerneregelen:  $f'(g(x)) \cdot g'(x) = 2e^x \cdot e^x$ ).

### Del 3 - SymPy-case:

## Simulerer ulik antall terningkast

```
Oppgave 1
import sympy as sp
x = sp.Symbol('x')
Qd = 100 - 2*x
Qs = 20 + 3*x
likevekt_pris = sp.solve(sp.Eq(Qd, Qs), x)[0]
likevekt_mengde = Qd.subs(x, likevekt_pris)
print("Pris:", likevekt_pris) # 16
print("Mengde:", likevekt_mengde) # 68

Oppgave 2
def finn_likevekt(Qd, Qs, pris):
 likevekt_pris = sp.solve(sp.Eq(Qd, Qs), pris)[0]
 likevekt_mengde = Qd.subs(pris, likevekt_pris)
 return likevekt_pris, likevekt_mengde

pris = sp.Symbol('p')
Qd = 100 - 2*pris
Qs = 20 + 3*pris
finn_likevekt(Qd, Qs, pris)

Oppgave 3
import matplotlib.pyplot as plt
import numpy as np
p_values = np.linspace(0, 40, 100)
Qd_values = [100 - 2*p for p in p_values]
Qs_values = [20 + 3*p for p in p_values]
plt.plot(p_values, Qd_values, label="Etterspørsel")
plt.plot(p_values, Qs_values, label="Tilbud")
plt.scatter([16], [68], color="red", zorder=5, label="Likevekt")
plt.xlabel("Pris (p)")
plt.ylabel("Mengde (Q)")
plt.legend()
plt.title("Markedslikevekt")
plt.grid(True)
plt.show()

Oppgave 4 (ekstra): skatt på 5 per enhet -> Qs = 20 + 3(p - 5)
Qs_skatt = 20 + 3*(pris - 5)
ny_pris, ny_Q = finn_likevekt(Qd, Qs_skatt, pris)
print("Ny pris:", ny_pris, "Ny mengde:", ny_Q)
```

For å visualisere endringen kan du lage et plott som ligner på oppgave 3, men som også plotter  $Qs\_skatt$ .

# Repetisjon: Simulering

## Repetisjon og teori

### Hva er simulering?

**Simulering** er en metode for å modellere og analysere komplekse prosesser ved å etterligne dem i datamaskinen. I stedet for å løse matematiske modeller eksakt, kan vi bruke simulering til å utforske hvordan et system oppfører seg under ulike betingelser. Med simulering kan vi blant annet generere tilfeldige tall for å modellere usikkerhet, estimere sannsynligheter og forventede utfall, og undersøke økonomiske prosesser som prisutvikling over tid.

### Eksempel: simulering av terningkast

Anta at du vil gjøre  $N$  kast med en terning. Hva blir gjennomsnittsverdien for ulike  $N$ ? Vi starter med pseudokode:

1. Velg hvor mange kast vi skal gjøre ( $N$ )
2. For hvert kast:
  - a. Generer et tilfeldig tall mellom 1 og 6
  - b. Lagre utfallet
3. Regn ut gjennomsnittet av alle kastene
4. Returner gjennomsnittet

#### Simulerer ulik antall terningkast

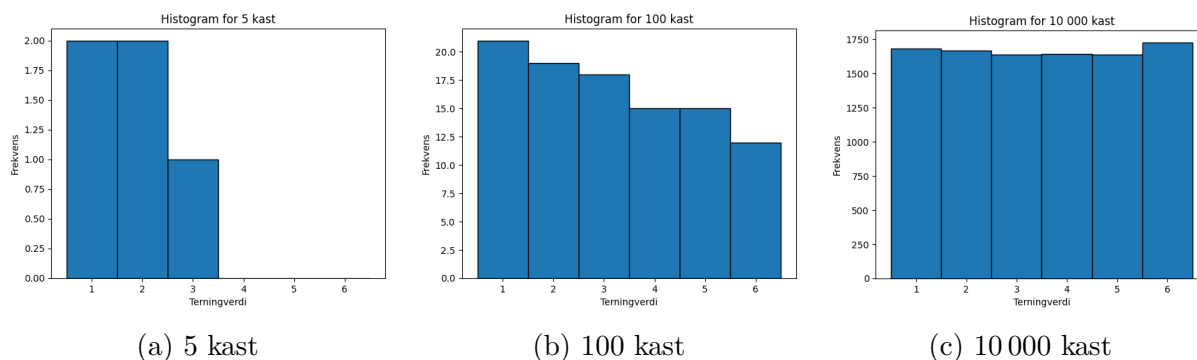
```
import random

def simuler_terningkast(N):
 kast = []
 for i in range(N):
 verdi = random.randint(1, 6) # tilfeldig tall mellom 1 og 6
 kast.append(verdi)
 gjennomsnitt = sum(kast) / len(kast)
 return gjennomsnitt

print("5 kast:", simuler_terningkast(5))
print("100 kast:", simuler_terningkast(100))
print("10 000 kast:", simuler_terningkast(10000))
```

### Observasjoner

- Vi får ulike svar hver gang, fordi `random.randint(a, b)` gir et tilfeldig tall mellom  $a$  og  $b$  (her 1 og 6).
- Med bare 5 kast kan gjennomsnittet variere mye (stor usikkerhet).
- Med 100 kast blir gjennomsnittet mer stabilt.



Figur 29: Histogrammer for henholdsvis 5, 100 og 10 000 terningkast. Vi ser at fordelingen av terningverdiene blir jevnere når antall kast øker.

- Med 10 000 kast nærmer gjennomsnittet seg den teoretiske verdien 3.5.
- Vi **kan** bruke `np.random.randint(1, 6+1)`, men da må vi ha `+1` fordi `numpy`-versjonen **ikke** er inklusiv det siste tallet. Med `random.randint(1, 6)` trenger vi **ikke** `+1`, fordi den **er** inklusiv og tar med tallet 6.

## Visualisering og de store talls lov

Vi kan plote fordelingen med et histogram. Med få kast ser fordelingen skjev og tilfeldig ut, men når vi øker antall kast (f.eks. 10 000), nærmer frekvensen av hver terningverdi seg den teoretiske sannsynligheten  $1/6$ :

### Terningkast simulering

```
import matplotlib.pyplot as plt

kast_10000 = simuler_terningkast(10000) # (her som liste av kast)
plt.hist(kast_10000, bins=range(1, 8), edgecolor="black", align="left")
plt.xticks(range(1, 7))
plt.title("Histogram for 10000 kast")
plt.xlabel("Terningverdi")
plt.ylabel("Frekvens")
plt.show()
```

Dette illustrerer **loven om store tall** i praksis (se Figur 29): selv om enkeltkast er helt tilfeldige, får vi med et stort antall kast et stabilt og forutsigbart mønster. Det er nettopp dette Monte Carlo-simuleringer bygger på – ved å generere mange tilfeldige scenarioer kan vi estimere forventede verdier og sannsynligheter.

### Replikerbarhet med `seed()`

Datamaskiner lager ikke *ekte* tilfeldige tall, bare tall som ser tilfeldige ut. Vil du ha nøyaktig de samme "tilfeldige" tallene hver gang (nyttig ved feilsøking eller eksamen), bruk `np.random.seed(et_tall)`.

## Del 1: Flervalgsoppgaver

1. Hva menes med simulering her?
  - (a) Tegne grafer
  - (b) Etterligne virkelige prosesser på datamaskinen med tilfeldige tall
  - (c) Beregne eksakte løsninger
  - (d) Løse ligninger symbolsk
2. Hvilken funksjon gir tilfeldige tall mellom 1 og 6 (inkludert)?
  - (a) `np.random.choice(6)`
  - (b) `random.random()`
  - (c) `random.randint(1, 6)`
  - (d) `random.uniform(1, 6)`
3. Hva skjer når antall kast øker fra 5 til 10 000?
  - (a) Mer tilfeldige resultater
  - (b) Færre ulike verdier
  - (c) Mer skjevt histogram
  - (d) Frekvensene jevner seg ut mot  $1/6$
4. Teoretisk forventningsverdi for en rettferdig terning?
  - (a) 3
  - (b) 3.5
  - (c) 4
  - (d) 6
5. Hvorfor ikke samme resultat hver kjøring?
  - (a) Koden er feil
  - (b) Python lagrer tall midlertidig
  - (c) `random.randint()` genererer tilfeldige tall
  - (d) Løkken itererer feil
6. Hva illustrerer ”loven om store tall”?
  - (a) Tilfeldige tall blir mer uforutsigbare
  - (b) Summen er alltid konstant
  - (c) Gjennomsnittet av mange utfall nærmer seg forventningsverdien
  - (d) Sannsynligheten for alle utfall øker
7. Hva viser histogrammet med 10 000 kast best?
  - (a) 6 kommer sjeldnere enn 1
  - (b) Terningen er urettferdig
  - (c) Alle verdier forekommer omtrent like ofte
  - (d) Tilfeldigheter forsvinner helt
8. Forskjellen på `random.randint(1,6)` og `np.random.randint(1,6)`?



- (a) Ingen forskjell
  - (b) Begge ekskluderer 6
  - (c) `random` gir flyttall, `np` gir heltall
  - (d) `random.randint(1,6)` inkluderer 6, `np.random.randint(1,6)` ekskluderer 6
9. Fordel med simulering vs. eksakte beregninger?
- (a) Alltid mer presis
  - (b) Kan modellere komplekse prosesser uten enkle matematiske løsninger
  - (c) Krever ingen programmering
  - (d) Fjerner tilfeldigheter
10. Hvordan henger Monte Carlo sammen med terningkast?
- (a) Brukes bare til spill
  - (b) Beregner kun sannsynlighet for 6-ere
  - (c) Bruker mange tilfeldige utfall for å estimere et forventet resultat
  - (d) Eliminere behovet for tilfeldige tall

## Del 2: Forklar koden

### 1. Myntkast:

```
import random
def simuler_myntkast(N):
 resultat = []
 for i in range(N):
 kast = random.choice(["Kron", "Mynt"])
 resultat.append(kast)
 return resultat.count("Kron") / N
print(simuler_myntkast(1000))
```

Forklar hva koden gjør og hva tallet representerer. Hva forventer du dersom du øker antall kast fra 10 til 10 000?

### 2. Køsystem:

```
import random
def simuler_ko(antall_kunder):
 ventetider = []
 for i in range(antall_kunder):
 service_tid = random.uniform(2, 8) # minutter per kunde
 ventetider.append(service_tid)
 return sum(ventetider) / len(ventetider)
print("Gjennomsnittlig ventetid:", simuler_ko(500))
```

Forklar hva koden simulerer og hvordan tilfeldighetene brukes. Hvordan kan vi bruke simulering til å forbedre køsystemet?

### Del 3: Praktisk oppgave (aksjepris over tid)

En aksje starter på 100 kroner. Den daglige prosentvise endringen er tilfeldig og følger en normalfordeling med forventning 0 og standardavvik 1%.

- Bruk `np.random.normal(0, 0.01, antall_dager)` til å generere prosentvise endringer for 252 børsdager.
- Simuler prisutviklingen dag for dag ( $P_{t+1} = P_t \cdot (1 + r_t)$ ) og lagre resultatene.
- Visualiser prisutviklingen med Matplotlib.
- **Utvidelse:** Kjør for 10 ulike utfall og plott alle på samme figur.

**Refleksjon:** Hva skjer med variasjonen når du øker antall dager? Hva representerer de ulike prisbanene? Hvordan kan en slik simulering være nyttig for økonomer eller investorer?

### Løsningsforslag

#### Del 1: Flervalg

1: b    2: c    3: d    4: b    5: c    6: c    7: c    8: d    9: b    10: c

#### Del 2 - forklaring av kode:

1. Funksjonen simulerer  $N$  myntkast: for hvert kast velges tilfeldig mellom "Kron" og "Mynt" med `random.choice()`, og funksjonen returnerer *andelen* "Kron" (et tall mellom 0 og 1). For små  $N$  varierer andelen mye; for store  $N$  (10 000) nærmer den seg 0.5 – nok en gang loven om store tall.
2. Funksjonen simulerer ventetiden til  $N$  kunder. Hver kunde får en tilfeldig servicetid mellom 2 og 8 minutter (`random.uniform`), og funksjonen returnerer gjennomsnittlig ventetid. Med flere kunder blir estimatet mer stabilt. Dette er en enkel Monte Carlo-simulering som kan brukes til å modellere og forbedre køsystemer.

#### Del 3 - praktisk oppgave (ett, av mange, mulig løsningsforslag):

##### Løsningsforslag (fort. neste side)

```
Simulering av aksjeprisbaner
import numpy as np
import matplotlib.pyplot as plt

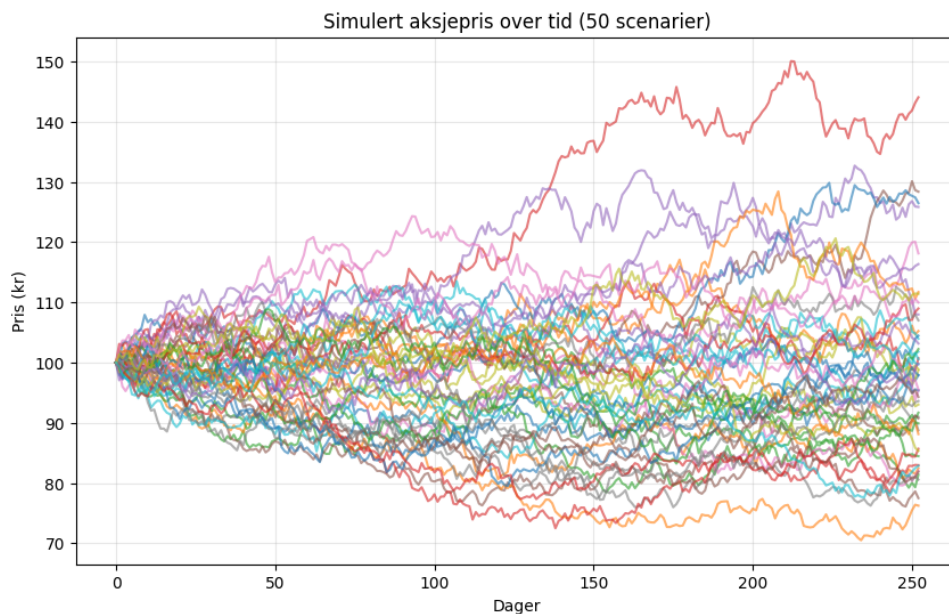
start_pris = 100.0 # startpris i kroner
antall_dager = 252 # handelsdager i ett år
antall_simuleringer = 50 # hvor mange prisbaner vi simulerer
sigma = 0.01 # daglig standardavvik (1%)
mu = 0.0 # daglig forventet endring (0%)
np.random.seed(42) # for replikerbarhet (kan fjernes)
```

### Løsningsforslag

```
alle_baner = [] # lagrer alle prisbanene
for sim in range(antall_simuleringer):
 priser = [start_pris]
 for dag in range(antall_dager):
 endring = np.random.normal(mu, sigma) # tilfeldig prosentvis endring
 ny_pris = priser[-1] * (1 + endring)
 priser.append(ny_pris)
 alle_baner.append(priser)

plt.figure(figsize=(10, 6))
for i in range(antall_simuleringer):
 plt.plot(alle_baner[i], alpha=0.6)
plt.title(f"Simulert aksjepris over tid ({antall_simuleringer} scenarier)")
plt.xlabel("Dager")
plt.ylabel("Pris (kr)")
plt.grid(True, alpha=0.3)
plt.show()
```

Her er det en for-løkke inne i en annen for-løkke: den ytre løkken lager hver enkelt prisbane (scenario), mens den indre løkken regner ut prisen dag for dag innenfor ett scenario. Øker du `antall_dager`, sprer banene seg mer utover (større usikkerhet jo lenger frem i tid), se Figur 30.



Figur 30: Viser 50 ulike scenarier vha simulering.

## Referanser

- [1] Yves Hilpisch. *Python for Finance: Mastering Data-Driven Finance*. O'Reilly Media, Inc., 2nd edition, 2018.
- [2] Wes McKinney. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. O'Reilly Media, Inc., 3rd edition, 2022.
- [3] Thomas J. Sargent and John Stachurski. Quantitative economics with python. <https://quantecon.org/python-lectures/>, 2024. Tilgjengelig online.